University of the Philippines Cebu

Data Structures

---

# Second Long Examination

## Lecture Notes, Practice Examination and Worked Solutions

---

*Array Lists and Vectors — Singly and Doubly Linked Lists*
*Stacks — Queues — Deques*

Compiled from the course lecture notes, slides and sample programs
of the Second Long Examination coverage

D. Juevesano

November 2026

Every program in this volume was compiled and executed;
every printed output is the output of the program shown.

# Preface

This volume prepares the reader for the Second Long Examination of the course on data structures. The coverage of the examination is: *lists* (the array list, the growable array or vector, the singly linked list and the doubly linked list), *stacks*, *queues* and *deques*. Trees are not part of it. The volume has three parts.

**Part I** is a set of consolidated lecture notes. It merges the lecture handouts on abstract data types, array-based lists and linked lists, the slide deck on the List ADT, the sample program `arraylist.c` and the linked-list reviewer into one continuous exposition with definitions, theorems and proofs. Chapters 4, 5 and 6 treat stacks, queues and deques in full: their array and linked implementations, the classical applications, and the two techniques behind most examination problems on them, the *monotonic stack* and the *monotonic deque*.

**Part II** is a practice examination written in the manner of the departmental examination: short conceptual questions, *tracing problems* that carry a twist, and *problem-solving* items in the style of an online judge (LeetCode, HackerRank), each with starter code, test cases, the permitted operations, a required time complexity and, where it applies, a bonus for a faster solution. **Part III** contains complete solutions with explanations, diagrams and the usual mistakes.

***The rule of abstraction.*** Every problem states the *permitted operations*. A solution that reaches into the representation of an ADT — for instance, indexing the array inside a stack when only push and pop are permitted — forfeits the point of the exercise and, in an examination, usually the credit. Where a problem asks for a new operation, the solution implements it *on top of* the given interface or, when the problem says so, directly on the representation (for example, relinking nodes).

***Verification.*** Every listing in this book is included from a source file that was compiled with `gcc`/`g++` (with all warnings enabled) and run. The outputs printed in the tracing problems and in Part III are the real outputs of these programs. Every solution of Part III was additionally run against its sample tests and, wherever a simple reference implementation exists, against that reference on thousands of random inputs; for the linked structures the tests also count the heap blocks, so that a leaked or doubly freed node is detected. The two departmental implementation problems (*eraseItems* and *appendList*) reproduce their sample outputs character for character.

***Conventions.*** Positions in the lecture-slide array list are 0-based; the sample program `arraylist.c` uses *1-based* positions in `itemAt` and `setItem`. Always read the problem statement for the convention in force. Throughout, $n$ denotes the number of items. The code is plain C (and a little C++): no global variables, no `static` and no `const`, as in the course.

***Sources.*** The chapter structure follows P. Morin, *Open Data Structures* (ArrayStack, ArrayQueue, ArrayDeque, SLList, DLList), which the handouts adapt; the problem bank is adapted from LeetCode and from the classical exercises of the cited textbooks. All errors in this volume are the compiler's, or mine.

Juevesano D. 2026 All rights reserved. 

# Contents

**Preface** **i**

**I Lecture Notes** **1**

**1 Foundations** **2**
1.1 Abstract data types . . . 2
1.2 Asymptotic analysis . . . 2
1.3 Pointers, memory and parameter passing . . . 3
1.4 Reasoning about correctness . . . 4

**2 Array-Based Lists** **6**
2.1 Arrays and their shortcomings . . . 6
2.2 The array list . . . 6
2.3 Dynamic arrays and the vector . . . 8
2.4 Sorted array lists and binary search . . . 10
2.5 Index techniques for array problems . . . 10

**3 Linked Lists** **12**
3.1 The singly linked list . . . 12
3.1.1 Insertion and removal at the ends . . . 13
3.1.2 Case analysis: draw before you code . . . 14
3.2 The doubly linked list . . . 15
3.2.1 Sentinel (dummy) nodes . . . 16
3.3 Circular linked lists . . . 18
3.4 Programming patterns . . . 19
3.5 Arrays versus linked lists . . . 20

**4 Stacks** **21**
4.1 The Stack ADT . . . 21
4.2 Implementations . . . 21
4.3 Brackets and expressions . . . 22
4.3.1 Balanced brackets . . . 22
4.3.2 Postfix notation . . . 23
4.4 The monotonic stack . . . 25
4.5 Augmented stacks . . . 27

Juevesano D. 2026 All rights reserved. 

4.6 Stack permutations . . . 27
4.7 Stacks and recursion . . . 27

**5 Queues** **29**
5.1 The Queue ADT . . . 29
5.2 Array implementations . . . 29
5.3 Linked implementations . . . 30
5.4 Queues built from stacks, and stacks from queues . . . 31
5.5 Applications . . . 32
5.5.1 Rotation and the Josephus problem . . . 32
5.5.2 Sliding windows over a stream . . . 33
5.5.3 Combining a queue with a stack . . . 33

**6 Deques** **35**
6.1 The Deque ADT . . . 35
6.2 The circular-array deque . . . 35
6.3 The ArrayDeque: positional access in a circular array . . . 36
6.4 The linked deque . . . 38
6.5 The monotonic deque . . . 40
6.6 Further applications . . . 41
6.7 The standard library . . . 42

**II Practice Examination** **43**

**7 Instructions and Marking Scheme** **44**
7.1 How to use this part . . . 44
7.2 Conventions used in the problem statements . . . 44
7.3 A sample marking scheme . . . 45

**8 Section A: Concept Questions** **46**

**9 Section B: Tracing Problems** **50**
Tracing 1. Array list, return values and stale cells . . . 50
Tracing 2. What does the callee really change? . . . 51
Tracing 3. Counting loop iterations . . . 52
Tracing 4. The raw array after erase and insert . . . 53
Tracing 5. Circular queue: wrap-around . . . 54
Tracing 6. SLL with head, tail and size . . . 55
Tracing 7. One star or two? . . . 55
Tracing 8. Erasing from a doubly linked list . . . 56
Tracing 9. Recursion on a list . . . 56
Tracing 10. Stacks and queues in the same function . . . 57

Juevesano D. 2026 All rights reserved. 

Tracing 11. Infix to postfix with a depth counter . . . 58
Tracing 12. A queue from two stacks . . . 59
Tracing 13. Which operand is popped first? . . . 60
Tracing 14. Stack permutations . . . 60
Tracing 15. The stock spanner . . . 61
Tracing 16. The mid-pointer list . . . 62
Tracing 17. Binary search: counting the steps . . . 62
Tracing 18. Josephus with a twist . . . 63
Tracing 19. Splicing lists . . . 63
Tracing 20. Shallow and deep copies . . . 64
Tracing 21. Circular deque: the raw array . . . 64
Tracing 22. ArrayDeque: which side moves? . . . 65
Tracing 23. Sliding window maximum . . . 66
Tracing 24. A circular list as a queue . . . 67
Tracing 25. The sentinel deque, and a struct copy . . . 67

**10 Section C: Problem Solving** **69**
10.1 Arrays and array lists . . . 69
Problem 1. String compression (run-length encoding) . . . 69
Problem 2. Matrix transpose . . . 70
Problem 3. The missing number . . . 70
Problem 4. A sorted array list . . . 71
Problem 5. Rotate an array . . . 71
Problem 6. Merge two sorted arrays . . . 72
Problem 7. Remove duplicates from a sorted array . . . 72
Problem 8. Two-sum in a sorted array . . . 72
Problem 9. Maximum subarray sum . . . 73
Problem 10. Move zeroes . . . 73
Problem 11. Majority element . . . 73
Problem 12. Rotate a matrix by 90 degrees . . . 74
10.2 Linked lists . . . 74
Problem 13. Implementation: eraseItems (DLL) . . . 74
Problem 14. Implementation: appendList (copying a list) . . . 76
Problem 15. Implementation: recursive append with a double pointer . . . 77
Problem 16. Reverse a singly linked list . . . 77
Problem 17. Middle node and $k$-th node from the end . . . 78
Problem 18. Splicing singly linked lists . . . 78
Problem 19. Merge two sorted doubly linked lists . . . 79
Problem 20. Palindromic doubly linked list . . . 79
Problem 21. Reverse a doubly linked list . . . 79
Problem 22. Remove duplicates from a sorted doubly linked list . . . 80

Juevesano D. 2026 All rights reserved. 

Problem 23. Insert into a sorted doubly linked list . . . 80
Problem 24. An SLL with a middle pointer . . . 81
Problem 25. Rotate a linked list . . . 81
Problem 26. Partition a linked list around a value . . . 82
Problem 27. Remove the $n$-th node from the end . . . 82
Problem 28. Swap nodes in pairs . . . 83
Problem 29. Cycle detection . . . 83
Problem 30. Group odd and even positions . . . 83
Problem 31. Add two numbers stored in linked lists . . . 84
10.3 Stacks . . . 84
Problem 32. Valid brackets . . . 84
Problem 33. Infix to postfix and postfix evaluation . . . 85
Problem 34. A stack with constant-time minimum . . . 85
Problem 35. Next greater element . . . 86
Problem 36. Daily temperatures . . . 86
Problem 37. Online stock span . . . 86
Problem 38. Asteroid collision . . . 87
Problem 39. Decode string . . . 87
Problem 40. Remove adjacent duplicates . . . 88
Problem 41. Sort a stack . . . 88
Problem 42. Validate stack sequences . . . 88
10.4 Queues . . . 89
Problem 43. A queue from two stacks . . . 89
Problem 44. A stack from a queue . . . 89
Problem 45. The Josephus problem . . . 90
Problem 46. Reverse the first $k$ items of a queue . . . 90
Problem 47. Number of recent calls . . . 90
Problem 48. Interleave the two halves of a queue . . . 91
Problem 49. First negative number in every window . . . 91
10.5 Deques . . . 91
Problem 50. Design a circular deque . . . 92
Problem 51. A linked deque with a sentinel . . . 93
Problem 52. Valid palindrome . . . 93
Problem 53. Sliding window maximum . . . 94
Problem 54. Longest subarray with absolute difference at most a limit . . . 94
Problem 55. Front-middle-back queue . . . 94
Problem 56. ArrayDeque: positional insertion and removal . . . 95
Problem 57. Jump game VI . . . 95

Juevesano D. 2026 All rights reserved. 

**III Solutions and Explanations** **96**

**11 Solutions to Section A** **97**

**12 Solutions to Section B** **102**
Solution to Tracing 1 . . . 102
Solution to Tracing 2 . . . 102
Solution to Tracing 3 . . . 103
Solution to Tracing 4 . . . 103
Solution to Tracing 5 . . . 103
Solution to Tracing 6 . . . 104
Solution to Tracing 7 . . . 104
Solution to Tracing 8 . . . 104
Solution to Tracing 9 . . . 105
Solution to Tracing 10 . . . 105
Solution to Tracing 11 . . . 105
Solution to Tracing 12 . . . 105
Solution to Tracing 13 . . . 106
Solution to Tracing 14 . . . 106
Solution to Tracing 15 . . . 106
Solution to Tracing 16 . . . 107
Solution to Tracing 17 . . . 107
Solution to Tracing 18 . . . 108
Solution to Tracing 19 . . . 108
Solution to Tracing 20 . . . 108
Solution to Tracing 21 . . . 109
Solution to Tracing 22 . . . 109
Solution to Tracing 23 . . . 109
Solution to Tracing 24 . . . 110
Solution to Tracing 25 . . . 110

**13 Solutions to Section C** **111**
13.1 Arrays and array lists . . . 111
Solution to Problem 1 . . . 111
Solution to Problem 2 . . . 113
Solution to Problem 3 . . . 114
Solution to Problem 4 . . . 114
Solution to Problem 5 . . . 116
Solution to Problem 6 . . . 116
Solution to Problem 7 . . . 117
Solution to Problem 8 . . . 118

Juevesano D. 2026 All rights reserved. 

Solution to Problem 9 . . . 118
Solution to Problem 10 . . . 119
Solution to Problem 11 . . . 119
Solution to Problem 12 . . . 120
13.2 Linked lists . . . 121
Solution to Problem 13 . . . 121
Solution to Problem 14 . . . 122
Solution to Problem 15 . . . 123
Solution to Problem 16 . . . 124
Solution to Problem 17 . . . 125
Solution to Problem 18 . . . 126
Solution to Problem 19 . . . 126
Solution to Problem 20 . . . 127
Solution to Problem 21 . . . 128
Solution to Problem 22 . . . 128
Solution to Problem 23 . . . 129
Solution to Problem 24 . . . 129
Solution to Problem 25 . . . 132
Solution to Problem 26 . . . 132
Solution to Problem 27 . . . 133
Solution to Problem 28 . . . 134
Solution to Problem 29 . . . 134
Solution to Problem 30 . . . 135
Solution to Problem 31 . . . 136
13.3 Stacks . . . 136
Solution to Problem 32 . . . 136
Solution to Problem 33 . . . 137
Solution to Problem 34 . . . 137
Solution to Problem 35 . . . 138
Solution to Problem 36 . . . 138
Solution to Problem 37 . . . 139
Solution to Problem 38 . . . 139
Solution to Problem 39 . . . 140
Solution to Problem 40 . . . 141
Solution to Problem 41 . . . 142
Solution to Problem 42 . . . 142
13.4 Queues . . . 143
Solution to Problem 43 . . . 143
Solution to Problem 44 . . . 143
Solution to Problem 45 . . . 144

Juevesano D. 2026 All rights reserved. 

Solution to Problem 46 . . . 144
Solution to Problem 47 . . . 145
Solution to Problem 48 . . . 146
Solution to Problem 49 . . . 146
13.5 Deques . . . 147
Solution to Problem 50 . . . 147
Solution to Problem 51 . . . 149
Solution to Problem 52 . . . 149
Solution to Problem 53 . . . 150
Solution to Problem 54 . . . 150
Solution to Problem 55 . . . 151
Solution to Problem 56 . . . 152
Solution to Problem 57 . . . 153

**A Complexity Summary** **154**

**B Pitfall Checklist** **156**

**C STL Quick Reference** **157**

**Bibliography** **158**

Juevesano D. 2026 All rights reserved. 

Part I

# Lecture Notes

Juevesano D. 2026 All rights reserved.

Chapter 1

# Foundations

## 1.1 Abstract data types

**Definition 1.1** (Abstract data type)**.** An *abstract data type* (ADT) is a mathematical model consisting of a set of values together with a set of operations that access or manipulate those values. The ADT specifies *what* the operations do, never *how* they are carried out.

An ADT is therefore a contract. The *interface* lists the operations and their meaning; an *implementation* chooses a concrete representation and supplies the algorithms. One interface may have many implementations with different costs, and the costs — not the results — are what distinguish them. The slides put it as *abstract: the implementation is hidden*.

**Example 1.2.** The *List* ADT is a sequence $x_0, x_1, \ldots, x_{n-1}$ with the operations `size()`, `get(i)`, `set(i,x)`, `add(i,x)` (insert $x$ at position $i$, displacing $x_i, \ldots, x_{n-1}$ by $+1$) and `remove(i)` (delete $x_i$, displacing $x_{i+1}, \ldots, x_{n-1}$ by $-1$). The slides name the same operations `insert`, `push_front`, `push_back`, `erase`, `pop_front`, `pop_back`, `front`, `back`. Two families of implementations are studied: the *array list*, which stores $x_i$ in `a[i]`, and the *linked list*, which stores the items in nodes joined by pointers.

| ADT | Operations (course names) | Discipline |
|---|---|---|
| Stack | `push(x)`, `pop()`, `top()` | last in, first out (LIFO) |
| Queue | `enqueue(x)`, `dequeue()`, `front()` | first in, first out (FIFO) |
| Deque | `push_front`, `push_back`, `pop_front`, `pop_back` | both ends |
| List | `size`, `get(i)`, `set(i,x)`, `add(i,x)`, `remove(i)` | indexed sequence |

**Table 1.1:** The sequence ADTs of this coverage. A stack is a deque restricted to one end; a FIFO queue is a deque restricted to `push_back` and `pop_front`; a deque is a list restricted to positions 0 and $n-1$. Each restriction makes the interface smaller and the implementation easier to make fast.

> The rule of abstraction. When a problem lists the permitted operations, those and only those may be applied to the structure. A client of a stack writes push, pop, top, empty; it never touches the array inside. New operations are allowed *if you implement them* from the permitted ones (or, when a problem explicitly says so, from the representation).

## 1.2 Asymptotic analysis

**Definition 1.3** (Big-Oh, Big-Omega, Big-Theta)**.** Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Then $f(n) = O(g(n))$ if there are constants $c > 0$ and $n_0$ such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$; $f(n) = \Omega(g(n))$ if $f(n) \geq c\, g(n)$ for all $n \geq n_0$ (some $c > 0$); $f(n) = \Theta(g(n))$ if both hold.

Juevesano D. 2026 All rights reserved. 

**Example 1.4** (The thought exercise of the slides)**.**

(i) $23 = O(\log n)$ *is true.* For $n \geq 2$ we have $\lg n \geq 1$, so $23 \leq 23 \lg n$; take $c = 23$, $n_0 = 2$. (Constants are $O(1) \subseteq O(\log n)$: $O$ is an *upper* bound only.)

(ii) $5 \log n = O(1)$ *is false.* Suppose $5 \lg n \leq c$ for all $n \geq n_0$. Choosing $n > \max(n_0, 2^{c/5})$ gives $5 \lg n > c$, a contradiction.

(iii) $5n - 7 = O(n)$ *is true.* For $n \geq 2$, $5n - 7 \leq 5n$; take $c = 5$, $n_0 = 2$ ($n_0 = 2$ is needed only to keep the left side non-negative).

***Calculating running time.*** The running time of a program is estimated by counting how often its dominant statement executes.

- A loop that runs $n$ times around a constant-time body costs $\Theta(n)$; two nested such loops cost $\Theta(n^2)$.
- *Triangular* loops `for(i<n) for(j<=i)` execute the body $1 + 2 + \cdots + n = n(n+1)/2 = \Theta(n^2)$ times.
- A loop whose index is *multiplied or divided* by a constant each round (`i*=2`, `i/=2`) runs $\Theta(\log n)$ times.
- Sequential blocks add; the larger term wins: $n^2 + n \log n = \Theta(n^2)$.
- A recursive procedure that does $\Theta(1)$ work per call and makes one call on $n - 1$ items costs $\Theta(n)$.
- *Amortised* counting: if each item is pushed onto and popped from a stack at most once, a loop of $n$ rounds with an inner `while(pop)` costs $\Theta(n)$ in total, although one round may pop many items (Chapters 4 and 6).

| Class | Name | Typical source |
|---|---|---|
| $\Theta(1)$ | constant | `a[i]`, push on a stack, pointer update |
| $\Theta(\log n)$ | logarithmic | binary search |
| $\Theta(n)$ | linear | traversal, search in an unsorted list |
| $\Theta(n \log n)$ | linearithmic | merge sort, sorting before binary search |
| $\Theta(n^2)$ | quadratic | nested loops, insertion at the front $n$ times |

**Table 1.2:** The growth classes met in this course, in increasing order.

*Remark* 1.5 (Worst, best, amortised). *Worst case* bounds the cost of a single operation over all inputs of size $n$. *Amortised* cost bounds the average over a *sequence* of operations, even when an individual operation is expensive (Theorems 2.3 and 5.2). Neither is the "average over random inputs".

## 1.3 Pointers, memory and parameter passing

Every local variable lives on the *call stack*; objects obtained with `malloc` (C) or `new` (C++) live on the *heap* until released with `free` or `delete`. A pointer is a variable holding an address; `p->f` abbreviates `(*p).f`; and `NULL` is the address that refers to nothing.

**Memory leak.** A heap block that is no longer reachable but was never freed. Every `malloc/new` on the path of a removal must be matched by a `free/delete` of the removed node.

Juevesano D. 2026 All rights reserved. 

**Dangling pointer.** A pointer to a block that has been freed. After `free(cur)`, reading `cur->next` is undefined behaviour — *save the successor first*.

**Mixing.** Memory from `malloc` must be released with `free`, memory from `new` with `delete`; never cross them.

***Pass by value, by pointer, by reference.*** A C function always receives *copies* of its arguments. Hence:

(i) A `struct` passed *by value* is copied *member by member*. If the struct contains an *array* (the course's array list `int items[MAX]`, the course's stack and queue), the whole array is copied, and changes made in the callee are lost. If it contains a *pointer* (a linked list's `head`), only the pointer is copied: callee and caller share the *same nodes*. Modifying the shared nodes is visible to the caller; reassigning the copied `head` or `size` is not (Tracing 2).

(ii) A raw array parameter `int a[]` is really a pointer to the first element, so the callee modifies the caller's array.

(iii) To change a variable of the caller, pass its address: `void f(list *l)`, called as `f(&l)`. To change a *pointer* variable — for example the `head` of a list — pass the address of the pointer, a *pointer to a pointer* `node **h`, called as `f(&l.head)` (Section 3.4; Tracing 7).

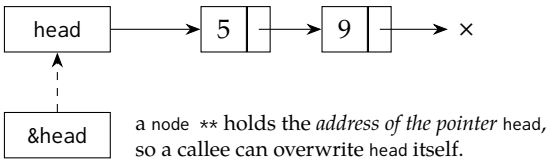



**Figure 1.1:** A pointer to a pointer.

> Evaluation order. In C and C++ the order in which function *arguments* (and the operands of `+`) are evaluated is *unspecified*. The statement `printf("%d %d", pop(&s), pop(&s));` may print the two pops in either order. A tracing question that is well posed never relies on it; write each call as its own statement.

## 1.4 Reasoning about correctness

**Definition 1.6** (Invariant)**.** A *loop invariant* is a statement that is true before the loop, is preserved by each iteration, and, together with the exit condition, implies the result. A *class invariant* is a statement about a data structure that every operation assumes on entry and re-establishes on exit.

**Example 1.7** (Invariant of the array list)**.** **(I)** $0 \le$ `size` $\le$ `MAX`, and the list is exactly `items[0..size-1]`. Cells with index $\ge$ `size` are *not part of the list*: their content is irrelevant, which is why `removeLast` only decrements `size` (the slides' question "should we worry about having two items equal to 10?" has the answer *no*).

**Example 1.8** (Invariant of the singly linked list with tail)**.** **(I)** `size` equals the number of nodes reachable from `head`; `head==NULL` iff `tail==NULL` iff `size==0`; if non-empty, `tail` is the node whose `next` is `NULL`.

Juevesano D. 2026 All rights reserved. 

**Example 1.9** (Invariant of the circular queue and deque). **(I)** $0 \le n \le$ `CAP`, $0 \le$ `front` $<$ `CAP`, and item $i$ of the sequence is `a[(front+i) % CAP]` for $0 \le i < n$. Every operation of Chapters 5 and 6 must leave `front` inside $[0,$ `CAP`$)$ — the reason for the `+ CAP` in `(front - 1 + CAP) % CAP`.

***The edge-case checklist.*** Before submitting any list, stack, queue or deque routine, run it by hand on: (a) the empty structure; (b) one element; (c) two elements; (d) the operation applied at the first position; (e) at the last position; (f) in the middle; (g) every element equal to the key; (h) no element equal to the key; (i) a full fixed-capacity structure; (j) an index that has to wrap around. Most lost marks in this course come from (a), (b), (d), (e) and (j): forgetting to update `head`, `tail`, `size` or `front`.

Juevesano D. 2026 All rights reserved. 

Chapter 2

# Array-Based Lists

## 2.1 Arrays and their shortcomings

An array stores $m$ items of one type in $m$ consecutive memory cells, so the address of cell $i$ is computed in one step: `a[i]` costs $\Theta(1)$. The *primitive* C/C++ array has well-known shortcomings, which motivate every container in this chapter (lecture slides, *why the primitive array is problematic*):

- its size is not stored — a function receiving `int a[]` cannot ask how long it is;
- bounds (subscripts) are not checked: `a[m]` silently reads or corrupts a neighbouring cell;
- a dynamically sized array needs manual heap allocation;
- inserting or deleting in the middle requires manual shifting;
- an array parameter is passed by address, so the callee changes the caller's array.

The remedy is to *wrap* the array together with its bookkeeping, so that the pair behaves as a list. The slides do this twice: the *array container* keeps only the array; the *array list* keeps the array *and the size*.

## 2.2 The array list

**Definition 2.1** (Array list)**.** An *array list* keeps a fixed-capacity array `items[MAX]` and an integer `size`. The list is `items[0..size-1]`; invariant (I) of Example 1.7 holds.

The sample program `arraylist.c` defines the structure and the operations of Table 2.1; Listing 2.1 is its completed form. The slides show the same list as a C++ class `ArrayList` with `push_back`, `insert(pos,x)` and `erase(pos)`, both 0-based (Tracing 4).

| Operation | Effect | Positions | Cost |
|---|---|---|---|
| `init(l)` | `size=0` | — | $\Theta(1)$ |
| `append(l,x)` / `push_back` | store at `items[size]` | — | $\Theta(1)$ |
| `insertFront(l,x)` | shift all right, store at `items[0]` | — | $\Theta(n)$ |
| `insert(pos,x)` | shift `items[pos..size-1]` right | 0-based (slides) | $\Theta(n - \text{pos})$ |
| `removeLast(l)` | `size--` | — | $\Theta(1)$ |
| `removeFirst(l)` | shift all left, `size--` | — | $\Theta(n)$ |
| `erase(pos)` | shift `items[pos+1..]` left | 0-based (slides) | $\Theta(n - \text{pos})$ |
| `itemAt(l,p)` / `setItem(l,x,p)` | read / write | **1-based** (`arraylist.c`) | $\Theta(1)$ |
| `isFull(l)` | `size==MAX` | — | $\Theta(1)$ |

**Table 2.1:** Operations of the array list. Check the statement for the position convention.

Juevesano D. 2026 All rights reserved. 

**Listing 2.1:** The array list of the sample program `arraylist.c`, completed.

```c
#define MAX 10
typedef struct { int items[MAX]; int size; } list;

void init(list *l) { l->size = 0; }

int append(list *l, int x) {                /* O(1) */
    if (l->size == MAX) return 0;           /* full: report failure, change nothing */
    l->items[l->size++] = x;                /* store, THEN increment */
    return 1;
}
int insertFront(list *l, int x) {           /* O(n) */
    int i;
    if (l->size == MAX) return 0;
    for (i = l->size; i > 0; i--)           /* move the LAST item first, so that */
        l->items[i] = l->items[i - 1];      /* nothing is overwritten before it  */
    l->items[0] = x;                        /* is copied                          */
    l->size++;
    return 1;
}
int removeFirst(list *l) {                  /* O(n) */
    int i;
    if (l->size == 0) return 0;
    for (i = 0; i < l->size - 1; i++)       /* move the FIRST item first */
        l->items[i] = l->items[i + 1];
    l->size--;
    return 1;
}
int removeLast(list *l) {                   /* O(1): the old cell is just ignored */
    if (l->size == 0) return 0;
    l->size--;
    return 1;
}
int setItem(list *l, int x, int p) {        /* p is 1-based, as in arraylist.c */
    if (p < 1 || p > l->size) return 0;
    l->items[p - 1] = x;
    return 1;
}
int itemAt(list l, int p) {                 /* 1-based; no bounds check! */
    return l.items[p - 1];
}
```

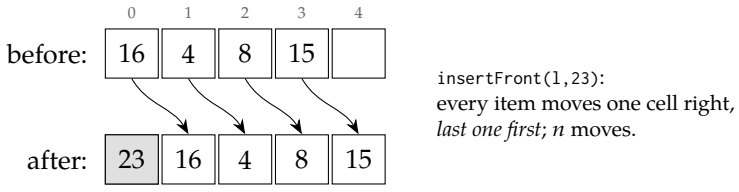



**Figure 2.1:** Insertion at the front shifts the whole list.

**Theorem 2.2** (Cost of positional insertion and deletion)**.** *Inserting at (0-based) position $p$ into an array list of size $n$ performs exactly $n - p$ element moves; deleting the item at position $p$ performs exactly $n - 1 - p$ moves. Hence both are $\Theta(n)$ in the worst case ($p = 0$) and $\Theta(1)$ at the*

*rear* ($p = n$ *or* $p = n - 1$).

*Proof.* Insertion moves `items[p..n-1]`, which are $n - p$ cells, each once. Deletion moves `items[p+1..n-1]`, which are $n - 1 - p$ cells. □

> Direction of the shift. Shift to the *right* (insertion) from the *end* towards the position; shift to the *left* (deletion) from the position towards the end. Doing it the other way overwrites items before they are copied and turns the whole tail into copies of one value — a classical tracing question. The slides show the intermediate states `1 3 3 4 …`, `1 3 4 4 …`: during the shift one value is always duplicated, and the duplicate at the end ("two items equal to 10") is harmless because it lies beyond `size`.

## 2.3 Dynamic arrays and the vector

A fixed `MAX` fails when the list outgrows it. A *dynamic array* (the *ArrayStack* of the notes, the `std::vector` of C++) keeps a backing array of length `cap` $\geq n$ and replaces it when necessary.

- **Grow.** If an insertion finds $n + 1 >$ `cap`, allocate a new array of length $\max(1, 2n)$, copy the $n$ items, release the old array.
- **Shrink.** After a removal, if `cap` $\geq 3n$, resize to $\max(1, 2n)$ so that memory is not wasted.

**Listing 2.2:** A growable array of integers (tested against an array model on random operations).

```
// A growable array of int ("vector list"), after the course notes.
// Invariant: 0 <= n <= cap and the list is a[0..n-1].
class IntVector {
    int *a;          // the backing array (heap)
    int cap;         // its length
    int n;           // number of list items
    // resize: allocate a new array of length max(1, 2n) and copy the n items.
    void resize() {
        int newCap = (2 * n > 1) ? 2 * n : 1;
        int *b = (int *)malloc(newCap * sizeof(int));
        for (int i = 0; i < n; i++) b[i] = a[i];      // O(n) copy
        free(a);
        a = b;
        cap = newCap;
    }
public:
    IntVector() { a = NULL; cap = 0; n = 0; }
    ~IntVector() { free(a); }
    int size()     { return n; }
    int capacity() { return cap; }
    int get(int i) { return a[i]; }           // O(1); caller checks 0 <= i < n
    void set(int i, int x) { a[i] = x; }      // O(1)

    // add(i, x): insert x at position i (0 <= i <= n); shift a[i..n-1] right.
    void add(int i, int x) {
        if (n + 1 > cap) resize();                    // full: double the array
        for (int j = n; j > i; j--) a[j] = a[j - 1];  // shift, from the END
```

```
28          a[i] = x;
29          n++;
30      }
31      void push_back(int x) { add(n, x); }              // amortised O(1)
32
33      // remove(i): delete position i, shifting a[i+1..n-1] left.
34      int remove(int i) {
35          int x = a[i];
36          for (int j = i; j < n - 1; j++) a[j] = a[j + 1];
37          n--;
38          if (cap >= 3 * n) resize();                   // too empty: shrink (3n!)
39          return x;
40      }
41      int pop_back() { return remove(n - 1); }          // amortised O(1)
42  };
```

Twenty push_back calls followed by twenty pop_back calls print the pairs *size*/*capacity*:

```
1/1 2/2 3/4 4/4 5/8 6/8 7/8 8/8 9/16 10/16
11/16 12/16 13/16 14/16 15/16 16/16 17/32 18/32 19/32 20/32
19/32 18/32 17/32 16/32 15/32 14/32 13/32 12/32 11/32 10/20
9/20 8/20 7/20 6/12 5/12 4/8 3/8 2/4 1/2 0/1
```

The capacity doubles exactly when the array is full, and shrinks only when it is at least three times the size.

**Theorem 2.3** (Amortised cost of push_back)**.** *Starting from an empty dynamic array that doubles when full, any sequence of $m$ calls to* push_back *costs $O(m)$ in total, i.e. $O(1)$ amortised per call.*

*Proof.* A call that does not resize costs 1. Resizes occur when the size is $1, 2, 4, \dots, 2^k$ with $2^k < m$, and the resize at size $2^j$ copies $2^j$ items. The total copying cost is $\sum_{j=0}^{k} 2^j = 2^{k+1} - 1 < 2m$. Adding the $m$ unit costs gives a total below $3m$. □

*Remark* 2.4 (Why shrink at $3n$ and not $2n$). If the array shrank as soon as cap $\geq 2n$, then right after a doubling (cap $= 2n$) one removal would trigger a shrink and one insertion a grow: every operation would cost $\Theta(n)$. Waiting until cap $\geq 3n$ leaves a gap of $\Omega(n)$ operations between consecutive resizes, which restores $O(1)$ amortised cost. Growing by a constant (cap $+ 10$) instead of doubling copies $10 + 20 + 30 + \dots$ items, $\Theta(m^2)$ in total.

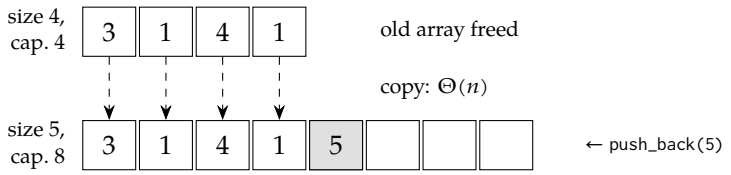



**Figure 2.2:** push_back on a full vector: a new array of double capacity is allocated, the items are copied, the old array is freed, then 5 is stored. All pointers and iterators into the old array are invalid afterwards.

Juevesano D. 2026 All rights reserved. 

## 2.4 Sorted array lists and binary search

If the list is kept in non-decreasing order, searching need not be linear.

**Definition 2.5** (Lower and upper bound)**.** $\mathrm{lb}(x)$ is the smallest index $i$ with $\texttt{items}[i] \geq x$, or $n$ if none; $\mathrm{ub}(x)$ is the smallest index $i$ with $\texttt{items}[i] > x$, or $n$ if none.

Then $x$ occurs in the list iff $\mathrm{lb}(x) < n$ and $\texttt{items}[\mathrm{lb}(x)] = x$; the number of occurrences is $\mathrm{ub}(x) - \mathrm{lb}(x)$; and the sorted position for inserting $x$ *after* equal items is $\mathrm{ub}(x)$.

**Listing 2.3:** Lower bound by bisection.

```
int lowerBound(int a[], int n, int x) {
    int lo = 0, hi = n, mid;
    /* Invariant: a[i] <  x for every i < lo,
                  a[i] >= x for every i >= hi.   */
    while (lo < hi) {
        mid = lo + (hi - lo) / 2;      /* never (lo+hi)/2: avoids overflow     */
        if (a[mid] < x) lo = mid + 1;  /* mid and everything left is too small */
        else            hi = mid;      /* mid could be the answer: keep it     */
    }
    return lo;                         /* lo == hi: the boundary               */
}
```

**Theorem 2.6** (Correctness and cost of bisection)**.** *The invariant holds initially (the ranges are empty), is preserved by both branches, and when $lo = hi$ it says exactly that $lo = \mathrm{lb}(x)$. Each round halves $hi - lo$, so the loop runs at most $\lfloor \lg n \rfloor + 1$ times: search is $\Theta(\log n)$.*

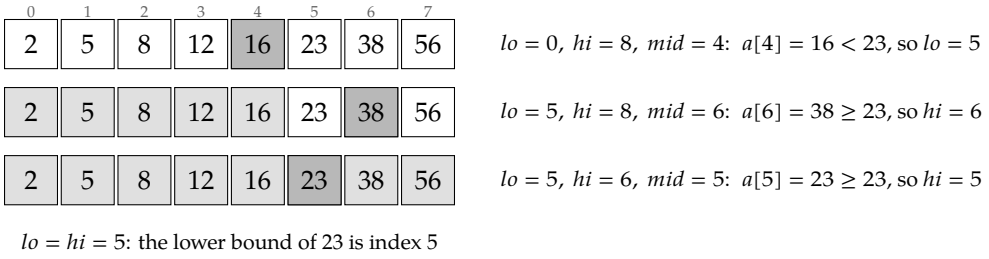



**Figure 2.3:** `lowerBound(a, 8, 23)`. Shaded cells are already excluded; the dark cell is `mid`. The range $[lo, hi)$ halves each round.

**Proposition 2.7** (Cost of the sorted array list)**.** `search`, `count`, `lowerBound`, `upperBound`: $\Theta(\log n)$. `insert` *and* `erase`: $\Theta(\log n)$ *to locate plus* $\Theta(n)$ *to shift, i.e.* $\Theta(n)$ *overall. Sorting does not make* insertion *cheaper; it makes* searching *cheaper. Complete code appears in Problem 4.*

## 2.5 Index techniques for array problems

Most array problems of the online-judge kind are solved by one of four patterns. Each replaces a quadratic double loop by a single pass.

***Read index and write index.*** One index $r$ reads every item; a second index $w$ marks the end of the part already built. An item that must be kept is copied to `a[w++]`. Because $w \leq r$

at all times, nothing unread is overwritten. Used for removing duplicates, moving zeroes, in-place compression (Problems 1, 7, 10).

***Two indices from the two ends.*** On a *sorted* array, `lo` starts at the left and `hi` at the right; at each step one of them moves inward, and the sortedness proves that the discarded index cannot be part of an answer. Used for two-sum and for palindrome checks (Problem 8; on a doubly linked list, Problem 20).

***The reversal trick.*** Reversing a block is a two-ends loop. Rotation by $k$ is three reversals, transpose followed by row reversal rotates a matrix (Problems 5, 12).

***Running summaries.*** One left-to-right pass keeps a summary of the prefix read so far: a running sum, the best sum of a subarray ending here (Kadane, Problem 9), or a vote counter (Boyer–Moore, Problem 11). Chapters 4 and 6 extend this idea: the summary becomes a *stack* or a *deque* of candidate indices.

Juevesano D. 2026 All rights reserved. 

Chapter 3

# Linked Lists

An array list is rigid: insertion in the middle moves everything behind it, and growth copies the whole array. A *linked list* gives up random access in exchange for cheap structural change. Instead of consecutive cells it uses *nodes*, each holding an item and pointers to its neighbours.

## 3.1 The singly linked list

**Definition 3.1** (Singly linked list)**.** A *node* is a pair $(x, next)$. A *singly linked list* (SLL) is a pointer `head` to the first node, optionally a pointer `tail` to the last, and a counter `size`. The last node's `next` is `NULL`; the empty list has `head==NULL`.

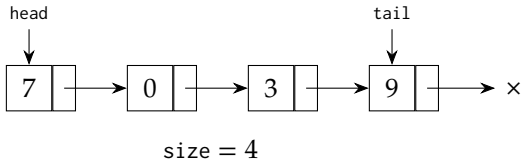



**Figure 3.1:** A singly linked list with head and tail pointers.

**Listing 3.1:** The SLL with head, tail and size (file `sll.h`); `pushBack` and `pushFront` are $\Theta(1)$.

```
typedef struct snode {
    int item;
    struct snode *next;
} snode;

typedef struct {
    snode *head;
    snode *tail;
    int size;
} sll;

void sinit(sll *l) {
    l->head = NULL;
    l->tail = NULL;
    l->size = 0;
}

// Adds x at the back. Returns 1 on success, 0 if malloc fails.
int pushBack(sll *l, int x) {
    snode *n = (snode *)malloc(sizeof(snode));
    if (n == NULL) return 0;
    n->item = x;
    n->next = NULL;
```

Juevesano D. 2026 All rights reserved. 

```c
    if (l->tail == NULL) l->head = n;
    else l->tail->next = n;
    l->tail = n;
    l->size++;
    return 1;
}

// Adds x at the front. Returns 1 on success, 0 if malloc fails.
int pushFront(sll *l, int x) {
    snode *n = (snode *)malloc(sizeof(snode));
    if (n == NULL) return 0;
    n->item = x;
    n->next = l->head;
    l->head = n;
    if (l->tail == NULL) l->tail = n;
    l->size++;
    return 1;
}

// Prints the list as [1 2 3].
void sdisplay(sll *l) {
    snode *c;
    printf("[");
    for (c = l->head; c != NULL; c = c->next) {
        printf("%d", c->item);
        if (c->next != NULL) printf(" ");
    }
    printf("]\n");
}

void sdestroy(sll *l) {
    snode *c = l->head;
    while (c != NULL) {
        snode *nxt = c->next;
        free(c);
        c = nxt;
    }
    sinit(l);
}
```

### *3.1.1 Insertion and removal at the ends*

***push_front* (*prepend*).** Allocate $u$; set $u.next =$ `head`; set `head` $= u$; if the list was empty also set `tail` $= u$; increment `size`. $\Theta(1)$.

***push_back* (*append*).** Allocate $u$; if the list is empty set `head` $=$ `tail` $= u$, otherwise `tail->next` $= u$ and `tail` $= u$. $\Theta(1)$ *because the tail pointer exists*; without it, appending must walk the whole list, $\Theta(n)$.

***pop_front.*** Save the old head; `head` $=$ `head->next`; if the list became empty set `tail=NULL`; free the old node. $\Theta(1)$.

***pop_back.*** The new tail is the node *before* the old tail, and a singly linked node cannot reach its predecessor: one must walk from `head` to the second-to-last node, $n - 2$ steps.

Juevesano D. 2026 All rights reserved. 

$\Theta(n)$.

**Listing 3.2:** `popFront` is $\Theta(1)$; `popBack` is $\Theta(n)$.

```
int popFront(sll *l, int *out) {
    snode *old;
    if (l->head == NULL) return 0;          /* underflow                  */
    old = l->head;
    *out = old->item;
    l->head = old->next;                    /* unlink                     */
    if (l->head == NULL) l->tail = NULL;    /* list became empty          */
    free(old);
    l->size--;
    return 1;
}
int popBack(sll *l, int *out) {
    snode *cur;
    if (l->head == NULL) return 0;
    *out = l->tail->item;
    if (l->head == l->tail) {               /* exactly one node           */
        free(l->head);
        l->head = l->tail = NULL;
    } else {
        cur = l->head;                      /* walk to the SECOND-TO-LAST */
        while (cur->next != l->tail) cur = cur->next;
        free(l->tail);
        cur->next = NULL;
        l->tail = cur;
    }
    l->size--;
    return 1;
}
```

| Operation | SLL (head+tail) | Why |
|---|---|---|
| push_front, pop_front | $\Theta(1)$ | direct access to `head` |
| push_back | $\Theta(1)$ | direct access to `tail` |
| pop_back | $\Theta(n)$ | predecessor of `tail` is unreachable |
| insert / erase at position $p$ | $\Theta(p)$ | walk to the node at $p-1$ |
| `get(i)`, `set(i,x)`, search | $\Theta(i)$, $\Theta(n)$ | no random access |

**Table 3.1:** Costs for the singly linked list. A stack (push and pop at the head) and a FIFO queue (push at the tail, pop at the head) are therefore both $\Theta(1)$; a deque is not (pop_back).

### *3.1.2 Case analysis: draw before you code*

Almost every linked-list bug is a missed special case. When removing the node $t$ with predecessor $p$ (if any), the cases are:

| Case | Pointer repair | Also update |
|---|---|---|
| $t$ is the only node | `head=tail=NULL` | `size` |
| $t$ is the head (not only) | `head=t->next` | `size` |
| $t$ is the tail (not only) | `p->next=NULL` | `tail=p`, `size` |
| $t$ in the middle | `p->next=t->next` | `size` |

Juevesano D. 2026 All rights reserved. 

Insertion has the analogous cases: into an empty list, at the front, at the end, in the middle. A *dummy head node* (Section 3.4) removes the "head" case from the code.

## 3.2 The doubly linked list

**Definition 3.2** (Doubly linked list)**.** Each node holds $(prev, x, next)$. A *doubly linked list* (DLL) keeps `head`, `tail` and `size`. Invariant: for every node $u$ with $u.next \neq$ `NULL` we have $u.next.prev = u$; `head->prev` and `tail->next` are `NULL`.

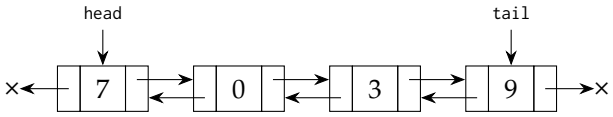



**Figure 3.2:** A doubly linked list.

***What the extra pointer buys.***

- pop_back becomes $\Theta(1)$: `tail = tail->prev; tail->next = NULL`.
- Given a pointer to *any* node, that node can be unlinked in $\Theta(1)$ (no walk to find its predecessor).
- Traversal works in both directions, so a node at index $i$ can be reached from the nearer end in $\Theta(1 + \min(i, n - i))$, and two pointers can walk towards each other (palindromes, Problem 20).
- All four end operations are $\Theta(1)$: the DLL is a deque (Chapter 6).

The price is an extra pointer per node and *twice as many pointer updates* per operation — forgetting the backward link is the most common DLL bug.

**Listing 3.3:** The doubly linked list used by the departmental exercises (file `dll.h`: `list`, `node`, `append`, `display`).

```
typedef struct node {
    int item;
    struct node *next;
    struct node *prev;
} node;

typedef struct {
    node *head;
    node *tail;
    int size;
} list;

void init(list *l) {
    l->head = NULL;
    l->tail = NULL;
    l->size = 0;
}

// Adds x at the tail. Returns 1 on success, 0 if malloc fails.
int append(list *l, int x) {
```

```
    node *n = (node *)malloc(sizeof(node));
    if (n == NULL) return 0;
    n->item = x;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail == NULL) l->head = n;
    else l->tail->next = n;
    l->tail = n;
    l->size++;
    return 1;
}

// Prints head to tail, separated by spaces.
void display(list *l) {
    node *c;
    for (c = l->head; c != NULL; c = c->next) {
        printf("%d", c->item);
        if (c->next != NULL) printf(" ");
    }
    printf("\n");
}

// Prints tail to head, separated by spaces.
void displayBackward(list *l) {
    node *c;
    for (c = l->tail; c != NULL; c = c->prev) {
        printf("%d", c->item);
        if (c->prev != NULL) printf(" ");
    }
    printf("\n");
}

void destroy(list *l) {
    node *c = l->head;
    while (c != NULL) {
        node *nxt = c->next;
        free(c);
        c = nxt;
    }
    init(l);
}
```

***Removing a node (the four updates).*** For a node `cur` in a list `l`:

```
if (cur->prev != NULL) cur->prev->next = cur->next; else l->head = cur->next;
if (cur->next != NULL) cur->next->prev = cur->prev; else l->tail = cur->prev;
free(cur);  l->size--;
```

Each `else` handles the case that `cur` is at an end. See Problem 13.

### *3.2.1 Sentinel (dummy) nodes*

Every special case above exists because `NULL` terminates the list. If a *dummy* node that carries no item is linked into a cycle — `dummy.next` is the head, `dummy.prev` is the tail, and the

Juevesano D. 2026 All rights reserved. 

empty list has `dummy.next=dummy.prev=&dummy` — then no pointer is ever `NULL` and insertion before a node $w$ and removal of $w$ are uniform. The notes' DLList is built this way, with `getNode(i)` walking from the nearer end:

**Listing 3.4:** DLList: a doubly linked list with a dummy node (tested against an array model).

```
// DLList with a DUMMY node (a circular list), after the course notes.
// dummy.next is the head, dummy.prev is the tail; an empty list points to
// itself, so no pointer is ever NULL and the special cases disappear.
struct dnode {
    int x;
    dnode *next, *prev;
};
class DLList {
    dnode dummy;
    int n;
public:
    DLList() { dummy.next = &dummy; dummy.prev = &dummy; n = 0; }
    ~DLList() { while (n > 0) remove(0); }
    int size() { return n; }

    // getNode(i): start from the nearer end. Cost O(1 + min(i, n-i)).
    dnode *getNode(int i) {
        dnode *p;
        int j;
        if (i < n / 2) {
            p = dummy.next;
            for (j = 0; j < i; j++) p = p->next;
        } else {
            p = &dummy;
            for (j = n; j > i; j--) p = p->prev;
        }
        return p;
    }
    int get(int i) { return getNode(i)->x; }
    int set(int i, int x) {
        dnode *u = getNode(i);
        int y = u->x;
        u->x = x;
        return y;
    }

    // addBefore(w, x): link a new node u between w->prev and w. O(1).
    dnode *addBefore(dnode *w, int x) {
        dnode *u = new dnode;
        u->x = x;
        u->prev = w->prev;
        u->next = w;
        u->next->prev = u;
        u->prev->next = u;
        n++;
        return u;
    }
    void add(int i, int x) { addBefore(getNode(i), x); }   // i in 0..n

    // remove(w): make the neighbours skip w. O(1) once w is known.
```

```
    void remove(dnode *w) {
        w->prev->next = w->next;
        w->next->prev = w->prev;
        delete w;
        n--;
    }
    int remove(int i) {
        dnode *w = getNode(i);
        int x = w->x;
        remove(w);
        return x;
    }
};
```

All of `get`, `set`, `add` and `remove` cost $\Theta(1 + \min(i, n - i))$; the pointer surgery itself is $\Theta(1)$ once the node is found. The C version of the same idea, specialised to a deque, is Problem 51.

## 3.3 Circular linked lists

**Definition 3.3** (Circular list)**.** In a *circular* singly linked list the last node points back to the first instead of to `NULL`. It suffices to keep a pointer `last` to the last node: the first node is `last->next`.

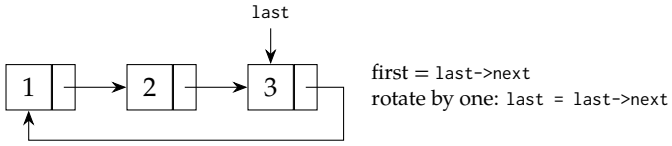



**Figure 3.3:** A circular singly linked list kept by its last node only.

With one pointer, both queue operations are $\Theta(1)$: enqueue links the new node after `last` and makes it the new `last`; dequeue unlinks `last->next`. *Rotating* the circle (moving the front item to the back) is a single assignment `last = last->next`, which is why circular lists suit round-robin scheduling and the Josephus problem.

**Listing 3.5:** A queue on a circular list with only a `last` pointer.

```
// A queue on a circular singly linked list that keeps ONLY a pointer to the
// last node: last->next is the first node. Both operations are O(1).
typedef struct cnode { int item; struct cnode *next; } cnode;

void enq(cnode **last, int x) {
    cnode *n = (cnode *)malloc(sizeof(cnode));
    n->item = x;
    if (*last == NULL) n->next = n;          // a one-node circle
    else { n->next = (*last)->next; (*last)->next = n; }
    *last = n;                               // the new node is the last one
}

int deq(cnode **last, int *x) {
    if (*last == NULL) return 0;
    cnode *first = (*last)->next;
```

```
16      *x = first->item;
17      if (first == *last) *last = NULL;        // it was the only node
18      else (*last)->next = first->next;
19      free(first);
20      return 1;
21  }
```

The traps: the one-node circle points to itself (`n->next = n`); removing the only node must set `last` to `NULL`; and a traversal must stop when it comes back to its starting node, not at `NULL` (Tracing 24).

### 3.4 Programming patterns

***Erase while iterating.*** Never advance with `cur = cur->next` *after* freeing `cur`. Save `nxt = cur->next` *first*, then free, then `cur = nxt`.

***Dummy head.*** For a singly linked list, create a local node `dummy` with `dummy.next = head`. The code that reaches "the node before the target" then works even when the target is the head. At the end set `head = dummy.next` (and repair `tail` if the removal could have reached the end).

***Two pointers at different speeds or gaps.***

- *Middle*: `slow` moves one node, `fast` two, per step; when `fast` falls off the end, `slow` is at the middle. One pass.
- *k-th node from the end*: move `lead` $k$ nodes ahead of `lag`, then advance both until `lead` is `NULL`.
- *Cycle detection* (*Floyd*): if the list has a cycle `fast` laps `slow`; they meet. Restarting one pointer at `head` and advancing both by one meets at the first cycle node.

***Reversal.*** The invariant of the three-pointer loop is: the nodes before `cur` already form a reversed list headed by `prev`.

***Recursion on lists.*** A list is either empty or a node followed by a list, so recursive procedures mirror the definition. To *change* the list structurally with recursion, pass the address of the pointer that must change: `void append(node **h, int x)`. Then `*h` is "the pointer that currently points at the rest of the list"; the base case `*h==NULL` stores the new node *into that pointer*, and the recursive case passes `&((*h)->next)`. This is exactly why the double pointer is needed — a single `node *h` would be a copy (Tracing 7).

***Copying: shallow versus deep.*** Assigning one list *struct* to another copies `head`, `tail`, `size` but not the nodes: both variables then share every node, and freeing one invalidates the other. A *deep copy* allocates a new node for every item. A routine such as `appendList(l, m)` that must survive the later destruction of `m` has to deep copy (Problem 14). A struct that contains its own sentinel node must never be copied at all (Tracing 25).

***Moving versus copying; splicing.*** If the donor list is allowed to give up its nodes, the nodes can simply be *relinked*: with a tail pointer, appending a whole list to another costs $\Theta(1)$, and inserting it after a known node is $\Theta(1)$. This is the list *splice* (Problem 18); it is the operation that arrays can never do in constant time.

***Augmenting the list.*** Extra bookkeeping pointers (such as a pointer to the middle node) make some queries $\Theta(1)$ at the price of keeping the extra pointer correct in *every* update.

Juevesano D. 2026 All rights reserved. 

The design question is always: how does this operation change the index of the node the extra pointer must designate? Problem 24 works this out in full.

## 3.5 Arrays versus linked lists

| Operation | Array list | SLL (head+tail) | DLL |
|---|---|---|---|
| `get(i)` | $\Theta(1)$ | $\Theta(i)$ | $\Theta(\min(i, n-i))$ |
| insert / erase at front | $\Theta(n)$ | $\Theta(1)$ | $\Theta(1)$ |
| insert at rear | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ |
| erase at rear | $\Theta(1)$ | $\Theta(n)$ | $\Theta(1)$ |
| insert / erase after a known node | $\Theta(n)$ | $\Theta(1)$ | $\Theta(1)$ |
| erase a known node | $\Theta(n)$ | $\Theta(n)$ | $\Theta(1)$ |
| splice one list into another | $\Theta(n)$ | $\Theta(1)$ | $\Theta(1)$ |
| memory per item | item | item + 1 pointer | item + 2 pointers |
| cache behaviour | contiguous | scattered | scattered |

**Table 3.2:** When position matters, choose the array; when structural change matters, choose the linked list.

Juevesano D. 2026 All rights reserved. 

# Chapter 4

# Stacks

## 4.1 The Stack ADT

**Definition 4.1** (Stack)**.** A *stack* is a sequence in which insertions and deletions take place at one end only, the *top*. Its operations are `push(x)` (add $x$ on top), `pop()` (remove and return the top item), `top()` or `peek()` (return the top item without removing it), `empty()` and `size()`. The item removed by `pop` is always the one inserted most recently: *last in, first out* (LIFO).

A stack is the right structure whenever the most recent unfinished thing must be finished first: an opening bracket waits for its closing bracket, an operator waits for its right operand, a function call waits for the calls it made, an editor's last action is the first to be undone.

> Popping an empty stack. `pop` and `top` on an empty stack are errors (*stack underflow*); `push` on a full fixed-capacity stack is *stack overflow*. The library below reports both through a return value of 0 and then leaves the output parameter *unchanged*. A client that ignores the return value reads whatever was in its variable before — a favourite twist in tracing questions.

## 4.2 Implementations

***Array stack.*** Keep the items in `items[0..size-1]` with the top at `items[size-1]`. Then push writes `items[size++]` and pop reads `items[--size]`: the top is at the *rear* of the array list, where insertion and deletion cost $\Theta(1)$ (Theorem 2.2). Putting the top at `items[0]` would make both $\Theta(n)$.

**Listing 4.1:** The array stack used throughout Parts II and III (first part of `stackqueue.h`). The struct contains the array, so it must be passed by pointer.

```
typedef struct {
    int items[100];
    int size;           // the top is items[size - 1]
} stack;

void stInit(stack *s) { s->size = 0; }
int stEmpty(stack *s) { return s->size == 0; }
int stSize(stack *s) { return s->size; }

// Returns 1 on success, 0 if the stack is full.
int stPush(stack *s, int x) {
    if (s->size == 100) return 0;
    s->items[s->size] = x;
    s->size++;
    return 1;
}
```

Juevesano D. 2026 All rights reserved. 

```
17
18 // Removes the top into *x. Returns 1 on success, 0 if the stack is empty.
19 int stPop(stack *s, int *x) {
20     if (s->size == 0) return 0;
21     s->size--;
22     *x = s->items[s->size];
23     return 1;
24 }
25
26 // Copies the top into *x without removing it. Returns 0 if empty.
27 int stTop(stack *s, int *x) {
28     if (s->size == 0) return 0;
29     *x = s->items[s->size - 1];
30     return 1;
31 }
```

***Growable stack.*** With a vector instead of a fixed array (Section 2.3 of Chapter 2), push is $\Theta(1)$ amortised (Theorem 2.3) and there is no overflow; this is `std::stack<int>` on top of `std::deque` or `std::vector`.

***Linked stack.*** In a singly linked list the head is the only end at which both insertion and deletion are $\Theta(1)$ (`pushFront` of Listing 3.1 and `popFront` of Listing 3.2). Hence the top of a linked stack is the *head*; no tail pointer is needed. Putting the top at the tail would make pop $\Theta(n)$, because `popBack` has to find the predecessor of the tail.

| Operation | array stack | vector stack | linked stack (top = head) |
|---|---|---|---|
| `push` | $\Theta(1)$, may overflow | $\Theta(1)$ amortised | $\Theta(1)$ + one `malloc` |
| `pop`, `top` | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ + one `free` |
| `size` | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ if a counter is kept |
| space | `MAX` cells | $\leq 3n$ cells | $n$ nodes, one pointer each |

**Table 4.1:** Stack implementations.

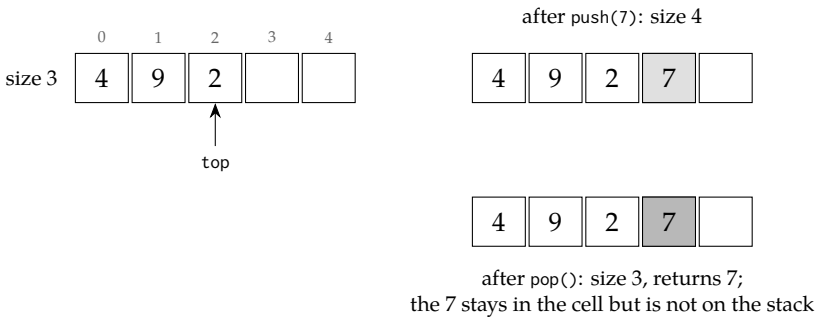



**Figure 4.1:** The array stack. A popped value remains in its cell (Example 1.7).

## 4.3 Brackets and expressions

### *4.3.1 Balanced brackets*

Scan the string from left to right. Push every opening bracket. At a closing bracket, the opener it must match is the most recent one not yet matched, the top of the stack: pop it

Juevesano D. 2026 All rights reserved. 

and compare. The string is balanced if and only if no comparison fails, no pop finds the stack empty, and the stack is empty at the end.

**Listing 4.2:** Balanced brackets (Problem 32).

```
// Returns 1 if every bracket in s is closed by the same kind of bracket,
// in the right order.
// s contains only the characters ( ) [ ] { }.
int isBalanced(char s[]) {
    stack st;
    stInit(&st);

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            stPush(&st, c);                    // remember the opener
        } else {
            int open;
            if (!stPop(&st, &open)) return 0;   // closer with nothing to close
            if (c == ')' && open != '(') return 0;
            if (c == ']' && open != '[') return 0;
            if (c == '}' && open != '{') return 0;
        }
    }
    return stEmpty(&st);                       // leftover openers: not balanced
}
```

The three ways to fail are three different strings: `"(]"` (wrong kind), `"())"` (closer with nothing open) and `"(()"` (opener never closed). A counter suffices when there is only one kind of bracket; with several kinds the counter accepts `"([)]"`, which the stack rejects.

### *4.3.2 Postfix notation*

**Definition 4.2** (Postfix)**.** In *postfix* (reverse Polish) notation an operator follows its two operands: *a b* op. Thus $2+3\cdot4$ is `234*+` and $(2+3)\cdot4$ is `23+4*`. Postfix needs no parentheses and no precedence rules.

***Evaluation.*** Scan left to right; push operands; at an operator pop *two* values, apply the operator, push the result. At the end the stack holds exactly one value, the result. The value popped *first* is the *right* operand: for `93-` the stack is $[9,3]$, the first pop gives $b=3$, the second $a=9$, and the result is $a-b=6$. Swapping them gives $-6$; for `+` and `*` the mistake is invisible, which is why tracing questions use `-` and `/` (Tracing 13).

***Infix to postfix (shunting-yard).*** Operands go straight to the output. Operators wait on a stack. When an operator `c` arrives, every operator on the stack that must be applied *before* `c` is popped to the output first: those of higher precedence and, for left-associative operators, those of *equal* precedence. Then `c` is pushed. A `(` is pushed and acts as a floor; a `)` pops to the output until the matching `(`, which is discarded. At the end, the remaining operators are popped.

**Listing 4.3:** Infix to postfix and postfix evaluation (Problem 33); single-digit operands.

```
// Precedence of an operator: * and / bind tighter than + and -.
int prec(char op) {
```

```
    if (op == '*' || op == '/') return 2;
    return 1;
}

// Converts infix to postfix. Operands are single digits; operators are
// + - * / and parentheses; there are no spaces. "2+3*4" -> "234*+".
void toPostfix(char in[], char out[]) {
    stack ops;                      // operators waiting for their operands
    int k = 0;                      // next free place in out
    stInit(&ops);

    for (int i = 0; in[i] != '\0'; i++) {
        char c = in[i];
        if (c >= '0' && c <= '9') {
            out[k++] = c;           // operands go straight to the output
        } else if (c == '(') {
            stPush(&ops, c);
        } else if (c == ')') {
            int top;                // pop until the matching '('
            while (stPop(&ops, &top) && top != '(') out[k++] = top;
        } else {
            int top;                // pop operators that must run first
            while (stTop(&ops, &top) && top != '(' && prec(top) >= prec(c)) {
                stPop(&ops, &top);
                out[k++] = top;
            }
            stPush(&ops, c);
        }
    }
    int top;
    while (stPop(&ops, &top)) out[k++] = top;
    out[k] = '\0';
}

// Evaluates a postfix expression of single digits and + - * /
// (integer division). "234*+" -> 14. The input is always valid.
int evalPostfix(char p[]) {
    stack st;
    stInit(&st);

    for (int i = 0; p[i] != '\0'; i++) {
        char c = p[i];
        if (c >= '0' && c <= '9') {
            stPush(&st, c - '0');
        } else {
            int b, a;               // b was pushed last: the right side
            stPop(&st, &b);
            stPop(&st, &a);
            if (c == '+') stPush(&st, a + b);
            if (c == '-') stPush(&st, a - b);
            if (c == '*') stPush(&st, a * b);
            if (c == '/') stPush(&st, a / b);
        }
    }
    int result;
```

Juevesano D. 2026 All rights reserved. 

```
58      stPop(&st, &result);
59      return result;
60  }
```

| Read | Action | Operator stack (top right) | Output |
|---|---|---|---|
| `2` | output | | `2` |
| `*` | push | `*` | `2` |
| `(` | push | `* (` | `2` |
| `3` | output | `* (` | `23` |
| `+` | top is `(`: push | `* ( +` | `23` |
| `4` | output | `* ( +` | `234` |
| `)` | pop to `(` | `*` | `234+` |
| `-` | pop `*` (prec. $2 \geq 1$), push | `-` | `234+*` |
| `5` | output | `-` | `234+*5` |
| end | pop all | | `234+*5-` |

**Table 4.2:** Shunting-yard on `2*(3+4)-5`. Evaluating `234+*5-` gives $2 \cdot 7 - 5 = 9$.

**Proposition 4.3** (Why `>=`)**.** *With the test* `prec(top) >= prec(c)`*, operators of equal precedence leave the stack in arrival order, so* `8-3-2` *becomes* `83-2-`*, which is* $(8 - 3) - 2 = 3$*. With* `prec(top) > prec(c)` *it becomes* `832--`*, which is* $8 - (3 - 2) = 7$*: the operators would be treated as right-associative, which is correct only for exponentiation.*

## 4.4 The monotonic stack

Many problems ask, for every position $i$ of an array, for the nearest position to the left or right whose value is greater (or smaller) than `a[i]`. The brute force scans from each $i$ and costs $\Theta(n^2)$. The *monotonic stack* answers all $n$ queries in $\Theta(n)$.

***Idea* (*next greater element*).** Scan left to right. Keep on a stack the indices whose answer is still unknown. When `a[i]` arrives, every waiting index $j$ with `a[j]` $<$ `a[i]` has found its answer, `a[i]`, and is popped; then $i$ itself starts waiting. An index that is still waiting has no greater value to its right.

**Listing 4.4:** Next greater element (Problem 35): the template of all monotonic-stack solutions.

```
// out[i] = the first value to the right of a[i] that is greater than a[i],
// or -1 if there is none.   [2 1 5 3] -> [5 5 -1 -1]
// The stack holds indices whose answer is still unknown; their values
// decrease from bottom to top.
void nextGreater(int a[], int n, int out[]) {
    stack waiting;
    stInit(&waiting);

    for (int i = 0; i < n; i++) out[i] = -1;

    for (int i = 0; i < n; i++) {
        int j;
        // a[i] answers every waiting index whose value is smaller
        while (stTop(&waiting, &j) && a[j] < a[i]) {
            out[j] = a[i];
            stPop(&waiting, &j);
```

Juevesano D. 2026 All rights reserved. 

```
17          }
18          stPush(&waiting, i);
19      }
20  }
```

**Proposition 4.4** (The stack is monotonic)**.** *At every moment the values* `a[j]` *of the waiting indices are non-increasing from the bottom to the top of the stack.*

*Proof.* Before `i` is pushed, the loop has popped every waiting index with a smaller value, so the new top is at least `a[i]`. By induction the whole stack is non-increasing. Consequently the pops of one round remove exactly the waiting indices with a value below `a[i]`; they all sit at the top. □

**Theorem 4.5** (Linear time)**.** `nextGreater` *runs in* $\Theta(n)$ *time although its inner* `while` *loop may run up to i times in round i.*

*Proof.* Every index is pushed exactly once, and every successful iteration of the `while` pops one index. An index can be popped at most once after being pushed once, so the total number of iterations of the `while` over *all* rounds is at most $n$. Each round also does one failed test. The total is at most $n + n + n = O(n)$ steps, and clearly $\Omega(n)$. □

| $i$ | `a[i]` | popped (answer `a[i]`) | stack after (values) |
|---|---|---|---|
| 0 | 2 | — | 2 |
| 1 | 7 | 2 | 7 |
| 2 | 3 | — | 7 3 |
| 3 | 5 | 3 | 7 5 |
| 4 | 4 | — | 7 5 4 |
| 5 | 6 | 4, 5 | 7 6 |
| 6 | 8 | 6, 7 | 8 |

**Table 4.3:** Next greater on $[2, 7, 3, 5, 4, 6, 8]$. The answer is $[7, 8, 5, 6, 6, 8, -1]$; the 8 still waiting at the end gets $-1$.

***Variants.*** Four choices produce the whole family.

- *Greater* or *smaller*: flip the comparison (the stack becomes increasing).
- *Next* (to the right) or *previous* (to the left): for "previous greater", the answer for $i$ is whatever is on top *after* the pops, just before $i$ is pushed. Alternatively, scan from right to left.
- *Strict* or not: with `<` equal values wait for each other; with `<=` a later equal value counts as an answer. Read the problem: "strictly warmer" (Problem 36) means `<`.
- What is recorded: the value `a[i]`, the index $i$, or the distance $i - j$ (daily temperatures).

The *stock span* (Problem 37, Tracing 15) is the "previous greater" problem solved online: each day's price arrives after the previous day's answer was given, so the input cannot be scanned twice. The stack keeps pairs (price, span); a popped pair hands over the days it already covered.

Juevesano D. 2026 All rights reserved. 

## 4.5 Augmented stacks

***Min stack.*** A stack that also answers `min()` in $\Theta(1)$ cannot recompute the minimum after every pop. Keep a second stack `mins` in step with the first: at each level it holds the minimum of that level and everything below it. Push $x$ onto `mins` as $\min(x, \text{top of } \texttt{mins})$; pop both stacks together. Then `min()` is the top of `mins` (Problem 34).

The general lesson: *a stack's history is itself a stack*. Anything that depends only on the items at or below a level can be stored at that level and is restored for free by the pops.

## 4.6 Stack permutations

Push the numbers $1, 2, \ldots, n$ in this order onto an initially empty stack, and pop at any moments, printing each popped number. Which output orders are possible?

**Definition 4.6** (Stack permutation)**.** A permutation $p_1 p_2 \cdots p_n$ of $1..n$ is a *stack permutation* if it can be printed this way. An operation sequence is written as a word over `u` (push the next number) and `o` (pop and print); it has $n$ `u`'s and $n$ `o`'s, and no prefix has more `o`'s than `u`'s.

***Greedy test.*** Process the target output from left to right. To print $p_k$: while the top is not $p_k$, push the next input number (if none is left, *fail*); then pop. The greedy choice is forced: a number must be pushed before it is printed, and pushing *beyond* $p_k$ only buries it. Thus each target has at most one operation sequence, and Problem 42 decides the question in $\Theta(n)$.

**Theorem 4.7** (The forbidden pattern)**.** *A permutation is a stack permutation if and only if there are no positions $i < j < k$ with $p_j < p_k < p_i$ (no subsequence with the shape* high, low, middle*, the pattern* `312`*).*

*Proof sketch.* If $p_i$ is printed first, then $p_j$ and $p_k$, both smaller, were pushed before $p_i$. Since $p_j < p_k$, $p_j$ was pushed first and lies *below* $p_k$; it cannot be printed before $p_k$ is popped. Conversely, if the greedy test gets stuck, the top $t$ is not the wanted $p_k$ and $p_k < t$ is buried below; $t$ was pushed for some earlier printed value $p_i > t$, and $p_i, p_k, t$ form the forbidden pattern (with $t$ printed later). □

For $n = 3$ the stack permutations are `123`, `132`, `213`, `231` and `321`; the only impossible one is `312`. In general their number is the *Catalan number* $C_n = \frac{1}{n+1}\binom{2n}{n}$: $1, 2, 5, 14, 42, \ldots$ (Tracing 14).

## 4.7 Stacks and recursion

Each call of a function pushes an *activation record* (parameters, locals, return address) on the call stack; each return pops it. Hence every recursive procedure can be rewritten with an explicit stack, and conversely a stack-based procedure can often be written recursively, with the call stack doing the work: printing a singly linked list backwards (Tracing 9) is exactly "push every node, then pop every node". The recursion depth is the maximum stack size, so a recursive traversal of an $n$-node list uses $\Theta(n)$ memory and may overflow the call stack for long lists, while the loop version uses $\Theta(1)$.

Juevesano D. 2026 All rights reserved. 

| If the problem says ... | think of ... |
|---|---|
| brackets, tags, nested structure | push openers, pop on closers (32, 39) |
| evaluate / convert an expression | two stacks or shunting-yard (33) |
| next / previous greater or smaller, span, "until warmer" | monotonic stack (35–37) |
| cancel adjacent pairs, collisions | the stack holds the survivors so far (38, 40) |
| undo, back button, most recent first | a plain stack |
| min / max in $\Theta(1)$ | a second stack in step (34) |
| is this pop order possible? | simulate greedily (42) |
| reverse something | push all, pop all |

**Table 4.4:** Recognising a stack problem. Numbers refer to the problems of Chapter 10.

Juevesano D. 2026 All rights reserved. 

# Chapter 5

# Queues

## 5.1 The Queue ADT

**Definition 5.1** (Queue)**.** A *queue* is a sequence in which insertions take place at one end, the *rear* (or back), and deletions at the other, the *front*. Its operations are `enqueue(x)` (also `push`, `add`, `offer`), `dequeue()` (also `pop`, `remove`, `poll`), `front()` (`peek`), `empty()` and `size()`. The item removed is always the one that has waited longest: *first in, first out* (FIFO).

Queues model waiting lines: print jobs, requests to a server, processes waiting for the processor (round-robin scheduling), messages between programs, and the frontier of a breadth-first search. In each case fairness means serving in order of arrival.

## 5.2 Array implementations

Both ends of a queue change, so the array list of Chapter 2, which is fast only at its rear, is not enough. There are three attempts.

***1. Front at index 0.*** Enqueue is `items[size++] = x`, $\Theta(1)$; dequeue reads `items[0]` and shifts the remaining $n-1$ items left, $\Theta(n)$ (Theorem 2.2).

***2. A moving front.*** Keep an index `front` and dequeue by `front++`. Both operations become $\Theta(1)$, but the occupied part `items[front..rear]` drifts to the right. When it reaches the end of the array the queue reports "full" although the cells before `front` are free (*false overflow*).

***3. The circular buffer.*** Let the indices wrap around: the cell after `CAP-1` is `0`. With `front` and `size`, item $i$ of the queue is `items[(front+i) % CAP]`; the rear item is $i = \texttt{size} - 1$ and the next free cell is `(front+size) % CAP`.

- `enqueue(x)`: `items[(front+size)%CAP] = x; size++;`
- `dequeue()`: `x = items[front]; front = (front+1)%CAP; size--;`

Every operation is $\Theta(1)$, and all `CAP` cells are usable.

**Listing 5.1:** The circular queue used in Parts II and III (second part of `stackqueue.h`).

```
typedef struct {
    int items[100];
    int front;          // index of the first item
    int size;
} queue;                // circular: item i is at items[(front + i) % 100]

void qInit(queue *q) { q->front = 0; q->size = 0; }
int qEmpty(queue *q) { return q->size == 0; }
int qSize(queue *q) { return q->size; }

// Returns 1 on success, 0 if the queue is full.
```

Juevesano D. 2026 All rights reserved. 

```c
int qEnq(queue *q, int x) {
    if (q->size == 100) return 0;
    q->items[(q->front + q->size) % 100] = x;
    q->size++;
    return 1;
}

// Removes the front into *x. Returns 1 on success, 0 if the queue is empty.
int qDeq(queue *q, int *x) {
    if (q->size == 0) return 0;
    *x = q->items[q->front];
    q->front = (q->front + 1) % 100;
    q->size--;
    return 1;
}

// Copies the front into *x without removing it. Returns 0 if empty.
int qFront(queue *q, int *x) {
    if (q->size == 0) return 0;
    *x = q->items[q->front];
    return 1;
}
```

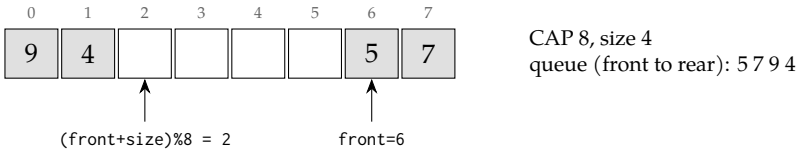



**Figure 5.1:** A circular queue that has wrapped around. The next `enqueue` writes cell 2.

> FULL OR EMPTY?. Some textbooks store `front` and `rear` instead of `size`. Then `front == rear` happens both when the queue is empty and when it is full. The usual cure is to leave one cell always unused: the queue is empty when `front == rear` and full when `(rear+1)%CAP == front`, so the capacity is `CAP-1`. Storing the size (as here) avoids the problem and uses every cell. Read which convention a question uses before you compute "is it full?".

***Growing.*** When the circular buffer is full, a growable version allocates an array of twice the size and copies item $i$ to cell $i$ of the new array (not to the same index!), so that the wrapped part is straightened; then `front = 0`. As with the vector, the cost is $\Theta(1)$ amortised per operation.

## 5.3 Linked implementations

In a singly linked list with head and tail, removal is $\Theta(1)$ only at the head and insertion is $\Theta(1)$ at both ends (Chapter 3). Hence the *front* of a linked queue is the *head* and the *rear* is the *tail*: enqueue is `pushBack`, dequeue is `popFront`. The opposite orientation would need `popBack`, which is $\Theta(n)$. Two details matter:

- dequeueing the last item must also set `tail = NULL`, otherwise the next enqueue writes through a dangling pointer;
- enqueueing into an empty queue must set *both* `head` and `tail`.

Juevesano D. 2026 All rights reserved. 

The circular list of Section 3.3 gives a queue with a single pointer `last` (Listing 3.5, Tracing 24).

| | front at index 0 | circular buffer | linked (head = front) |
|---|---|---|---|
| `enqueue` | $\Theta(1)$ | $\Theta(1)$ (amortised if growable) | $\Theta(1)$ |
| `dequeue` | $\Theta(n)$ | $\Theta(1)$ | $\Theta(1)$ |
| `front`, `size` | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ |

**Table 5.1:** Queue implementations.

## 5.4 Queues built from stacks, and stacks from queues

***A queue from two stacks.*** Push every new item on a stack `in`. To dequeue, take the top of a second stack `out`; if `out` is empty, first pop *all* of `in` and push the items onto `out`. The transfer reverses the order, so the oldest item ends on top of `out`.

**Listing 5.2:** A queue from two stacks (Problem 43, Tracing 12).

```
// A queue built from two stacks. Items enter through "in" and leave through
// "out"; moving everything from in to out reverses the order, which turns
// last-in-first-out into first-in-first-out.
typedef struct {
    stack in;
    stack out;
} twoStackQueue;

void tsqInit(twoStackQueue *q) {
    stInit(&q->in);
    stInit(&q->out);
}

void tsqPush(twoStackQueue *q, int x) {
    stPush(&q->in, x);
}

// Moves items only when out is empty, so each item moves at most once.
void shift(twoStackQueue *q) {
    if (stEmpty(&q->out)) {
        int x;
        while (stPop(&q->in, &x)) stPush(&q->out, x);
    }
}

// Removes the front into *x. Returns 0 if the queue is empty.
int tsqPop(twoStackQueue *q, int *x) {
    shift(q);
    return stPop(&q->out, x);
}

// Copies the front into *x without removing it. Returns 0 if empty.
int tsqPeek(twoStackQueue *q, int *x) {
    shift(q);
    return stTop(&q->out, x);
```

Juevesano D. 2026 All rights reserved. 

```
}

int tsqEmpty(twoStackQueue *q) {
    return stEmpty(&q->in) && stEmpty(&q->out);
}
```

**Theorem 5.2** (Two-stack queue)**.** *Starting from an empty queue, any sequence of $m$ operations of Listing 5.2 performs at most $4m$ stack operations. Hence each queue operation costs $\Theta(1)$ amortised, although a single dequeue may cost $\Theta(n)$.*

*Proof.* Follow one item. It is pushed onto `in` once, popped from `in` once, pushed onto `out` once and popped from `out` once: at most four stack operations, because items never go back from `out` to `in`. The items are at most $m$ in number, and besides moving items each queue operation does $O(1)$ work (one failed pop, one test), so the total is $O(m)$. □

The order is correct because `shift` moves items *only when* `out` *is empty*. Moving whenever `in` is non-empty would place newer items on top of older ones still in `out`, and the next dequeue would return a newer item first.

***A stack from one queue.*** After each enqueue of $x$, rotate the $n-1$ older items (dequeue and re-enqueue each) so that $x$ is at the front. Then the front is always the top: push is $\Theta(n)$, pop and top are $\Theta(1)$ (Problem 44). The other choice — push $\Theta(1)$, and pop rotates $n-1$ items to bring the newest to the front — costs the same in total.

## 5.5 Applications

### *5.5.1 Rotation and the Josephus problem*

Dequeueing an item and enqueueing it again moves it from the front to the rear: *a queue is a circle*. In the Josephus problem $n$ people stand in a circle, numbered $1..n$; counting starts at person 1, and every $k$-th person leaves the circle. Which one remains? Simulate: rotate $k-1$ people, then remove the next.

**Listing 5.3:** Josephus by simulation, $\Theta(nk)$, and by the recurrence, $\Theta(n)$ (Problem 45).

```
// n people stand in a circle, numbered 1..n. Starting at 1, every k-th
// person is removed until one is left. Returns that person's number.
int josephus(int n, int k) {
    queue q;
    int x;
    qInit(&q);
    for (int i = 1; i <= n; i++) qEnq(&q, i);

    while (qSize(&q) > 1) {
        for (int i = 1; i < k; i++) {    // the first k-1 are skipped:
            qDeq(&q, &x);                // they go to the back
            qEnq(&q, x);
        }
        qDeq(&q, &x);                    // the k-th is removed
    }
    qDeq(&q, &x);
    return x;
}
```

Juevesano D. 2026 All rights reserved. 

```
19
20 // Bonus, O(n) time and O(1) space. With i people the survivor's 0-based
21 // position is (survivor with i-1 people + k) % i.
22 int josephusFast(int n, int k) {
23     int pos = 0;                          // one person: position 0
24     for (int i = 2; i <= n; i++) pos = (pos + k) % i;
25     return pos + 1;
26 }
```

**Proposition 5.3** (Josephus recurrence)**.** *Number the positions* $0..i-1$*, starting at the person where counting begins, and let* $J(i)$ *be the position of the survivor among* $i$ *people. Then* $J(1) = 0$ *and* $J(i) = (J(i-1) + k) \bmod i$ *for* $i \geq 2$*.*

*Proof.* With $i$ people the first person removed is at position $(k-1) \bmod i$. Counting restarts at the next person, position $k \bmod i$; renumber the remaining $i-1$ people from there. The person with new number $p$ had old number $(p+k) \bmod i$. The survivor has new number $J(i-1)$, hence old number $(J(i-1)+k) \bmod i$. □

For $n = 7, k = 3$ the removal order is $3, 6, 2, 7, 5, 1$ and the survivor is 4; the recurrence gives $J = 0, 1, 1, 0, 3, 0, 3$ for $i = 1..7$, and $J(7) + 1 = 4$.

### 5.5.2 *Sliding windows over a stream*

When items arrive in time order and only those of the last $w$ time units matter, the items that expire are always the *oldest*: they leave from the front, new ones join at the rear. The queue holds exactly the current window (*recent counter*, Problem 47). For a window of $k$ consecutive array positions, a queue of *indices* of interesting items suffices; an index leaves when it falls out of the window (*first negative in each window*, Problem 49). Each index enters and leaves once, so the total cost is $\Theta(n)$, as for the monotonic stack.

When the question is the *maximum* or *minimum* of each window, items must also leave from the *rear*: this needs a deque (Section 6.5).

### 5.5.3 *Combining a queue with a stack*

A stack reverses, a queue preserves order. Pouring part of a queue into a stack and back reverses that part. Reversing the first $k$ items of a queue (Problem 46): dequeue $k$ items onto a stack, pop them back into the queue (now they are reversed, but at the rear), then rotate the remaining $n - k$ items once around. Interleaving the two halves of a queue (Problem 48) is solved the same way with a few more passes. In such problems, remember that `qSize(&q)` *changes* while the loop runs: read it into a variable first.

Juevesano D. 2026 All rights reserved. 

| If the problem says ... | think of ... |
|---|---|
| in order of arrival, first come first served, round robin | a plain queue |
| every $k$-th, circle, rotate | dequeue + enqueue (45) |
| last $w$ milliseconds, recent, expire | queue of time stamps (47) |
| something about each window of size $k$ | queue of indices (49); deque if min/max (53) |
| reverse part of a queue | a stack as helper (46) |
| implement X using only Y | Section 5.4 (43, 44) |

**Table 5.2:** Recognising a queue problem.

Juevesano D. 2026 All rights reserved. 

Chapter 6

# Deques

## 6.1 The Deque ADT

**Definition 6.1** (Deque)**.** A *deque* (double-ended queue, pronounced "deck") is a sequence in which insertions and deletions may take place at both ends. Its operations are `push_front(x)`, `push_back(x)`, `pop_front()`, `pop_back()`, `front()`, `back()`, `empty()` and `size()`.

A deque generalises both earlier structures. Using only `push_back` and `pop_back` it is a stack; using only `push_back` and `pop_front` it is a queue. Two restricted forms appear in textbooks: an *input-restricted* deque (insertion at one end only, deletion at both) and an *output-restricted* deque (insertion at both ends, deletion at one only).

**Example 6.2.** Starting from an empty deque, the operations `push_back(1)`, `push_front(2)`, `push_back(3)`, `push_front(4)` produce `4 2 1 3`. Then `pop_back()` returns 3 and `pop_front()` returns 4, leaving `2 1`.

## 6.2 The circular-array deque

The circular buffer of Chapter 5 already supports `push_back` and `pop_front` in $\Theta(1)$. The other two operations are the mirror images:

- `push_front(x)`: step `front` *back* by one cell, wrapping from 0 to `CAP-1`, then write `x` there and increase `size`;
- `pop_back()`: the last item is at `(front+size-1)%CAP`; read it and decrease `size` (`front` does not change).

**Listing 6.1:** The circular-array deque used in Parts II and III (file `deque.h`).

```c
typedef struct {
    int items[100];
    int front;          // index of the first item
    int size;
} deque;                // circular: item i is at items[(front + i) % 100]

void dqInit(deque *d) { d->front = 0; d->size = 0; }
int dqEmpty(deque *d) { return d->size == 0; }
int dqSize(deque *d) { return d->size; }

// Adds x before the first item. Returns 0 if the deque is full.
int dqPushFront(deque *d, int x) {
    if (d->size == 100) return 0;
    d->front = (d->front - 1 + 100) % 100;   // one step back, wrapping to 99
    d->items[d->front] = x;
    d->size++;
```

Juevesano D. 2026 All rights reserved. 

```c
    return 1;
}

// Adds x after the last item. Returns 0 if the deque is full.
int dqPushBack(deque *d, int x) {
    if (d->size == 100) return 0;
    d->items[(d->front + d->size) % 100] = x;
    d->size++;
    return 1;
}

// Removes the first item into *x. Returns 0 if the deque is empty.
int dqPopFront(deque *d, int *x) {
    if (d->size == 0) return 0;
    *x = d->items[d->front];
    d->front = (d->front + 1) % 100;
    d->size--;
    return 1;
}

// Removes the last item into *x. Returns 0 if the deque is empty.
int dqPopBack(deque *d, int *x) {
    if (d->size == 0) return 0;
    *x = d->items[(d->front + d->size - 1) % 100];
    d->size--;
    return 1;
}

// Copy the first / last item into *x without removing it. 0 if empty.
int dqFront(deque *d, int *x) {
    if (d->size == 0) return 0;
    *x = d->items[d->front];
    return 1;
}

int dqBack(deque *d, int *x) {
    if (d->size == 0) return 0;
    *x = d->items[(d->front + d->size - 1) % 100];
    return 1;
}
```

> THE MODULO TRAP. In C and C++ the result of `%` has the sign of the left operand, because integer division truncates toward zero: `-7 / 2 == -3` and `-1 % 5 == -1`, not 4. Hence `front = (front - 1) % CAP` sets `front` to $-1$ when `front` was 0, and the next write lands outside the array. Always add `CAP` first: `(front - 1 + CAP) % CAP`, or equivalently `(front + CAP - 1) % CAP`. With a fixed capacity 5 the same step is often written `(front + 4) % 5` (Tracing 21). In Python, `-1 % 5` is 4; do not carry that habit into C.

## 6.3 The ArrayDeque: positional access in a circular array

The course notes (after Morin's *ArrayDeque*) go one step further and support the whole List interface — `get(i)`, `set(i,x)`, `add(i,x)`, `remove(i)` — in a circular array with fields `j` (the index of item 0) and `n` (the size). Item $i$ lives at `a[(j+i)%CAP]`.

Juevesano D. 2026 All rights reserved. 

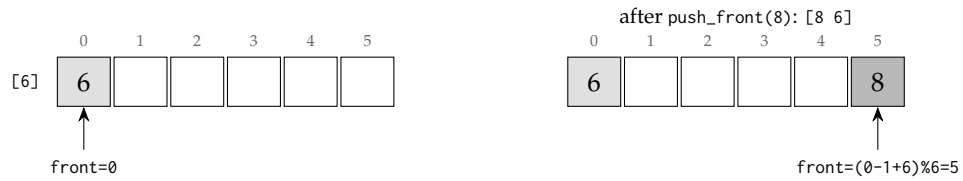



**Figure 6.1:** `push_front` on a circular deque of capacity 6 whose front is cell 0: the new front is the *last* cell.

***Shift the shorter side.*** To insert at position $i$, an array list moves the $n - i$ items to the right of $i$. In a circular array there is a second option: move the $i$ items to the *left* of $i$ one cell to the left, after stepping `j` back by one. The ArrayDeque chooses the side with fewer items: the left side if $i < n/2$, otherwise the right side. Removal is symmetric.

**Listing 6.2:** ArrayDeque with positional insertion and removal (Problem 56, Tracing 22).

```
// An array deque with positional add and remove (after the course notes).
// Item i lives at a[(j + i) % 100]; an insertion or a removal moves the
// SHORTER side, so at most min(i, n - i) items are moved.
typedef struct {
    int a[100];
    int j;                          // index of item 0
    int n;                          // number of items
} arrayDeque;

void adInit(arrayDeque *d) { d->j = 0; d->n = 0; }

int adGet(arrayDeque *d, int i) { return d->a[(d->j + i) % 100]; }

// Inserts x so that it becomes item i (0 <= i <= n). Returns 0 on failure.
int adAdd(arrayDeque *d, int i, int x) {
    if (d->n == 100 || i < 0 || i > d->n) return 0;
    if (i < d->n / 2) {                     // move items 0..i-1 one cell left
        d->j = (d->j - 1 + 100) % 100;
        for (int k = 0; k < i; k++)
            d->a[(d->j + k) % 100] = d->a[(d->j + k + 1) % 100];
    } else {                                // move items i..n-1 one cell right
        for (int k = d->n; k > i; k--)
            d->a[(d->j + k) % 100] = d->a[(d->j + k - 1) % 100];
    }
    d->a[(d->j + i) % 100] = x;
    d->n++;
    return 1;
}

// Removes item i into *x (0 <= i < n). Returns 0 on failure.
int adRemove(arrayDeque *d, int i, int *x) {
    if (i < 0 || i >= d->n) return 0;
    *x = d->a[(d->j + i) % 100];
    if (i < d->n / 2) {                     // move items 0..i-1 one cell right
        for (int k = i; k > 0; k--)
            d->a[(d->j + k) % 100] = d->a[(d->j + k - 1) % 100];
        d->j = (d->j + 1) % 100;
    } else {                                // move items i+1..n-1 one cell left
```

Juevesano D. 2026 All rights reserved. 

```
        for (int k = i; k < d->n - 1; k++)
            d->a[(d->j + k) % 100] = d->a[(d->j + k + 1) % 100];
    }
    d->n--;
    return 1;
}
```

**Theorem 6.3** (Cost of ArrayDeque)**.** `get` *and* `set` *cost* $\Theta(1)$. `add(i,x)` *and* `remove(i)` *move at most* $\min(i, n-i)$ *items and cost* $O(1+\min(i, n-i))$. *In particular all four operations at either end* ($i = 0$ *or* $i = n$, *resp.* $i = n-1$) *are* $\Theta(1)$, *and the worst case, in the middle, is about* $n/2$ *moves — half that of an array list.*

*Proof.* If $i < n/2$ the code moves the $i$ items before position $i$; otherwise $i \geq n/2$ and it moves the $n-i$ items from position $i$ onward. In both cases the number moved is $\min(i, n-i)$ up to one, and each move is one assignment. □

## 6.4 The linked deque

A singly linked list cannot be a deque with all operations in $\Theta(1)$: `pop_back` needs the predecessor of the tail. A doubly linked list can. With a *sentinel* (Section 3.2.1) closed into a circle — `sentinel.next` is the first item, `sentinel.prev` the last, and both point to the sentinel itself when the deque is empty — every insertion is "link a node between two neighbours" and every removal is "unlink a node from its two neighbours". No pointer is ever `NULL`, so the four operations have no special cases at all.

**Listing 6.3:** A linked deque with an embedded sentinel (Problem 51, Tracing 25).

```
// A deque on a circular doubly linked list with one sentinel node.
// sentinel.next is the first item, sentinel.prev the last; when the deque is
// empty both point back to the sentinel, so no pointer is ever NULL.
typedef struct dqnode {
    int item;
    struct dqnode *prev;
    struct dqnode *next;
} dqnode;

typedef struct {
    dqnode sentinel;                // inside the struct: never copy a
    int size;                       // linkedDeque, always pass its address
} linkedDeque;

void ldInit(linkedDeque *d) {
    d->sentinel.next = &d->sentinel;
    d->sentinel.prev = &d->sentinel;
    d->size = 0;
}

int ldEmpty(linkedDeque *d) { return d->size == 0; }

// Links a new node with item x between the neighbours a and b.
int ldLinkBetween(linkedDeque *d, dqnode *a, dqnode *b, int x) {
    dqnode *n = (dqnode *)malloc(sizeof(dqnode));
    if (n == NULL) return 0;
```

```
    n->item = x;
    n->prev = a;
    n->next = b;
    a->next = n;
    b->prev = n;
    d->size++;
    return 1;
}

// Unlinks node n, stores its item in *x and frees it.
void ldUnlink(linkedDeque *d, dqnode *n, int *x) {
    *x = n->item;
    n->prev->next = n->next;
    n->next->prev = n->prev;
    free(n);
    d->size--;
}

int ldPushFront(linkedDeque *d, int x) {
    return ldLinkBetween(d, &d->sentinel, d->sentinel.next, x);
}

int ldPushBack(linkedDeque *d, int x) {
    return ldLinkBetween(d, d->sentinel.prev, &d->sentinel, x);
}

int ldPopFront(linkedDeque *d, int *x) {
    if (ldEmpty(d)) return 0;
    ldUnlink(d, d->sentinel.next, x);
    return 1;
}

int ldPopBack(linkedDeque *d, int *x) {
    if (ldEmpty(d)) return 0;
    ldUnlink(d, d->sentinel.prev, x);
    return 1;
}

int ldFront(linkedDeque *d, int *x) {
    if (ldEmpty(d)) return 0;
    *x = d->sentinel.next->item;
    return 1;
}

int ldBack(linkedDeque *d, int *x) {
    if (ldEmpty(d)) return 0;
    *x = d->sentinel.prev->item;
    return 1;
}

void ldFree(linkedDeque *d) {
    int x;
    while (ldPopFront(d, &x)) {}
}
```

Juevesano D. 2026 All rights reserved.

> NEVER COPY A STRUCT THAT CONTAINS ITS OWN SENTINEL. Here the sentinel is a member of the `linkedDeque` struct, and the first and last nodes point at its address. A copy `linkedDeque e = d;` (or passing the struct by value) creates a second sentinel at a different address whose `next` and `prev` still lead into `d`'s nodes, while those nodes still point back to `d`'s sentinel. Walking `e` from its sentinel never returns to it. Always pass `linkedDeque *`. (A sentinel allocated with `malloc`, as in Listing 3.4, avoids this particular trap but still shares the nodes.)

| | circular array | ArrayDeque | doubly linked + sentinel |
|---|---|---|---|
| push/pop at either end | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ |
| `get(i)`, `set(i,x)` | $\Theta(1)$ | $\Theta(1)$ | $\Theta(\min(i, n-i))$ |
| `add(i,x)`, `remove(i)` | — | $\Theta(1+\min(i, n-i))$ | $\Theta(\min(i, n-i)) + \Theta(1)$ link |
| memory per item | one cell | one cell | item + two pointers |

**Table 6.1:** Deque implementations. A doubly linked list walks from the nearer end to reach position $i$.

## 6.5 The monotonic deque

***Problem.*** Given an array of $n$ values and a window size $k$, report the maximum of every window `a[i-k+1..i]`. Recomputing each maximum costs $\Theta(nk)$. A queue of the window is not enough: when the maximum leaves the window, the next maximum is unknown.

***Idea.*** Keep a deque of *indices* of the current window whose values *decrease* from front to back. When `a[i]` arrives:

1. if the front index has slid out of the window ($\leq i-k$), pop it from the *front*;
2. pop from the *back* every index $j$ with `a[j]` $\leq$ `a[i]`: such $j$ is older *and* not larger than $i$, so it can never again be the maximum of a window that contains $i$;
3. push $i$ at the back; the front is now the index of the maximum of the window.

The back behaves like the monotonic stack of Section 4.4, the front like the queue of a sliding window. A deque is exactly the structure that does both.

**Listing 6.4:** Sliding window maximum (Problem 53, Tracing 23).

```
// out[i] = max(a[i..i+k-1]) for every window of size k. Returns n - k + 1.
// [1 3 -1 -3 5 3 6 7], k = 3 -> [3 3 5 5 6 7]
// The deque holds indices of the current window whose values decrease from
// front to back; the front is always the index of the window maximum.
int maxSlidingWindow(int a[], int n, int k, int out[]) {
    deque d;
    int j, m = 0;
    dqInit(&d);

    for (int i = 0; i < n; i++) {
        // 1. the front leaves when it slides out of the window a[i-k+1..i]
        if (dqFront(&d, &j) && j <= i - k) dqPopFront(&d, &j);
        // 2. smaller values at the back can never be a maximum again
        while (dqBack(&d, &j) && a[j] <= a[i]) dqPopBack(&d, &j);
        dqPushBack(&d, i);
        // 3. once the first window is complete, its maximum is at the front
        if (i >= k - 1) {
```

Juevesano D. 2026 All rights reserved. 

```
            dqFront(&d, &j);
            out[m] = a[j];
            m++;
        }
    }
    return m;
}
```

| $i$ | a[i] | popped | deque after (indices: values) | output |
|---|---|---|---|---|
| 0 | 1 | — | 0:1 | |
| 1 | 3 | back 0 | 1:3 | |
| 2 | −1 | — | 1:3 2:−1 | 3 |
| 3 | −3 | — | 1:3 2:−1 3:−3 | 3 |
| 4 | 5 | front 1; back 3, 2 | 4:5 | 5 |
| 5 | 3 | — | 4:5 5:3 | 5 |
| 6 | 6 | back 5, 4 | 6:6 | 6 |
| 7 | 7 | back 6 | 7:7 | 7 |

**Table 6.2:** Sliding window maximum on $[1, 3, -1, -3, 5, 3, 6, 7]$ with $k = 3$.

**Theorem 6.4** (Sliding window maximum)**.** *The algorithm is correct and runs in $\Theta(n)$ time and $O(k)$ extra space.*

*Proof. Invariant*: after round $i$ the deque contains, in increasing order of index, exactly those indices $j$ of the window $[i - k + 1, i]$ for which a[j] is *greater* than every later value of the window; their values therefore decrease from front to back, and the front holds the window maximum (the maximum is greater than every later value, ties going to the latest). Step 1 removes the only index that can have left the window, because the window advances by one. Step 2 removes exactly the indices whose value is $\leq$ a[i], which are the ones no longer greater than every later value. Step 3 adds $i$, trivially greater than every later value. *Cost*: each index is pushed once and popped at most once (from one end or the other), so the loops of all rounds together do at most $2n$ deque operations. The deque never holds more than $k$ indices. □

***Variants.***

- *Window minimum*: reverse the comparison; values increase from front to back.
- *Variable window* (two pointers): the window a[left..right] grows at the right and shrinks at the left. Keeping one decreasing deque (maximum) and one increasing deque (minimum) answers "longest subarray with $\max - \min \leq limit$" in $\Theta(n)$ (Problem 54).
- *Dynamic programming over a window*: when $best[i] = a[i] + \max(best[i-k..i-1])$, the deque keeps the indices of the last $k$ values of *best*, and the whole table costs $\Theta(n)$ instead of $\Theta(nk)$ (Jump Game VI, Problem 57).

## 6.6 Further applications

***Palindromes.*** Push the characters into a deque; while at least two are left, pop one from each end and compare (Problem 52). The deque makes the "two pointers from both ends"

Juevesano D. 2026 All rights reserved. 

pattern explicit.

***Two deques back to back.*** A sequence split into a left half and a right half, each in its own deque, allows $\Theta(1)$ access to the *middle* as well as to the ends: the middle is the back of the left deque or the front of the right one. After every operation one item at most has to cross the boundary to restore the balance $|right| - |left| \in \{0, 1\}$ (front-middle-back queue, Problem 55). The same trick with a single middle pointer on a linked list is Problem 24.

***Undo with a limit.*** An editor that keeps only the last 100 actions pushes new actions at the back, undoes from the back (a stack), and discards the oldest action from the *front* when the limit is exceeded. Neither a stack nor a queue suffices; a deque does.

## 6.7 The standard library

In C++ the containers below implement the ADTs of this volume. Examinations that permit the STL accept them; examinations that ask you to *implement* an ADT do not.

| Container | Main operations (all $\Theta(1)$ unless noted) | Notes |
|---|---|---|
| `vector<int>` | `push_back`, `pop_back`, `back`, `[i]`, `size`; `insert`/`erase` at an iterator $\Theta(n)$ | amortised push; growth invalidates iterators |
| `list<int>` | `push_front/back`, `pop_front/back`, `front`, `back`, `insert`/`erase` at an iterator, `splice` | doubly linked with sentinel; no `[i]` |
| `forward_list<int>` | `push_front`, `pop_front`, `insert_after`, `erase_after` | singly linked; no `size()` |
| `deque<int>` | `push_front/back`, `pop_front/back`, `front`, `back`, `[i]`; middle `insert` $\Theta(n)$ | blocks of arrays |
| `stack<int>` | `push`, `pop`, `top`, `empty`, `size` | adapter over `deque` |
| `queue<int>` | `push`, `pop`, `front`, `back`, `empty`, `size` | adapter over `deque` |

**Table 6.3:** STL containers. In all of them `pop`, `pop_front` and `pop_back` return `void`: read `top()`/`front()`/`back()` first, and never call them on an empty container (undefined behaviour, not an error code).

| If the problem says ... | think of ... |
|---|---|
| both ends, front and back | a deque |
| maximum / minimum of every window | monotonic deque of indices (53) |
| longest / shortest window with a max–min condition | two monotonic deques + two pointers (54) |
| $best[i]$ from the best of the last $k$ | DP + monotonic deque (57) |
| compare from both ends | pop front and back (52) |
| operations in the middle | two balanced deques (55) |
| design a circular deque | the modulo trap (50) |

**Table 6.4:** Recognising a deque problem.

Juevesano D. 2026 All rights reserved. 

Part II

# Practice Examination

Juevesano D. 2026 All rights reserved. 

Chapter 7

# Instructions and Marking Scheme

This part simulates the departmental written examination. It has three sections, in the order in which they are usually set.

| Section | Chapter | Content |
|---|---|---|
| A. Concepts | 8 | short written answers: definitions, complexities, small proofs and hand simulations |
| B. Tracing | 9 | read a program and state its exact output; most items hide a twist |
| C. Problem solving | 10 | write code against a specification, with test cases, permitted operations and a required time complexity |

## 7.1 How to use this part

(i) *Attempt each item on paper first.* Tracing is a paper skill; reading the answer teaches little.

(ii) *Time yourself.* A realistic budget is 2–3 minutes per concept item, 5–8 minutes per tracing item and 20–30 minutes per easy-to-medium problem.

(iii) *Check against Part III*, which gives the real program output, the reasoning, and for problems an idea, a complete commented solution, its complexity and the usual mistakes.

(iv) *Run the code.* Every program in the book was compiled and run; retyping the solutions is excellent practice, and altering the inputs shows what each line does.

## 7.2 Conventions used in the problem statements

- **Permitted operations.** Each problem lists the operations you may call. Anything else must be written by you, from the representation, if the statement allows it. Calling an operation that is not listed (for instance indexing the array inside a stack when only push and pop are permitted) forfeits the credit even if the output is correct.
- **Starter code and constraints.** The box under the statement is the function (or the few functions) you have to write; the types and helper operations it uses are those of the cited listing. The constraints give the sizes the tests may reach.
- **Required.** The time (and sometimes space) complexity needed for full credit. A slower correct solution earns partial credit.
- **Bonus.** Extra credit for beating the required bound, or for an additional property (in place, single pass, $O(1)$ extra space).

Juevesano D. 2026 All rights reserved. 

- **Tests.** Sample inputs and expected outputs. They are illustrations, not the full test set: a real judge adds the empty case, the one-element case, duplicates, the maximum size, a full fixed-capacity structure, indices that wrap around, and adversarial orderings.
- $n$ is the number of items. Positions are 0-based unless the statement says otherwise.
- Unless stated otherwise, you may assume the memory returned by `malloc/new` is available, and you must not leak nodes that your code removes.

### 7.3 A sample marking scheme

The departmental scheme differs from term to term; the following is a plausible shape, useful for self-assessment.

| Criterion | Share |
|---|---|
| Correct on all tests, including edge cases | 50% |
| Meets the required complexity | 20% |
| Uses only permitted operations (abstraction respected) | 10% |
| No memory errors, no leaks; `head/tail/size/front` consistent | 10% |
| Readability and comments | 10% |
| Bonus for a faster or leaner solution | up to +20% of the item |

***Exam technique.***

(i) Write the invariant or the picture before the loop. A node-and-arrow sketch of the three cases (empty, one node, general), or an array with `front` and `size` marked, catches most bugs.

(ii) Handle the edge cases explicitly, with their own `if`: empty structure, one element, first, last, not found, duplicates, everything equal, full, wrap-around.

(iii) State the complexity in a comment. If you know a faster approach but cannot finish it, submit the correct slower one and say so.

(iv) Never call a library routine that does the very thing being asked (e.g. `std::reverse` in a problem about reversing); do not depend on headers beyond `stdio/stdlib/iostream` unless the statement permits.

(v) For a stack, queue or deque problem, first ask *which end* each item leaves from. "The most recent" means a stack; "the oldest" means a queue; "both", or "the oldest and also the most recent", means a deque.

Juevesano D. 2026 All rights reserved. 

Chapter 8

# Section A: Concept Questions

Answer briefly. Where a proof or a trace is requested, show it. Model answers are in Chapter 11; the numbers agree.

### Foundations and array lists

**1.** Define *abstract data type*. State the difference between an ADT and a data structure, and name the three ADTs of this coverage that are specialisations or relatives of the List.

**2.** Prove or disprove: (a) $23 = O(\log n)$; (b) $5 \log n = O(1)$; (c) $5n - 7 = O(n)$. Give constants $c, n_0$ where the claim is true.

**3.** State the tightest running time, in $\Theta$-notation, of each operation. (a) `push_back` on an array list that is not full; (b) insert at the front of an array list; (c) `push_front` on an SLL; (d) `pop_back` on an SLL with head and tail pointers; (e) `pop_back` on a DLL with head and tail; (f) worst-case `get(i)` on a DLL; (g) searching an unsorted list; (h) `push_front` on a circular-array deque that is not full.

**4.** A function `void f(list l)` receives an array-list struct (with an array inside it) by value and sets `l.items[0]=100`. Does the caller see the change? What if `l` is a struct with a `head` pointer to nodes, and the function sets `l.head->item=100`? What if the parameter is a raw array `int a[]`? Explain each answer.

**5.** After `erase(pos)` on an array list, the old last cell still holds a stale copy of the last value. Is this a bug? Which field makes the value harmless, and when would the stale value matter?

**6.** Distinguish `malloc/free` from `new/delete`. Define a *dangling pointer* and a *memory leak* and give a two-line example of each.

**7.** A function `void append(node *h, int x)` correctly appends to a non-empty list but silently fails when the list is empty. Why? Give two different repairs.

**8.** An array list currently holds $s$ items. How many items are moved by `insert(p,x)` (positions $0 \le p \le s$) and by `erase(p)`? For which $p$ is each worst, and what is that worst case?

**9.** A dynamic array doubles its capacity when full. Show that $n$ successive `push_back` calls cost $O(n)$ in total. Why does growing by a constant $+10$ instead give $\Theta(n^2)$ total?

**10.** Why is it a mistake to halve the capacity as soon as $n \le cap/2$? Describe a bad sequence and a safe shrinking rule.

Juevesano D. 2026 All rights reserved. 

**11.** Binary search on a sorted array list needs what precondition and takes what time? Explain why a sorted-list insert nevertheless remains $\Theta(n)$ in the worst case even though the position is found in $O(\log n)$.

**12.** In the circular-array queue with fields (`front`, `size`) and capacity `CAP`: write the index of the front item, of the back item, and of the next free cell. Which values of (`front`, `size`) with `CAP` $= 4$ describe a full queue whose back item is at index 2?

### Linked lists

**13.** For which operations does adding a tail pointer to a head-only SLL improve the running time, and from what to what? Which operation stays $\Theta(n)$ even with a tail pointer?

**14.** Explain precisely why `pop_back` on an SLL is $\Theta(n)$ even though a tail pointer is available.

**15.** A DLL node `c` must be inserted between neighbouring nodes `a` and `b` (with `a->next==b`). List every pointer assignment.

**16.** List the four cases that occur when removing a node `cur` from a DLL with `head/tail` pointers, and the assignments for each.

**17.** What is a sentinel (dummy) node? Name two special cases it removes from list code, and one cost.

**18.** Describe the slow–fast pointer technique and use it to find the middle node of an SLL in one pass. What is returned for a list of even length, with the usual loop condition?

**19.** State the loop invariant of the three-pointer in-place reversal of an SLL, and the running time and extra space.

**20.** Define *shallow copy* and *deep copy* of a linked list. If `l = m;` is executed for two list structs and `m` is then destroyed node by node, what happens to `l`?

**21.** What is *splicing*? Which pointers change when the whole of list `m` is spliced after node `p` of list `l`, and why is the cost $\Theta(1)$ only if some pointer is already known?

**22.** An SLL keeps `head`, `tail` and `mid`. State the invariant for `mid`. Which of `push_front`, `push_back`, `pop_front`, `pop_back`, `middle` can be $\Theta(1)$ and which not? Which change to the node type would make all of them $\Theta(1)$?

### Stacks

**23.** Define the stack ADT and give three applications. Which end of a singly linked list should serve as the top, and why?

**24.** Explain why a queue built from two stacks has amortised $\Theta(1)$ cost per operation although a single dequeue may cost $\Theta(n)$.

**25.** Show with the string `([)]` that counting the opening and closing brackets of each kind is not enough to test for balance, and say what data structure is.

Juevesano D. 2026 All rights reserved. 

**26.** Convert $(a + b) * c - d/e$ to postfix and to prefix.

**27.** Evaluate the postfix expression `5 1 2 + 4 * + 3 -`, showing the stack after each token.

**28.** Convert `a-b*c+d/(e-f)` to postfix with the shunting-yard algorithm, showing the operator stack after each token. Then explain what changes for `8-3-2` if the algorithm pops only operators of *strictly* higher precedence.

**29.** The numbers $1, 2, 3, 4, 5$ are pushed in this order onto a stack, with pops interleaved arbitrarily. Which of the following output sequences are possible? For each possible one give the push/pop word; for each impossible one name the forbidden pattern. (a) `3 2 5 4 1`; (b) `4 1 3 2 5`; (c) `1 2 3 4 5`; (d) `4 3 5 1 2`. How many orders are possible for $1, 2, 3, 4$?

**30.** Give the next greater element of every item of $[4, 5, 2, 25, 7, 18]$ (or $-1$), the stack after each step of the monotonic-stack algorithm, and an argument that the algorithm is $\Theta(n)$ although it contains a loop inside a loop.

**31.** A min stack stores two stacks `all` and `mins`. Give both after the pushes $5, 3, 7, 3, 1$, then after two pops, and the value of `min()` at the end. Why is it wrong to store only one variable `currentMin`?

## Queues and deques

**32.** A circular queue stores `front` and `rear` (the index *after* the last item) instead of `size`, in an array of `CAP` $= 6$ cells. (a) How many items does it hold when `front` $= 4$, `rear` $= 1$? (b) Why must one cell stay unused, and what are the tests for empty and for full?

**33.** In a linked queue, why is the front the *head* and not the tail? Which field must `dequeue` update when it removes the last item, and what happens on the next `enqueue` if it does not?

**34.** Give the values of the C expressions `-7/2`, `-7%2`, `7%-2`, `(0-1)%5` and `(0-1+5)%5`. Explain the bug in `front = (front - 1) % CAP;` inside `push_front`.

**35.** Starting from an empty deque, perform `push_front(1)`, `push_back(2)`, `push_front(3)`, `pop_back()`, `push_back(4)`, `push_front(5)`, `pop_front()`. Give the value returned by each pop and the final contents from front to back.

**36.** Why can a plain queue not maintain the maximum of a sliding window, while a deque can? Give the window maxima of $[9, 1, 8, 2, 7, 3]$ with $k = 2$, and prove that the monotonic deque algorithm performs at most $2n$ pushes and pops.

**37.** An ArrayDeque holds $n = 10$ items. How many items are moved by (a) `add(2,x)`, (b) `add(7,x)`, (c) `remove(5)`, each applied to the original deque? What is the worst case, and why is it half that of an array list?

**38.** A linked deque keeps its sentinel node *inside* the struct (`linkedDeque{ dqnode sentinel; int size; }`). How many pointer assignments does `push_front` perform on a non-empty deque, and on an empty one? What goes wrong after `linkedDeque e = d;`?

Juevesano D. 2026 All rights reserved. 

**39.** Seven people numbered 1..7 stand in a circle and every third person is removed, counting from person 1. Give the order of removal and the survivor, and check the survivor with the recurrence $J(i) = (J(i-1) + k) \bmod i$.

**40.** For each task name the most suitable ADT (stack, queue or deque) and say why: (a) undo in an editor; (b) a printer spooler; (c) browser history that keeps only the last 50 pages but supports *back*; (d) checking matching HTML tags; (e) the maximum of the last $k$ readings of a sensor; (f) serving customers in order of arrival.

Juevesano D. 2026 All rights reserved. 

Chapter 9

# Section B: Tracing Problems

Determine the exact output of each program, including spaces and line breaks. Each one hides at least one trap; the most common are listed in the checklist below. Unless stated, the code compiles and runs without error. Programs that rely on a library of Part I (`stackqueue.h`, `deque.h`, `sll.h`, `dll.h`, `circular.c`) or on the solution of a problem of Chapter 10 say so, and the behaviour of the functions they call is stated before the program. The headers `stdio.h` and `stdlib.h` are included everywhere and not shown.

***Tracing checklist.*** (i) Evaluation order of function arguments is unspecified; the programs here use sequential statements to avoid it. (ii) Pass by value copies a struct including an embedded array, but a pointer member still points to the same nodes. (iii) A loop bound such as `i<=n` or `i<size-1` changes the count by one. (iv) `printf` adds nothing: look for trailing spaces and `\n`. (v) Integer division truncates toward zero, and `%` keeps the sign of the left operand. (vi) A failed `pop` of the course library returns 0 and leaves its output variable *unchanged*. (vii) Keep a table with one column per variable and one row per step; do not trace in your head.

**Tracing 1.** Array list, return values and stale cells

A bounded array list with capacity 5 and 1-based `setItem` and `itemAt`. Give the exact output of the program.

```c
typedef struct { int items[5]; int size; } list;

void init(list *l) { l->size = 0; }
int append(list *l, int x) {
    if (l->size == 5) return 0;
    l->items[l->size++] = x;
    return 1;
}
int insertFront(list *l, int x) {
    int i;
    if (l->size == 5) return 0;
    for (i = l->size; i > 0; i--) l->items[i] = l->items[i - 1];
    l->items[0] = x;
    l->size++;
    return 1;
}
int removeFirst(list *l) {
    int i;
    if (l->size == 0) return 0;
    for (i = 0; i < l->size - 1; i++) l->items[i] = l->items[i + 1];
    l->size--;
    return 1;
}
```

Juevesano D. 2026 All rights reserved. 

```c
int removeLast(list *l) { if (l->size == 0) return 0; l->size--; return 1; }
int setItem(list *l, int x, int p) {          /* p is 1-based */
    if (p < 1 || p > l->size) return 0;
    l->items[p - 1] = x;
    return 1;
}
int itemAt(list l, int p) { return l.items[p - 1]; }
void display(list l) {
    int i;
    for (i = 0; i < l.size; i++) printf("%d ", l.items[i]);
    printf("\n");
}

int main() {
    list l;
    int r;
    init(&l);
    r = append(&l, 4);
    r = r + append(&l, 8);
    r = r + append(&l, 15);
    printf("%d\n", r);
    r = insertFront(&l, 16);
    r = r * 10 + insertFront(&l, 23);
    printf("%d\n", r);
    printf("%d\n", append(&l, 42));
    display(l);
    removeLast(&l);
    removeFirst(&l);
    display(l);
    r = setItem(&l, 99, 3);
    printf("%d ", r);
    r = setItem(&l, 77, 4);
    printf("%d\n", r);
    printf("%d\n", itemAt(l, 4));
    append(&l, 5);
    display(l);
    printf("%d\n", itemAt(l, 4));
    return 0;
}
```

## Tracing 2. What does the callee really change?

Three structs are passed by value, by pointer and as a raw array. For each of the lines A–E state what is printed.

```c
typedef struct { int items[3]; int size; } alist;          /* array inside   */
typedef struct node { int item; struct node *next; } node;
typedef struct { node *head; int size; } slist;            /* pointer inside */

void arrByValue(alist l) { l.items[0] = 100; l.size = 9; }
void arrByPtr(alist *l)  { l->items[0] = 100; l->size = 9; }
void rawArray(int a[])   { a[0] = 100; }
void llByValue(slist l)  { l.head->item = 100; l.head = NULL; l.size = 0; }

```

Juevesano D. 2026 All rights reserved. 

```
int main() {
    alist a = {{1, 2, 3}, 3};
    int raw[3] = {1, 2, 3};
    slist s;
    node *n2 = (node *)malloc(sizeof(node));
    node *n1 = (node *)malloc(sizeof(node));
    n2->item = 2; n2->next = NULL;
    n1->item = 1; n1->next = n2;
    s.head = n1; s.size = 2;

    arrByValue(a);
    printf("A: %d %d\n", a.items[0], a.size);
    arrByPtr(&a);
    printf("B: %d %d\n", a.items[0], a.size);
    a.items[0] = 1; a.size = 3;
    rawArray(a.items);
    printf("C: %d %d\n", a.items[0], a.size);
    rawArray(raw);
    printf("D: %d\n", raw[0]);
    llByValue(s);
    printf("E: %d %d %d\n", s.head == NULL, s.size, n1->item);
    return 0;
}
```

## **Tracing 3.** Counting loop iterations

Give the six counters. Then state each count as a function of $n$ in $\Theta$-notation.

```
int main() {
    int n = 16, i, j, c1 = 0, c2 = 0, c3 = 0, c4 = 0, c5 = 0, c6 = 0;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            c1++;

    for (i = 0; i < n; i++)
        for (j = 0; j <= i; j++)
            c2++;

    for (i = 1; i < n; i *= 2)
        for (j = 0; j < n; j++)
            c3++;

    for (i = n; i > 0; i /= 2)
        c4++;

    for (i = 0; i < n; i++)
        for (j = i; j < n; j += 4)
            c5++;

    for (i = 1; i < n; i *= 2)
        for (j = 0; j < i; j++)
            c6++;

```

Juevesano D. 2026 All rights reserved. 

```
    printf("%d %d %d %d %d %d\n", c1, c2, c3, c4, c5, c6);
    return 0;
}
```

**Tracing 4.** The raw array after erase and insert

The class prints its *whole* array, including the cells beyond `size`. Positions are 0-based.

```
class ArrayList {
    int items[8];
    int size;
public:
    ArrayList() { size = 0; for (int i = 0; i < 8; i++) items[i] = 0; }
    bool push_back(int x) {
        if (size == 8) return false;
        items[size++] = x;
        return true;
    }
    bool erase(int pos) {                    /* 0-based */
        if (pos < 0 || pos >= size) return false;
        for (int i = pos; i < size - 1; i++) items[i] = items[i + 1];
        size--;
        return true;
    }
    bool insert(int pos, int x) {            /* 0-based */
        if (size == 8 || pos < 0 || pos > size) return false;
        for (int i = size; i > pos; i--) items[i] = items[i - 1];
        items[pos] = x;
        size++;
        return true;
    }
    void raw() {
        printf("size=%d  [", size);
        for (int i = 0; i < 8; i++)
            printf("%d%s", items[i], i < 8 - 1 ? " " : "");
        printf("]\n");
    }
};

int main() {
    ArrayList l;
    for (int i = 1; i <= 6; i++) l.push_back(i * 10);
    l.raw();
    l.erase(1);
    l.raw();
    l.insert(3, 99);
    l.raw();
    int r = l.erase(5);
    r = r + l.erase(5);
    printf("%d\n", r);
    l.raw();
    l.insert(0, 7);
    l.insert(0, 8);
    l.raw();
```

Juevesano D. 2026 All rights reserved. 

```
    r = l.insert(9, 1);
    printf("%d ", r);
    r = l.insert(8, 1);
    printf("%d\n", r);
    l.raw();
    return 0;
}
```

## Tracing 5. Circular queue: wrap-around

A circular queue of capacity 4. `show` prints the fields and the raw array.

```
// circular queue: item i of the queue lives at a[(j + i) % 4]
typedef struct {
    int a[4];
    int j;          // index of the front item
    int n;          // number of items
} cqueue;

int add(cqueue *q, int x) {
    if (q->n == 4) return 0;
    q->a[(q->j + q->n) % 4] = x;
    q->n++;
    return 1;
}

int removeFront(cqueue *q, int *out) {
    if (q->n == 0) return 0;
    *out = q->a[q->j];
    q->j = (q->j + 1) % 4;
    q->n--;
    return 1;
}

void show(cqueue *q) {
    printf("j=%d n=%d ", q->j, q->n);
    printf("[%d %d %d %d]\n", q->a[0], q->a[1], q->a[2], q->a[3]);
}

int main() {
    cqueue q = {{0, 0, 0, 0}, 0, 0};
    int x;
    add(&q, 1); add(&q, 2); add(&q, 3); show(&q);
    removeFront(&q, &x); printf("removed %d\n", x); show(&q);
    add(&q, 4); add(&q, 5); show(&q);
    printf("%d\n", add(&q, 6));
    removeFront(&q, &x); removeFront(&q, &x); show(&q);
    add(&q, 7); add(&q, 8); show(&q);
    while (removeFront(&q, &x)) printf("%d ", x);
    printf("\n");
    show(&q);
    return 0;
}
```

Juevesano D. 2026 All rights reserved. 

### Tracing 6. SLL WITH HEAD, TAIL AND SIZE

The program uses `sll.h` (Listing 3.1) and `popFront/popBack` of Listing 3.2: each pop returns 1 and stores the removed item in `*x`, or returns 0 and leaves `*x` unchanged when the list is empty. `sdisplay` prints the list as `[3 5]`. `%-11s` pads the operation name to 11 characters.

```
void state(char *op, sll *l) {
    printf("%-11s size=%d head=", op, l->size);
    if (l->head) printf("%d", l->head->item); else printf("NULL");
    printf(" tail=");
    if (l->tail) printf("%d", l->tail->item); else printf("NULL");
    printf("  ");
    sdisplay(l);
}

int main() {
    sll l;
    int x = -1;
    sinit(&l);
    pushBack(&l, 5);   state("pushBack 5", &l);
    pushFront(&l, 3);  state("pushFront 3", &l);
    pushBack(&l, 8);   state("pushBack 8", &l);
    popBack(&l, &x);   state("popBack", &l);
    popBack(&l, &x);   state("popBack", &l);
    popFront(&l, &x);  state("popFront", &l);
    int r = popFront(&l, &x);
    printf("%d %d\n", r, x);
    pushFront(&l, 1);  state("pushFront 1", &l);
    pushBack(&l, 2);   state("pushBack 2", &l);
    return 0;
}
```

### Tracing 7. ONE STAR OR TWO?

```
typedef struct node { int item; struct node *next; } node;

void appendA(node *h, int x) {           /* single pointer */
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = NULL;
    if (h == NULL) { h = n; return; }
    while (h->next != NULL) h = h->next;
    h->next = n;
}
void appendB(node **h, int x) {          /* double pointer */
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = NULL;
    if (*h == NULL) { *h = n; return; }
    node *cur = *h;
    while (cur->next != NULL) cur = cur->next;
    cur->next = n;
}
int length(node *h) { return h == NULL ? 0 : 1 + length(h->next); }
```

Juevesano D. 2026 All rights reserved. 

```
19
20 int main() {
21     node *a = NULL, *b = NULL, *c = NULL, *d = NULL;
22     for (int i = 1; i <= 3; i++) { appendA(a, i); appendB(&b, i); }
23     printf("%d %d\n", length(a), length(b));
24     appendB(&c, 7);
25     appendB(&d, 7);
26     for (int i = 1; i <= 3; i++) { appendA(c, i); appendB(&d, i); }
27     printf("%d %d\n", length(c), length(d));
28     return 0;
29 }
```

## Tracing 8. Erasing from a doubly linked list

The program uses `dll.h` (Listing 3.3: `display` prints the items separated by spaces and a newline; `displayBackward` prints them from the tail) and `eraseItems(list *l, int x)` of Problem 13, which removes *every* node holding `x`. The departmental output format ends each item with a space.

```
1  int main() {
2      int v[] = {2, 2, 5, 2, 7, 2, 2}, i;
3      list l;
4      init(&l);
5      for (i = 0; i < 7; i++) append(&l, v[i]);
6      eraseItems(&l, 2);
7      display(&l);
8      printf("size=%d head=%d tail=%d\n", l.size, l.head->item, l.tail->item);
9      displayBackward(&l);
10     eraseItems(&l, 9);
11     printf("size=%d\n", l.size);
12     eraseItems(&l, 7);
13     printf("size=%d head=%d tail=%d ", l.size, l.head->item, l.tail->item);
14     printf("prev=%d\n", l.tail->prev == NULL);
15     eraseItems(&l, 5);
16     printf("size=%d ", l.size);
17     printf("headIsNULL=%d tailIsNULL=%d\n", l.head == NULL, l.tail == NULL);
18     return 0;
19 }
```

## Tracing 9. Recursion on a list

The lists are built with a pointer to the last `next` field. Give the three lines.

```
1  typedef struct node { int item; struct node *next; } node;
2
3  void mystery(node *p) {
4      if (p == NULL) { printf("# "); return; }
5      printf("%d ", p->item);
6      mystery(p->next);
7      printf("%d ", p->item);
8  }
9
```

Juevesano D. 2026 All rights reserved. 

```c
int g(node *p) {
    if (p == NULL) return 0;
    if (p->item % 2 == 0) return p->item + g(p->next);
    return g(p->next) - 1;
}

int h(node *p) {
    if (p == NULL || p->next == NULL) return 0;
    if (p->item < p->next->item) return 1 + h(p->next);
    return h(p->next);
}

int main() {
    int v[] = {1, 2, 3, 4}, w[] = {3, 5, 2, 8, 6, 9}, i;
    node *a = NULL, *b = NULL, **t = &a;
    for (i = 0; i < 4; i++) {          // t: address of the last next field
        *t = (node *)malloc(sizeof(node));
        (*t)->item = v[i];
        (*t)->next = NULL;
        t = &((*t)->next);
    }
    t = &b;
    for (i = 0; i < 6; i++) {
        *t = (node *)malloc(sizeof(node));
        (*t)->item = w[i];
        (*t)->next = NULL;
        t = &((*t)->next);
    }
    mystery(a); printf("\n");
    printf("%d %d\n", g(a), g(b));
    printf("%d %d\n", h(a), h(b));
    return 0;
}
```

**Tracing 10.** Stacks and queues in the same function

The program uses `stackqueue.h` (Listings 4.1 and 5.1): every pop, dequeue, top or front returns 0 and leaves its output variable unchanged when the structure is empty.

```c
int main() {
    stack s;
    queue q;
    int x = -1, y = -1, r;
    stInit(&s);
    qInit(&q);
    stPush(&s, 1); stPush(&s, 2); stPush(&s, 3);
    qEnq(&q, 10); qEnq(&q, 20);
    stPop(&s, &x);
    qDeq(&q, &y);
    printf("%d %d\n", x, y);
    qEnq(&q, x);
    stPush(&s, y);
    r = stPop(&s, &x);
    r = r + stPop(&s, &x);
```

Juevesano D. 2026 All rights reserved. 

```
    r = r + stPop(&s, &x);
    r = r + stPop(&s, &x);
    printf("%d %d\n", r, x);
    r = qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    printf("%d %d\n", r, y);
    stPush(&s, 4); stPush(&s, 5);
    while (!stEmpty(&s)) { stPop(&s, &x); qEnq(&q, x); }
    while (!qEmpty(&q)) { qDeq(&q, &y); stPush(&s, y); }
    while (!stEmpty(&s)) { stPop(&s, &x); printf("%d ", x); }
    printf("\n");
    return 0;
}
```

## Tracing 11. Infix to postfix with a depth counter

A variant of the shunting-yard algorithm on letters. Give the output string and the maximum size the operator stack reaches.

```
int prec(char op) {
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int main() {
    char *e = "a+b*(c-d)/e";
    char out[50];
    int w = 0, t, maxDepth = 0;
    stack st;
    stInit(&st);
    for (int i = 0; e[i]; i++) {
        char c = e[i];
        if (c >= 'a' && c <= 'z') out[w++] = c;
        else if (c == '(') stPush(&st, c);
        else if (c == ')') {
            while (stTop(&st, &t) && t != '(') {
                stPop(&st, &t);
                out[w++] = (char)t;
            }
            stPop(&st, &t);
        } else {
            while (stTop(&st, &t) && prec((char)t) >= prec(c)) {
                stPop(&st, &t);
                out[w++] = (char)t;
            }
            stPush(&st, c);
        }
        if (stSize(&st) > maxDepth) maxDepth = stSize(&st);
    }
    while (stPop(&st, &t)) out[w++] = (char)t;
    out[w] = '\0';
```

Juevesano D. 2026 All rights reserved. 

```
    printf("%s %d\n", out, maxDepth);
    return 0;
}
```

## Tracing 12. A QUEUE FROM TWO STACKS

Give the output, and state how many items are moved from `in` to `out` in total.

```
// a queue built from two stacks:
// enqueue pushes onto in; dequeue pops from out (refilled from in when empty)
typedef struct {
    stack in;
    stack out;
} twoQ;

void enq(twoQ *q, int x) { stPush(&q->in, x); }

int deq(twoQ *q) {
    int x;
    if (stEmpty(&q->out))
        while (stPop(&q->in, &x)) stPush(&q->out, x);
    if (!stPop(&q->out, &x)) return -1;
    return x;
}

int main() {
    twoQ q;
    int a;
    stInit(&q.in);
    stInit(&q.out);
    enq(&q, 1); enq(&q, 2); enq(&q, 3);
    a = deq(&q); printf("%d ", a);
    enq(&q, 4);
    a = deq(&q); printf("%d ", a);
    enq(&q, 5); enq(&q, 6);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d\n", a);
    printf("in=%d out=%d\n", stSize(&q.in), stSize(&q.out));
    return 0;
}
```

Juevesano D. 2026 All rights reserved. 

### Tracing 13. Which operand is popped first?

Both functions evaluate a postfix expression of single digits with integer division. They differ only in the order of the two pops. Give the table.

```c
// evalA pops the right operand first (correct); evalB pops the left first.
int apply(char c, int a, int b) {
    if (c == '+') return a + b;
    if (c == '-') return a - b;
    if (c == '*') return a * b;
    return a / b;
}

int evalA(char p[]) {
    stack st; int a, b;
    stInit(&st);
    for (int i = 0; p[i] != '\0'; i++) {
        if (p[i] >= '0' && p[i] <= '9') stPush(&st, p[i] - '0');
        else { stPop(&st, &b); stPop(&st, &a); stPush(&st, apply(p[i], a, b)); }
    }
    stPop(&st, &a);
    return a;
}

int evalB(char p[]) {
    stack st; int a, b;
    stInit(&st);
    for (int i = 0; p[i] != '\0'; i++) {
        if (p[i] >= '0' && p[i] <= '9') stPush(&st, p[i] - '0');
        else { stPop(&st, &a); stPop(&st, &b); stPush(&st, apply(p[i], a, b)); }
    }
    stPop(&st, &a);
    return a;
}

int main() {
    char *e[] = {"34+", "93-", "82/", "27-3/", "234*-", "92-3-", "52-4*"};
    for (int i = 0; i < 7; i++)
        printf("%-6s %3d %3d\n", e[i], evalA(e[i]), evalB(e[i]));
    return 0;
}
```

### Tracing 14. Stack permutations

`produce` tries to pop the numbers in the order `want[]` while pushing $1, 2, 3, 4$ in order. Give the five lines and the count.

```c
// Pushes 1, 2, ..., n in this order and pops as soon as the top is the next
// wanted item. Prints u for a push, o for a pop. Returns 1 if want[] can be
// popped in this order.
int produce(int want[], int n) {
    stack s;
    int next = 1, j = 0, top;
```

Juevesano D. 2026 All rights reserved. 

```
    stInit(&s);
    while (j < n) {
        if (stTop(&s, &top) && top == want[j]) {
            stPop(&s, &top);
            printf("o");
            j++;
        } else if (next <= n) {
            stPush(&s, next);
            next++;
            printf("u");
        } else {
            printf(" stuck: top %d, want %d\n", top, want[j]);
            return 0;
        }
    }
    printf(" ok\n");
    return 1;
}

int main() {
    int w[5][4] = {{3, 2, 1, 4}, {1, 2, 3, 4}, {3, 1, 2, 4},
                   {2, 4, 3, 1}, {4, 1, 2, 3}};
    int count = 0;
    for (int i = 0; i < 5; i++) count = count + produce(w[i], 4);
    printf("%d\n", count);
    return 0;
}
```

## Tracing 15. The stock spanner

The program uses `stockSpanner` of Problem 37: `ssNext` pops every (price, span) pair whose price is ≤ today's price, adds their spans to 1, and pushes today's pair. `show` prints the pairs from bottom to top.

```
// show() looks inside the two stacks only to print them (bottom to top).
void show(stockSpanner *s) {
    printf("[");
    for (int i = 0; i < s->prices.size; i++)
        printf("%s%d:%d", i ? " " : "", s->prices.items[i], s->spans.items[i]);
    printf("]\n");
}

int main() {
    int p[] = {100, 80, 60, 70, 60, 75, 85, 85};
    stockSpanner s;
    ssInit(&s);
    for (int i = 0; i < 8; i++) {
        int span = ssNext(&s, p[i]);
        printf("%3d -> %d  ", p[i], span);
        show(&s);
    }
    return 0;
}
```

Juevesano D. 2026 All rights reserved. 

### Tracing 16. The mid-pointer list

The program uses `mlist` of Problem 24: a singly linked list with `head`, `mid`, `tail` and `size`, where `middle(l)` returns the item at index $\lfloor (\texttt{size} - 1)/2 \rfloor$. Give the `mid` column; the list contents follow from the operations.

```
void show(char *op, mlist *l) {
    mnode *c;
    printf("%-13s n=%d mid=", op, l->size);
    if (l->size) printf("%d", middle(l)); else printf("-");
    printf("  [");
    for (c = l->head; c != NULL; c = c->next)
        printf("%d%s", c->item, c->next ? " " : "");
    printf("]\n");
}

int main() {
    mlist l;
    int x;
    minit(&l);
    mPushBack(&l, 10);  show("mPushBack 10", &l);
    mPushBack(&l, 20);  show("mPushBack 20", &l);
    mPushBack(&l, 30);  show("mPushBack 30", &l);
    mPushBack(&l, 40);  show("mPushBack 40", &l);
    mPushFront(&l, 5);  show("mPushFront 5", &l);
    mPushFront(&l, 1);  show("mPushFront 1", &l);
    mPopFront(&l, &x);  show("mPopFront", &l);
    mPopBack(&l, &x);   show("mPopBack", &l);
    mPushBack(&l, 50);  show("mPushBack 50", &l);
    mPopFront(&l, &x);  show("mPopFront", &l);
    return 0;
}
```

### Tracing 17. Binary search: counting the steps

`lowerBound` returns the index of the first item $\geq x$ on the half-open range `[lo, hi)`.

```
int lowerBound(int a[], int n, int x) {
    int lo = 0, hi = n, mid, step = 1;
    while (lo < hi) {
        mid = lo + (hi - lo) / 2;
        printf("step %d: lo=%d hi=%d ", step++, lo, hi);
        printf("mid=%d a[mid]=%d\n", mid, a[mid]);
        if (a[mid] < x) lo = mid + 1;
        else            hi = mid;
    }
    return lo;
}

int main() {
    int a[] = {2, 4, 4, 4, 7, 9, 12, 15, 15};
    printf("answer %d\n", lowerBound(a, 9, 4));
    printf("answer %d\n", lowerBound(a, 9, 10));
```

Juevesano D. 2026 All rights reserved. 

```
    printf("answer %d\n", lowerBound(a, 9, 20));
    return 0;
}
```

### Tracing 18. Josephus with a twist

Six people, every third one leaves. Give the order of removal and the winner, and check the winner with the recurrence of Chapter 5.

```
int main() {
    queue q;
    int x, k = 3, step;
    qInit(&q);
    for (int i = 1; i <= 6; i++) qEnq(&q, i);
    while (qSize(&q) > 1) {
        for (step = 1; step < k; step++) { qDeq(&q, &x); qEnq(&q, x); }
        qDeq(&q, &x);
        printf("out:%d ", x);
    }
    qDeq(&q, &x);
    printf("\nwinner:%d\n", x);
    return 0;
}
```

### Tracing 19. Splicing lists

The program uses `sll.h` and the two functions of Problem 18: `spliceAfter(l, p, m)` moves all nodes of `m` after node `p` of `l`, and `splice(l, pos, m)` moves them so that the first one becomes item `pos` of `l` ($0 \le$ `pos` $\le$ `l->size`; it returns 0 and does nothing for an invalid `pos`). Both leave `m` empty.

```
int main() {
    sll a, b, c, d;
    int r;
    sinit(&a); sinit(&b); sinit(&c); sinit(&d);
    for (int i = 1; i <= 5; i++) pushBack(&a, i);          // 1 2 3 4 5
    for (int i = 1; i <= 3; i++) pushBack(&b, i * 10);     // 10 20 30
    for (int i = 7; i <= 9; i++) pushBack(&c, i);          // 7 8 9
    pushBack(&d, 100);
    splice(&a, 2, &b);
    sdisplay(&a); sdisplay(&b);
    printf("%d %d\n", a.size, b.size);
    spliceAfter(&a, a.head->next->next, &c);
    sdisplay(&a); sdisplay(&c);
    printf("%d %d tail=%d\n", a.size, c.size, a.tail->item);
    r = splice(&a, 99, &d);
    printf("%d %d\n", r, d.size);
    r = splice(&a, a.size, &d);
    sdisplay(&a);
    printf("%d %d tail=%d\n", r, a.size, a.tail->item);
    return 0;
}
```

Juevesano D. 2026 All rights reserved. 

**Tracing 20.** Shallow and deep copies

```c
typedef struct node { int item; struct node *next; } node;
typedef struct { node *head; int size; } slist;

slist shallowCopy(slist s) { return s; }
slist deepCopy(slist s) {
    slist d;
    node *c = s.head, **t = &d.head;
    d.head = NULL; d.size = s.size;
    while (c != NULL) {
        *t = (node *)malloc(sizeof(node));
        (*t)->item = c->item; (*t)->next = NULL;
        t = &((*t)->next);
        c = c->next;
    }
    return d;
}
void show(slist s) {
    for (node *c = s.head; c != NULL; c = c->next) printf("%d ", c->item);
    printf("| %d\n", s.size);
}
void pushFront(slist *s, int x) {
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = s->head; s->head = n; s->size++;
}

int main() {
    slist orig = {NULL, 0};
    pushFront(&orig, 3); pushFront(&orig, 2); pushFront(&orig, 1);
    slist sh = shallowCopy(orig), dp = deepCopy(orig);
    sh.head->item = 99;
    dp.head->next->item = 77;
    pushFront(&sh, 0);
    show(orig); show(sh); show(dp);
    return 0;
}
```

**Tracing 21.** Circular deque: the raw array

A circular deque of capacity 5. `pushFront` steps the front back by adding 4 instead of subtracting 1. The last line prints three C expressions.

```c
// circular deque of capacity 5: item i lives at a[(f + i) % 5]
typedef struct {
    int a[5];
    int f;          // index of the front item
    int n;          // number of items
} cdq;

int pushFront(cdq *d, int x) {
    if (d->n == 5) return 0;
    d->f = (d->f + 4) % 5;          // one step back: + 4 is - 1 modulo 5
```

```
    d->a[d->f] = x;
    d->n++;
    return 1;
}
int pushBack(cdq *d, int x) {
    if (d->n == 5) return 0;
    d->a[(d->f + d->n) % 5] = x;
    d->n++;
    return 1;
}
int popFront(cdq *d) { int x = d->a[d->f]; d->f = (d->f + 1) % 5; d->n--; return x; }
int popBack(cdq *d) { d->n--; return d->a[(d->f + d->n) % 5]; }

void show(cdq *d) {
    printf("f=%d n=%d raw=[%d %d %d %d %d] deque=", d->f, d->n,
           d->a[0], d->a[1], d->a[2], d->a[3], d->a[4]);
    for (int i = 0; i < d->n; i++) printf("%d ", d->a[(d->f + i) % 5]);
    printf("\n");
}

int main() {
    cdq d = {{0, 0, 0, 0, 0}, 0, 0};
    pushBack(&d, 1); pushFront(&d, 2); pushFront(&d, 3); pushBack(&d, 4);
    show(&d);
    printf("%d ", popBack(&d));
    printf("%d\n", popFront(&d));
    show(&d);
    pushBack(&d, 5); pushBack(&d, 6); pushFront(&d, 7);
    show(&d);
    int r = pushFront(&d, 8);
    r = r * 10 + pushBack(&d, 9);
    printf("%d\n", r);
    printf("%d %d %d\n", (0 - 1) % 5, (0 - 1 + 5) % 5, -7 / 2);
    return 0;
}
```

## Tracing 22. ArrayDeque: which side moves?

Positional insertion in a circular array of capacity 8 that moves the shorter side. `add` returns the number of items moved.

```
// array deque of capacity 8 with positional add: the SHORTER side moves.
// Returns the number of items moved.
typedef struct { int a[8]; int j; int n; } adq;   // item i at a[(j + i) % 8]

int add(adq *d, int i, int x) {
    int moved = 0;
    if (i < d->n / 2) {                       // items 0..i-1 move one cell left
        d->j = (d->j + 7) % 8;
        for (int k = 0; k < i; k++, moved++)
            d->a[(d->j + k) % 8] = d->a[(d->j + k + 1) % 8];
    } else {                                  // items i..n-1 move one cell right
        for (int k = d->n; k > i; k--, moved++)
            d->a[(d->j + k) % 8] = d->a[(d->j + k - 1) % 8];
```

```
    }
    d->a[(d->j + i) % 8] = x;
    d->n++;
    return moved;
}

void show(adq *d) {
    printf("j=%d raw=[", d->j);
    for (int k = 0; k < 8; k++) printf("%d%s", d->a[k], k < 7 ? " " : "");
    printf("] list=");
    for (int k = 0; k < d->n; k++) printf("%d ", d->a[(d->j + k) % 8]);
    printf("\n");
}

int main() {
    adq d = {{0}, 0, 0};
    int m = 0;
    for (int x = 1; x <= 4; x++) m = m + add(&d, d.n, x * 10);   // 10 20 30 40
    printf("moved %d  ", m); show(&d);
    printf("moved %d  ", add(&d, 1, 15)); show(&d);
    printf("moved %d  ", add(&d, 3, 25)); show(&d);
    printf("moved %d  ", add(&d, 0, 5)); show(&d);
    printf("moved %d  ", add(&d, 6, 45)); show(&d);
    return 0;
}
```

## Tracing 23. Sliding window maximum

The program uses `deque.h` (Listing 6.1) and prints the deque of indices after each step of the algorithm of Problem 53. Give the eight lines. Watch the *two* equal values.

```
// The sliding-window maximum of Problem 53, printing the deque (indices,
// front first) after each step. show() looks inside only to print.
void show(deque *d) {
    printf("[");
    for (int t = 0; t < d->size; t++)
        printf("%s%d", t ? " " : "", d->items[(d->front + t) % 100]);
    printf("]");
}

int main() {
    int a[] = {4, 2, 12, 3, 8, 8, 1, 5}, n = 8, k = 3, j;
    deque d;
    dqInit(&d);
    for (int i = 0; i < n; i++) {
        if (dqFront(&d, &j) && j <= i - k) dqPopFront(&d, &j);
        while (dqBack(&d, &j) && a[j] <= a[i]) dqPopBack(&d, &j);
        dqPushBack(&d, i);
        printf("i=%d a[i]=%2d dq=", i, a[i]);
        show(&d);
        if (i >= k - 1) { dqFront(&d, &j); printf(" max=%d", a[j]); }
        printf("\n");
    }
    return 0;
```

Juevesano D. 2026 All rights reserved. 

```
}
```

### Tracing 24. A circular list as a queue

The program uses Listing 3.5: `enq(&last, x)` links a new node after `last` and makes it the new `last`; `deq(&last, &x)` removes `last->next` (the front), stores its item in `x` and returns 1, or returns 0 when the list is empty. The main program also moves the pointer by hand.

```
void show(cnode *last) {
    if (last == NULL) { printf("(empty)\n"); return; }
    cnode *c = last->next;
    do { printf("%d ", c->item); c = c->next; } while (c != last->next);
    printf("| last=%d\n", last->item);
}

int main() {
    cnode *q = NULL;
    int x = 0;
    enq(&q, 1); enq(&q, 2); enq(&q, 3); show(q);
    q = q->next; show(q);                    // one pointer move
    q = q->next->next; show(q);
    deq(&q, &x); printf("got %d  ", x); show(q);
    enq(&q, 4); show(q);
    while (deq(&q, &x)) printf("%d ", x);
    printf("\n");
    show(q);
    int r = deq(&q, &x);
    printf("%d %d\n", r, x);
    return 0;
}
```

### Tracing 25. The sentinel deque, and a struct copy

The program uses the linked deque of Problem 51 (Listing 6.3). Print 1 for true and 0 for false in the comparisons.

```
void forward(linkedDeque *d) {
    for (dqnode *c = d->sentinel.next; c != &d->sentinel; c = c->next)
        printf("%d ", c->item);
    printf("| size %d\n", d->size);
}

int main() {
    linkedDeque d;
    int x = -1;
    ldInit(&d);
    int r = ldPopBack(&d, &x);
    printf("%d %d %d\n", d.sentinel.next == &d.sentinel, r, x);
    ldPushBack(&d, 1); ldPushBack(&d, 2); ldPushFront(&d, 0); ldPushBack(&d, 3);
    forward(&d);
    ldPopBack(&d, &x); printf("%d ", x);
    ldPopFront(&d, &x); printf("%d\n", x);
```

Juevesano D. 2026 All rights reserved. 

```
    forward(&d);
    printf("%d %d\n", d.sentinel.prev->item, d.sentinel.next->prev == &d.sentinel);

    linkedDeque e = d;                      // copies the struct, sentinel too
    printf("%d %d\n", e.sentinel.next == d.sentinel.next,
           e.sentinel.next->prev == &e.sentinel);
    ldPushFront(&d, 9);
    printf("%d %d %d\n", d.sentinel.next->item, e.sentinel.next->item, e.size);
    ldFree(&d);
    printf("%d %d\n", d.size, d.sentinel.next == &d.sentinel);
    return 0;
}
```

Juevesano D. 2026 All rights reserved. 

Chapter 10

# Section C: Problem Solving

There are 57 problems in five groups. Every problem is of Easy or Medium difficulty and is written in the style of an online judge: you are given a structure and asked to implement one operation on it. Each statement shows the function to complete (the starter code), the constraints, the permitted operations, the required complexity, an optional bonus for doing better, and sample tests. The code is plain C/C++: no global variables, no `static` and no `const` are needed anywhere, and the solutions in Chapter 13 follow the same rule.

***Interfaces assumed throughout.*** Array list: `items[]` and `size`. SLL: Listing 3.1 (`snode`, `sll` with `head`, `tail`, `size`). DLL: Listing 3.3 (`node`, `list`). Stack and queue: Listings 4.1 and 5.1 (`stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`). Deque: Listing 6.1 (`dqInit`, `dqPushFront`, `dqPushBack`, `dqPopFront`, `dqPopBack`, `dqFront`, `dqBack`, `dqEmpty`, `dqSize`). Every pop, top, front and back returns 1 on success and 0 when the structure is empty, and passes the item through an `int *`. When a problem names a function, use exactly that signature; when it forbids an operation, even a correct use of it forfeits credit.

## 10.1 Arrays and array lists

**Problem 1.** String compression (run-length encoding) *Easy*

A *run* is a maximal block of equal consecutive characters. The run-length encoding of a string writes, for each run, the character followed by the length of the run. Given a string `s` of letters, implement `rleEncode`: it writes the encoding into `out` and returns the length of the encoding. The count is always written, even when it is 1, and it may have several digits.

Then implement `compress`, which does the same inside the array `a` of `n` characters, but a run of length 1 is written as the bare character. It returns the new length. No second array.

```
int rleEncode(char *s, char *out);   // out must end with '\0'
int compress(char *a, int n);        // in place, returns the new length
```

Permitted operations. Array indexing only. No `strlen`, `strcat`, `sprintf` or `itoa`: write the digits of a count yourself.

Constraints. $1 \le n \le 10^5$; the string contains only letters.

Required. $O(n)$ time; `compress` uses $O(1)$ extra space.

Bonus (extra credit). Implement `rleDecode(char *e, char *out)`.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `aaabccdddd` | `a3b1c2d4` |
| `wwwwwwwwwwwwbbb` | `w12b3` |
| `compress: aabcccccaaa` | `a2bc5a3 (length 7)` |
| `compress: abbbbbbbbbbbb` | `ab12 (length 4)` |
| `decode: x3y2z1` | `xxxyyz` |

### Problem 2. Matrix transpose *Easy*

Given an $n \times m$ matrix `a` ($n$ rows, $m$ columns), implement `transpose`, which stores the $m \times n$ transpose in `t`, so that $t_{ji} = a_{ij}$. Then implement `transposeInPlace` for a square $n \times n$ matrix.

```
void transpose(int a[][10], int n, int m, int t[][10]);
void transposeInPlace(int a[][10], int n);
```

Permitted operations. Array indexing only.

Constraints. $1 \le n, m \le 10$.

Required. $O(nm)$ time; `transposeInPlace` uses $O(1)$ extra space.

| Input | Expected output |
|---|---|
| `[[1,2,3],[4,5,6]]` | `[[1,4],[2,5],[3,6]]` |
| `[[1,2],[3,4]] (in place)` | `[[1,3],[2,4]]` |
| `[[7]]` | `[[7]]` |

### Problem 3. The missing number *Easy*

Given an array `a` of $m$ distinct integers in increasing order, taken from $\{1, 2, \dots, m+1\}$, exactly one number of that range is missing. Return it.

```
int missingNumber(int a[], int m);
```

Permitted operations. Array reads only. No sorting, no copy of the array, no counting array.

Constraints. $1 \le m \le 10^4$.

Required. $O(m)$ time and $O(1)$ extra space.

Bonus (extra credit). A second function `missingNumberFast` in $O(\log m)$ time, using the fact that the array is sorted.

| Input | Expected output |
|---|---|
| `1 2 4 5 6 7` | `3` |
| `1 2 3 4 5 6 8 9 10` | `7` |
| `2` | `1` |
| `1` | `2` |

Juevesano D. 2026 All rights reserved. 

**Problem 4.** A sorted array list *Medium*

An array list `slist` is kept in non-decreasing order at all times. Implement the following operations so that the order is preserved. Use binary search wherever the position of a value is needed. Write the helpers `lowerBound` (index of the first item $\geq x$) and `upperBound` (index of the first item $> x$) first. (i) `insert`: insert $x$ after the equal items; return 0 if the list is full, else 1. (ii) `erase`: remove one occurrence of $x$; return 1 if found, else 0. (iii) `search`: the index of the first occurrence of $x$, or $-1$. (iv) `count`: the number of occurrences of $x$.

```
typedef struct { int items[100]; int size; } slist;
int lowerBound(slist *l, int x);
int upperBound(slist *l, int x);
int insert(slist *l, int x);
int erase(slist *l, int x);
int search(slist *l, int x);
int count(slist *l, int x);
```

Permitted operations. Array access to `items` and `size`. No linear search for the position of a value.

Required. Finding a position, `search` and `count`: $O(\log n)$. `insert`, `erase`: $O(\log n)$ comparisons and $O(n)$ moves.

| Input | Expected output |
|---|---|
| `list 2 4 4 4 7 9: insert 5` | `2 4 4 4 5 7 9` |
| `list 2 4 4 4 7 9: erase 4` | `2 4 4 7 9` |
| `list 2 4 4 4 7 9: search 4, search 5` | `1, -1` |
| `list 2 4 4 4 7 9: count 4, count 9, count 3` | `3, 1, 0` |

**Problem 5.** Rotate an array *Easy*

Given an array `a` of $n$ integers, rotate it to the right by $k$ positions, in place. For example $[1, 2, 3, 4, 5, 6, 7]$ with $k = 3$ becomes $[5, 6, 7, 1, 2, 3, 4]$. The value $k$ may be larger than $n$.

```
void rotate(int a[], int n, int k);
```

Permitted operations. Array indexing and a scalar temporary. No second array.

Constraints. $0 \leq n \leq 10^5$, $0 \leq k \leq 10^9$.

Required. $O(n)$ time (shifting by one place $k$ times costs $O(nk)$ and earns partial credit).

Bonus (extra credit). $O(1)$ extra space with three reversals.

| Input | Expected output |
|---|---|
| `[1,2,3,4,5,6,7], k=3` | `[5,6,7,1,2,3,4]` |
| `[1,2], k=5` | `[2,1]` |
| `[1,2,3,4,5,6,7,8,9], k=4` | `[6,7,8,9,1,2,3,4,5]` |

Juevesano D. 2026 All rights reserved. 

**Problem 6.** Merge two sorted arrays *Easy*

Given sorted (non-decreasing) arrays `a` of length $n$ and `b` of length $m$, fill `c` with all $n + m$ items in sorted order and return $n + m$. Among equal items, those of `a` come first.

```
int mergeArrays(int a[], int n, int b[], int m, int c[]);
```

Permitted operations. Array indexing only. No sorting routine.

Constraints. $0 \le n, m \le 10^4$.

Required. $O(n + m)$ time.

| Input | Expected output |
|---|---|
| `[1,3,5,7], [2,3,6]` | `[1,2,3,3,5,6,7]` |
| `[], [4,5]` | `[4,5]` |
| `[1,2], []` | `[1,2]` |

**Problem 7.** Remove duplicates from a sorted array *Easy*

Given a sorted array `a` of $n$ integers, keep one copy of every value, in place, in the first positions. Return the new length. The cells after the new length may hold anything.

```
int dedupSorted(int a[], int n);
```

Permitted operations. Array indexing only.

Constraints. $0 \le n \le 10^5$.

Required. $O(n)$ time and $O(1)$ extra space.

| Input | Expected output |
|---|---|
| `[1,1,2,2,2,3,5,5]` | `length 4: [1,2,3,5]` |
| `[7]` | `length 1: [7]` |
| `[]` | `length 0` |

**Problem 8.** Two-sum in a sorted array *Easy*

Given a sorted array `a` of $n$ integers and a `target`, find indices $i < j$ with $a[i] + a[j] = target$. Store them in `*pi` and `*pj` and return 1, or return 0 if there is no such pair.

```
int twoSum(int a[], int n, int target, int *pi, int *pj);
```

Permitted operations. Array indexing only (no hash table).

Constraints. $2 \le n \le 10^5$; values fit in `int` and so do the sums.

Required. $O(n^2)$ earns partial credit; $O(n)$ time and $O(1)$ space earns full credit.

| Input | Expected output |
|---|---|
| `[1,3,4,6,8,11], target 10` | `(2,3)` |
| `[1,3,4,6,8,11], target 20` | `none` |
| `[2,2], target 4` | `(0,1)` |

Juevesano D. 2026 All rights reserved. 

### Problem 9. Maximum subarray sum *Medium*

Given an array `a` of $n \geq 1$ integers (positive, negative or zero), return the largest sum of a non-empty contiguous subarray.

```
int maxSubarray(int a[], int n);
```

Permitted operations. Array reads only.

Constraints. $1 \leq n \leq 10^5$; $|a[i]| \leq 10^4$.

Required. $O(n^2)$ earns partial credit (try all subarrays with a running sum); $O(n)$ time and $O(1)$ space earns full credit.

| Input | Expected output |
|---|---|
| `[-2,1,-3,4,-1,2,1,-5,4]` | `6 (subarray [4,-1,2,1])` |
| `[-3,-1,-2]` | `-1` |
| `[5]` | `5` |

### Problem 10. Move zeroes *Easy*

Given an array `a` of $n$ integers, move all zeros to the end, keeping the order of the non-zero items, in place.

```
void moveZeroes(int a[], int n);
```

Permitted operations. Array indexing only.

Constraints. $0 \leq n \leq 10^5$.

Required. $O(n)$ time, $O(1)$ extra space.

| Input | Expected output |
|---|---|
| `[0,1,0,3,12]` | `[1,3,12,0,0]` |
| `[0,0]` | `[0,0]` |
| `[4,5]` | `[4,5]` |

### Problem 11. Majority element *Easy*

Given an array `a` of $n$ integers, return the majority element, the element that appears more than $\lfloor n/2 \rfloor$ times. It is guaranteed to exist.

```
int majority(int a[], int n);
```

Permitted operations. Array reads only. No sorting, no counting array indexed by value.

Constraints. $1 \leq n \leq 10^5$.

Required. $O(n)$ time.

Bonus (extra credit). $O(1)$ extra space (Boyer–Moore voting).

| Input | Expected output |
|---|---|
| `[2,2,1,1,2,2,3,2,2]` | `2` |
| `[3,3,4]` | `3` |
| `[7]` | `7` |

Juevesano D. 2026 All rights reserved. 

**Problem 12.** Rotate a matrix by 90 degrees *Medium*

Given an $n \times n$ matrix `a`, rotate it by 90° clockwise, in place.

```
void rotate90(int a[][10], int n);
```

Permitted operations. Array indexing only.

Constraints. $1 \le n \le 10$.

Required. $O(n^2)$ time, $O(1)$ extra space.

| Input | Expected output |
|---|---|
| `[[1,2],[3,4]]` | `[[3,1],[4,2]]` |
| `[[1,2,3],[4,5,6],[7,8,9]]` | `[[7,4,1],[8,5,2],[9,6,3]]` |

## 10.2 Linked lists

**Problem 13.** Implementation: eraseItems (DLL) *Medium*

*This is one of the implementation problems of the departmental style, reproduced with its sample input and output.*

Implement the operation `void eraseItems(list *l, int x)` for a doubly linked list. Without calling `search` and `erase`, remove all items in `l` equal to `x`. The type `list` (Listing 3.3) has the fields `head`, `tail`, `size`; nodes have `item`, `next`, `prev`.

The test program reads $n$, then $n$ items, then $x$. It prints the list before, calls your function, prints the list after, then `New size:` and the list from tail to head. When the list becomes empty it also prints a check that `head` and `tail` are both `NULL`. (The template code around your function is reconstructed from the sample output; here the test cases are separated by a line `----`.)

```
void eraseItems(list *l, int x);
```

Permitted operations. Direct access to `head`, `tail`, `size`, `next`, `prev`. Not `search`, `erase`, `remove` or any helper that finds or deletes a node for you.

Required. $O(n)$ time in one pass; $O(1)$ extra space; `size`, `head`, `tail` and every `prev` link correct afterwards; removed nodes freed.

Bonus (extra credit). Explain why a solution that restarts from the head after every removal is $\Theta(n^2)$ on an input such as `1 1 1 1 1 3 3 3 3 3` with $x = 3$, although it is fast on `3 3 3 3 3`.

Input (the test cases one after another)

```
5
1 2 3 4 5
6
10
10 5 5 10 10 6 9 10 12 13
10
5
3 3 3 3 3
```

```
3
10
1 1 3 4 5 6 7 8 9 10
1
10
1 2 3 4 5 6 7 8 9 9
9
5
3 6 3 3 3
3
```

Expected output

```
1 2 3 4 5
1 2 3 4 5
New size: 5
Displaying from tail down to head:
5 4 3 2 1
----
10 5 5 10 10 6 9 10 12 13
5 5 6 9 12 13
New size: 6
Displaying from tail down to head:
13 12 9 6 5 5
----
3 3 3 3 3

New size: 0
Displaying from tail down to head:

Yes, correct. Both the head and tail are NULL.
----
1 1 3 4 5 6 7 8 9 10
3 4 5 6 7 8 9 10
New size: 8
Displaying from tail down to head:
10 9 8 7 6 5 4 3
----
1 2 3 4 5 6 7 8 9 9
1 2 3 4 5 6 7 8
New size: 8
Displaying from tail down to head:
8 7 6 5 4 3 2 1
----
3 6 3 3 3
6
New size: 1
Displaying from tail down to head:
6
----
```

Juevesano D. 2026 All rights reserved. 

**Problem 14.** Implementation: appendList (copying a list) *Medium*

Implement the function `void appendList(list *l, list m)`. The idea is simply to append the items of `m` to `l`, without calling `append`. Take note of the edge cases. Make sure that when `m` is destroyed, the items appended to the list pointed to by `l` are no longer affected. The type `list` (Listing 3.3) has the fields `head`, `tail`, `size`; nodes have `item`, `next`, `prev`.

The test program reads $n$ and the items of `l`, then $|m|$ and the items of `m`; it prints `list l:`, `l`, `list m:`, `m`, calls the function, destroys `m`, and prints `l`.

```
void appendList(list *l, list m);
```

Permitted operations. Direct access to `head`, `tail`, `size`, `next`, `prev`; `malloc`. Not `append`.

Required. $O(|m|)$ time; the nodes of `l` must be new nodes, never nodes of `m`.

Bonus (extra credit). The call `appendList(&x, x)` (a list appended to itself) must terminate and double the list. Say why a loop until `NULL` fails here and a counted loop does not.

Input (the test cases one after another)

```
5
1 2 3 4 5
3
9 10 11
1
10
5
9 8 7 6 5
5
5 4 3 2 1
0
0
5
1 2 3 4 5
10
1 2 3 4 5 6 7 8 9 10
10
11 12 13 14 15 16 17 18 19 20
```

Expected output

```
list l:
1 2 3 4 5
list m:
9 10 11
1 2 3 4 5 9 10 11
----
list l:
10
list m:
9 8 7 6 5
10 9 8 7 6 5
----
list l:
5 4 3 2 1
list m:
```

Juevesano D. 2026 All rights reserved. 

```
5 4 3 2 1
----
list l:

list m:
1 2 3 4 5
1 2 3 4 5
----
list l:
1 2 3 4 5 6 7 8 9 10
list m:
11 12 13 14 15 16 17 18 19 20
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
----
```

**Problem 15.** Implementation: recursive append with a double pointer *Medium*

Assume the singly linked list has no tail pointer. Implement `append(snode **h, int x)`, which adds a node with item `x` at the end of the list. Notice the double pointer: it lets the function change the real pointer and not a copy. `h` is the address of the pointer that heads the list. Implement it recursively. In `main` it is called as `append(&l.head, item)`: since the head must change when the list is empty, what is passed is the address of the head and not the head.

```
void append(snode **h, int x);   // called as append(&l.head, item)
```

Permitted operations. `malloc`; the fields `item` and `next`. No loops in `append`, no extra helper that returns a new head.

Required. $O(n)$ time, recursive.

Bonus (extra credit). Implement the same operation with a loop but still with a `snode **` (one variable, no special case for the empty list). State the recursion depth and when it is dangerous.

| Input | Expected output |
|---|---|
| `5 / 1 2 3 4 5` | `1 2 3 4 5` |
| `1 / 109` | `109` |
| `10 / 10 9 8 7 6 5 4 3 2 1` | `10 9 8 7 6 5 4 3 2 1` |

**Problem 16.** Reverse a singly linked list *Easy*

Given an SLL, reverse it in place by changing links: do not allocate nodes and do not copy items. Implement the iterative `reverseList`, which must also fix `head` and `tail`.

```
void reverseList(sll *l);          // iterative; fix head and tail
snode *reverseRec(snode *h);       // bonus: recursive, returns the new head
```

Permitted operations. Pointer manipulation of `next`, `head`, `tail`.

Constraints. $0 \le |l| \le 10^5$.

Required. $O(n)$ time, $O(1)$ extra space (iterative).

Bonus (extra credit). The recursive version `reverseRec`; explain its stack use.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `[1,2,3,4,5]` | `[5,4,3,2,1]` |
| `[]` | `[]` |
| `[7]` | `[7]` |

**Problem 17.** Middle node and $k$-th node from the end *Easy*

Given an SLL, implement (a) `middleItem`, which returns the middle item (the second middle when $n$ is even; the list is non-empty), and (b) `kthFromEnd`, which stores the $k$-th item from the end in `*out` and returns 1, or returns 0 if $k < 1$ or $k > n$. Both must work in one pass and without using `size`.

```
int middleItem(sll *l);                   // second middle if n is even
int kthFromEnd(sll *l, int k, int *out);  // k = 1 is the last item
```

Permitted operations. Pointer traversal only. You may not read `size`, count the nodes first, or copy the list into an array.

Constraints. $1 \le n \le 10^5$.

Required. $O(n)$ time, one traversal, $O(1)$ extra space.

Bonus (extra credit). Return the *first* middle for even $n$ by a change of one condition, and explain the change.

| Input | Expected output |
|---|---|
| `middle [1,2,3,4]` | `3` |
| `middle [1,2,3,4,5]` | `3` |
| `kth [10,20,30,40,50], k=2` | `40` |
| `kth [10,20,30,40,50], k=5 / k=6 / k=0` | `10 / fails / fails` |

**Problem 18.** Splicing singly linked lists *Medium*

Splicing moves nodes from one list to another by relinking, without allocating or copying. (i) `spliceAfter`: move all nodes of `m` into `l`, immediately after node `p` (`p==NULL` means "at the very front"). Afterwards `m` is empty; `head`, `tail` and both sizes must be correct. (ii) `splice`: the same, but the place is an index: `pos` nodes of `l` stay in front of the spliced part ($0 \le$ `pos` $\le |l|$). Return 0 if `pos` is invalid.

```
void spliceAfter(sll *l, snode *p, sll *m);  // p == NULL: at the front
int splice(sll *l, int pos, sll *m);          // 0 if pos is invalid
```

Permitted operations. Pointer relinking only. No `malloc`, no `free`, no item copying.

Required. `spliceAfter`: $O(1)$. `splice`: $O($`pos`$)$ to walk to the anchor, then $O(1)$.

Bonus (extra credit). Show that, given a tail pointer, appending a whole list to another list costs $O(1)$ whereas an array list needs $\Theta(|m|)$.

| Input | Expected output |
|---|---|
| `l=[1,2,5], m=[3,4], splice at pos 2` | `l=[1,2,3,4,5] m=[] sizes 5 0` |
| `l=[1,2,5], m=[3,4], splice at pos 0` | `l=[3,4,1,2,5]` |
| `l=[1,2,5], m=[3,4], splice at pos 3` | `l=[1,2,5,3,4] (tail is 4)` |
| `m empty` | `l unchanged` |

Juevesano D. 2026 All rights reserved. 

### Problem 19. Merge two sorted doubly linked lists *Medium*

Two doubly linked lists `a` and `b` are sorted in non-decreasing order. Implement `mergeSorted` so that afterwards `a` holds all the nodes in sorted order and `b` is empty. Relink the existing nodes (no allocation, no freeing); on equal items the node of `a` goes first; all `prev` links, `head`, `tail` and `size` must be correct.

```
void mergeSorted(list *a, list *b);
```

Permitted operations. Pointer manipulation of `next`, `prev`, `head`, `tail`, `size`.

Required. $O(|a| + |b|)$ time, $O(1)$ extra space.

Bonus (extra credit). Attach the remainder of the non-empty list in $O(1)$ instead of node by node.

| Input | Expected output |
|---|---|
| `a=[1,4,7], b=[2,4,5,9]` | `a=[1,2,4,4,5,7,9] size 7; backward: 9 7 5 4 4 2 1; b.size=0` |
| `a=[], b=[3]` | `a=[3]` |
| `a=[1], b=[]` | `a=[1]` |

### Problem 20. Palindromic doubly linked list *Easy*

Given a doubly linked list, return 1 if its items read the same forwards and backwards, else 0. The empty list and a one-element list are palindromes.

```
int isPalindrome(list *l);
```

Permitted operations. Pointer traversal. Not an auxiliary array or stack.

Constraints. $0 \le n \le 10^5$.

Required. $O(n)$ time, $O(1)$ extra space.

Bonus (extra credit). Stop after $\lfloor n/2 \rfloor$ comparisons.

| Input | Expected output |
|---|---|
| `[1,2,3,2,1]` | `1` |
| `[1,2,3,4]` | `0` |
| `[]` | `1` |
| `[7]` | `1` |
| `[4,5,5,4]` | `1` |

### Problem 21. Reverse a doubly linked list *Easy*

Given a doubly linked list, reverse it in place.

```
void reverseDLL(list *l);
```

Permitted operations. Pointer manipulation only.

Required. $O(n)$ time, $O(1)$ space.

Bonus (extra credit). Do it by swapping `next` and `prev` in every node, without moving any item.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `[1,2,3,4,5]` | `forward 5 4 3 2 1; backward 1 2 3 4 5` |
| `[]` | `[]` |

### Problem 22. Remove duplicates from a sorted doubly linked list *Easy*

Given a doubly linked list sorted in non-decreasing order, remove nodes so that each value appears once (keep the first node of each run). Free the removed nodes and keep `tail` and `size` correct.

```
void removeDuplicates(list *l);
```

Permitted operations. Pointer manipulation only; no `search` or `erase`.

Required. $O(n)$ time, $O(1)$ space.

Bonus (extra credit). Remove *all* nodes whose value occurs more than once.

| Input | Expected output |
|---|---|
| `[1,1,2,3,3,3,5]` | `[1,2,3,5], size 4` |
| `[2,2,2]` | `[2]` |
| `[]` | `[]` |

### Problem 23. Insert into a sorted doubly linked list *Easy*

Given a doubly linked list sorted in non-decreasing order, insert $x$ so that the list stays sorted (after equal items). Handle the empty list, the new minimum and the new maximum.

```
void insertSorted(list *l, int x);
```

Permitted operations. Allocation, `next`/`prev`, `head`/`tail`. No call to `append` except where it is the correct case.

Required. $O(n)$ time.

Bonus (extra credit). $O(1)$ when $x$ is not smaller than the tail item.

| Input | Expected output |
|---|---|
| `[2,4,4,8], insert 1` | `[1,2,4,4,8]` |
| `then insert 5` | `[1,2,4,4,5,8]` |
| `then insert 9` | `[1,2,4,4,5,8,9] (tail 9, size 7)` |
| `then insert 4` | `[1,2,4,4,4,5,8,9]` |
| `[], insert 3` | `[3]` |

Juevesano D. 2026 All rights reserved. 

## Problem 24. An SLL with a middle pointer *Medium*

A singly linked list `mlist` keeps, besides `head`, `tail` and `size`, a pointer `mid`. The invariant is

$$\texttt{size} = 0 \Rightarrow \texttt{head} = \texttt{mid} = \texttt{tail} = \texttt{NULL},$$
$$\texttt{size} > 0 \Rightarrow \texttt{mid} \text{ points to the node at index } \lfloor (\texttt{size} - 1)/2 \rfloor \text{ (0-based)}.$$

Implement `middle`, `mPushBack`, `mPushFront`, `mPopFront` and `mPopBack` so that every operation restores the invariant.

```
typedef struct mnode { int item; struct mnode *next; } mnode;
typedef struct { mnode *head, *mid, *tail; int size; } mlist;
void minit(mlist *l);
int middle(mlist *l);                // O(1)
void mPushBack(mlist *l, int x);     // O(1)
int mPopFront(mlist *l, int *x);     // O(1); 0 if empty
void mPushFront(mlist *l, int x);    // O(1) or O(n), see below
int mPopBack(mlist *l, int *x);      // O(n)
```

Permitted operations. Pointer manipulation. A node has only `item` and `next`.

Required. `middle`, `mPushBack`, `mPopFront`: $O(1)$. For `mPushFront` and `mPopBack` state the worst case.

Bonus (extra credit). Explain why a doubly linked list would make all four end operations $O(1)$.

| Input | Expected output |
|---|---|
| `pushBack 10,20,…,70 (sizes 1..7)` | `mid after each: 10 10 20 20 30 30 40` |
| `10,20,30,40; pushFront 5; pushFront 1` | `mid 20; then mid 10 (list 1 5 10 20 30 40)` |
| `then: popFront; popBack; pushBack 50; popFront` | `mid 20; 10; 20; 20 (list 10 20 30 50)` |

## Problem 25. Rotate a linked list *Medium*

Given an SLL and an integer $k \geq 0$, rotate the list to the left by $k$ positions: $[1, 2, 3, 4, 5]$ with $k = 2$ becomes $[3, 4, 5, 1, 2]$. $k$ may exceed $n$.

```
void rotateLeft(sll *l, int k);
```

Permitted operations. Pointer manipulation; `size` is available.

Constraints. $0 \leq n \leq 10^5$, $0 \leq k \leq 10^9$.

Required. $O(n)$ time.

Bonus (extra credit). No node is allocated or freed.

| Input | Expected output |
|---|---|
| `[1,2,3,4,5], k=2` | `[3,4,5,1,2]` |
| `[1,2,3,4,5], k=7` | `[3,4,5,1,2]` |
| `[1,2,3,4,5], k=5` | `[1,2,3,4,5]` |
| `[], k=3` | `[]` |

Juevesano D. 2026 All rights reserved. 

**Problem 26.** Partition a linked list around a value *Medium*

Given an SLL and a value $x$, rearrange the nodes so that all nodes with item $< x$ precede all nodes with item $\geq x$, keeping the original relative order inside each group. Relink nodes; do not allocate.

```
void partitionList(sll *l, int x);
```

Permitted operations. Pointer manipulation only.

Required. $O(n)$ time, one pass.

Bonus (extra credit). $O(1)$ extra space using two dummy nodes.

| Input | Expected output |
|---|---|
| `[1,4,3,2,5,2], x=3` | `[1,2,2,4,3,5]` |
| `[5,6], x=3` | `[5,6]` |
| `[1,2], x=3` | `[1,2]` |
| `[], x=3` | `[]` |

**Problem 27.** Remove the $n$-th node from the end *Medium*

Given an SLL, remove the $n$-th node from the end ($n = 1$ is the last node) in one pass. Return 1, or 0 if $n < 1$ or $n > |l|$. Keep `head`, `tail` and `size` correct and free the node.

```
int removeNthFromEnd(sll *l, int n);   // n = 1 is the last node
```

Permitted operations. Pointer manipulation; do not use `size` to compute the position.

Required. $O(|l|)$ time, one pass.

Bonus (extra credit). Handle the removal of the head with a dummy node and no special case.

| Input | Expected output |
|---|---|
| `[1,2,3,4,5], n=2` | `[1,2,3,5]` |
| `[1,2,3,5], n=4` | `[2,3,5] (size 3)` |
| `[2,3,5], n=1` | `[2,3] (tail 3)` |
| `[2,3], n=3` | `fails (0), list unchanged` |
| `[7], n=1` | `[] (head, tail NULL, size 0)` |

Juevesano D. 2026 All rights reserved. 

### Problem 28. Swap nodes in pairs *Medium*

Given an SLL, swap every two adjacent nodes: $[1,2,3,4] \rightarrow [2,1,4,3]$. A final unpaired node stays where it is. Swap nodes, not items.

```
void swapPairs(sll *l);
```

Permitted operations. Pointer manipulation only.

Required. $O(n)$ time, $O(1)$ space.

Bonus (extra credit). The same with recursion; also keep `tail` correct.

| Input | Expected output |
|---|---|
| `[1,2,3,4]` | `[2,1,4,3]` |
| `[1,2,3,4,5]` | `[2,1,4,3,5] (tail 5)` |
| `[1]` | `[1]` |

### Problem 29. Cycle detection *Medium*

Given the head of a singly linked list whose last node may point back to an earlier node, return 1 if the list contains a cycle and 0 otherwise. The list is not modified.

```
int hasCycle(snode *head);        // 1 if the list has a cycle
snode *cycleStart(snode *head);   // bonus: first node of the cycle, or NULL
```

Permitted operations. Pointer traversal. No visited-marker field, no auxiliary array or table.

Constraints. $0 \leq n \leq 10^5$.

Required. $O(n)$ time and $O(1)$ extra space.

Bonus (extra credit). Return the first node of the cycle (or `NULL`), in $O(n)$ time and $O(1)$ space.

| Input | Expected output |
|---|---|
| `[1,2,3,4,5], last node points to the 3rd node` | `1 (cycle starts at item 3)` |
| `[1,2,3,4,5], last node points to the head` | `1 (starts at item 1)` |
| `[1,2,3,4,5], no link back` | `0` |
| `[] (head is NULL)` | `0` |

### Problem 30. Group odd and even positions *Medium*

Given an SLL, group all nodes at odd positions (1st, 3rd, ...) first, followed by all nodes at even positions, each group keeping its order: $[1,2,3,4,5] \rightarrow [1,3,5,2,4]$. The positions are meant, not the values.

```
void oddEvenList(sll *l);
```

Permitted operations. Pointer manipulation only.

Required. $O(n)$ time, $O(1)$ extra space.

Bonus (extra credit). Keep `tail` correct.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `[1,2,3,4,5]` | `[1,3,5,2,4] (tail 4)` |
| `[1,2,3,4]` | `[1,3,2,4]` |
| `[1,2]` | `[1,2]` |

**Problem 31.** Add two numbers stored in linked lists *Medium*

Two non-negative integers are stored one digit per node, least significant digit first: 342 is $2 \to 4 \to 3$. Given the lists `a` and `b`, build in `out` the list of their sum, in the same form. The lists may have different lengths and there may be a final carry.

```
void addNumbers(sll *a, sll *b, sll *out);
```

Permitted operations. `sinit` and `pushBack` on the result. No conversion to a machine integer (the numbers may have hundreds of digits).

Required. $O(\max(|a|, |b|))$ time.

Bonus (extra credit). The case in which the digits are stored most significant first.

| Input | Expected output |
|---|---|
| `[2,4,3] + [5,6,4]` | `[7,0,8]` |
| `[9,9] + [1]` | `[0,0,1]` |
| `[0] + [0]` | `[0]` |

## 10.3 Stacks

In this section the stack of Listing 4.1 holds `int`s; characters are pushed as their codes. "Stack operations only" means `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`: no access to `items[]`.

**Problem 32.** Valid brackets *Easy*

Given a string `s` containing only the characters `( ) [ ] { }`, return 1 if every bracket is closed by the same kind of bracket and in the correct order, else 0.

```
int isBalanced(char s[]);
```

Permitted operations. Stack operations only.

Constraints. $0 \le |s| \le 10^4$.

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| `{[()]}` | `1` |
| `([)]` | `0` |
| `((` | `0` |
| `))` | `0` |
| `(empty string)` | `1` |
| `()[]{}` | `1` |

Juevesano D. 2026 All rights reserved. 

**Problem 33.** Infix to postfix and postfix evaluation *Medium*

(a) Given an infix expression `in` made of single-digit operands, the operators `+ - * /` and parentheses (no spaces), write its postfix form into `out`. The operators `*` and `/` have higher precedence than `+` and `-`; operators of equal precedence are left-associative. (b) Given a valid postfix expression `p` of single digits and `+ - * /`, return its value (integer division, truncating toward zero).

```
void toPostfix(char in[], char out[]);
int evalPostfix(char p[]);
```

Permitted operations. Stack operations only.

Constraints. $1 \le |in| \le 100$; the input is valid.

Required. $O(n)$ time for both.

Bonus (extra credit). Explain what changes if ^ (exponent, right-associative) is added.

| Input | Expected output |
|---|---|
| `2+3*4` | `234*+ ; value 14` |
| `(2+3)*4` | `23+4* ; value 20` |
| `2*(3+4)-5` | `234+*5- ; value 9` |
| `8-3-2` | `83-2- ; value 3` |
| `9-8/(4-2)` | `9842-/- ; value 5` |

**Problem 34.** A stack with constant-time minimum *Easy*

Design a stack that supports `push`, `pop` and `min` (the smallest item currently on the stack), each in $O(1)$ time.

```
typedef struct { stack all; stack mins; } minStack;
void msInit(minStack *m);
void msPush(minStack *m, int x);
int msPop(minStack *m, int *x);    // 0 if empty
int msMin(minStack *m, int *x);    // 0 if empty
```

Permitted operations. Stack operations on the two member stacks.

Required. Every operation $O(1)$.

Bonus (extra credit). Use less memory when many pushes are not new minima (push to `mins` only when $x \le$ the current minimum), and explain why the test must be $\le$, not $<$.

| Input | Expected output |
|---|---|
| `push 5 3 7 3 1; min` | `1` |
| `pop; pop; min` | `3 (stack 5 3 7)` |
| `push 2; min; pop; min` | `2; 3` |
| `min on an empty stack` | `fails (0)` |

Juevesano D. 2026 All rights reserved. 

### Problem 35. Next greater element *Medium*

For every item `a[i]` of an array, find the first item to its right that is *strictly* greater, or $-1$ if there is none.

```
void nextGreater(int a[], int n, int out[]);
```

Permitted operations. Array access to `a` and `out`; one stack.

Constraints. $0 \le n \le 10^5$.

Required. $O(n)$ time ($O(n^2)$ earns partial credit).

Bonus (extra credit). The *circular* version: the search may wrap around to the beginning of the array (`[1,2,1]` → `[2,-1,2]`).

| Input | Expected output |
|---|---|
| `[2,1,5,3]` | `[5,5,-1,-1]` |
| `[2,7,3,5,4,6,8]` | `[7,8,5,6,6,8,-1]` |
| `[5,4,3]` | `[-1,-1,-1]` |
| `[1,1]` | `[-1,-1]` |

### Problem 36. Daily temperatures *Medium*

Given the daily temperatures `t[0..n-1]`, compute `wait[i]`, the number of days after day $i$ until a *strictly* warmer day, or 0 if no warmer day follows.

```
void dailyTemperatures(int t[], int n, int wait[]);
```

Permitted operations. Array access to `t` and `wait`; one stack.

Constraints. $1 \le n \le 10^5$; $30 \le t[i] \le 100$.

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| `[73,74,75,71,69,72,76,73]` | `[1,1,4,2,1,1,0,0]` |
| `[30,40,50,60]` | `[1,1,1,0]` |
| `[30,60,90]` | `[1,1,0]` |
| `[50,50]` | `[0,0]` |

### Problem 37. Online stock span *Medium*

Prices of a stock arrive one per day. The *span* of today's price is the number of consecutive days, ending today and going backwards, on which the price was $\le$ today's price. Implement `ssNext`, which receives today's price and returns its span. The prices are given one at a time: you cannot look at future prices, and you should not store the whole history.

```
typedef struct { stack prices; stack spans; } stockSpanner;
void ssInit(stockSpanner *s);
int ssNext(stockSpanner *s, int price);
```

Permitted operations. Stack operations on the two member stacks.

Required. $O(1)$ amortised per call.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| 100 80 60 70 60 75 85 | 1 1 1 2 1 4 6 |
| 10 10 10 | 1 2 3 |
| 5 4 3 2 1 | 1 1 1 1 1 |

### Problem 38. Asteroid collision *Medium*

Each item of a is an asteroid in a row: its absolute value is its size, its sign its direction ($> 0$ moves right, $< 0$ moves left); all move at the same speed. When two meet, the smaller one explodes; if both are the same size, both explode. Two asteroids moving in the same direction never meet. Write the survivors, left to right, into out and return how many there are.

```
int asteroidCollision(int a[], int n, int out[]);
```

Permitted operations. Array access to a and out; one stack.

Constraints. $0 \le n \le 10^4$; $a[i] \neq 0$.

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| [5,10,-5] | [5,10] |
| [8,-8] | [] |
| [10,2,-5] | [10] |
| [-2,-1,1,2] | [-2,-1,1,2] |

### Problem 39. Decode string *Medium*

A string is encoded by the rule k[part]: the part inside the brackets is repeated exactly $k$ times ($k \ge 1$, possibly several digits). Encodings may be nested. Write the decoded string into out.

```
void decodeString(char s[], char out[]);
```

Permitted operations. Character access to s and out; stacks.

Constraints. The input is valid; the decoded string has at most $10^4$ characters.

Required. Time proportional to the length of the output.

| Input | Expected output |
|---|---|
| 3[a]2[bc] | aaabcbc |
| 3[a2[c]] | accaccacc |
| 2[abc]3[cd]ef | abcabccdcdcdef |
| 10[a] | aaaaaaaaaa |
| abc | abc |

Juevesano D. 2026 All rights reserved. 

### Problem 40. Remove adjacent duplicates *Easy*

Repeatedly remove two adjacent equal letters from `s` until no two adjacent letters are equal; write the result into `out`. The result is unique.

```
void removeAdjacent(char s[], char out[]);
```

Permitted operations. Character access; one stack.

Constraints. $1 \le |s| \le 10^5$.

Required. $O(n)$ time. (Rescanning the string after each removal is $O(n^2)$.)

| Input | Expected output |
|---|---|
| `abbaca` | `ca` |
| `azxxzy` | `ay` |
| `aa` | `(empty string)` |
| `abc` | `abc` |

### Problem 41. Sort a stack *Medium*

Sort the stack `s` so that the smallest item is on top. You may use one additional stack and scalar variables, nothing else.

```
void sortStack(stack *s);
```

Permitted operations. Stack operations only; one extra stack.

Constraints. $0 \le n \le 100$.

Required. $O(n^2)$ time.

| Input | Expected output |
|---|---|
| `bottom..top 3 1 4 1 5` | `popping gives 1 1 3 4 5` |
| `bottom..top 1 2 3` | `popping gives 1 2 3` |
| `empty` | `empty` |

### Problem 42. Validate stack sequences *Medium*

The distinct numbers `pushed[0..n-1]` are pushed onto an initially empty stack in this order, and pops may occur at any moment. Return 1 if the pops can produce exactly the order `popped[0..n-1]`, else 0.

```
int validateStackSequences(int pushed[], int popped[], int n);
```

Permitted operations. Array reads; one stack.

Constraints. $0 \le n \le 1000$.

Required. $O(n)$ time.

Bonus (extra credit). Prove that the greedy rule "pop whenever the top is the next wanted item" never misses a solution.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `pushed 1 2 3 4 5, popped 4 5 3 2 1` | `1` |
| `pushed 1 2 3 4 5, popped 4 3 5 1 2` | `0` |
| `pushed 2 1 0, popped 1 2 0` | `1` |

## 10.4 Queues

"Queue operations only" means `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize` of Listing 5.1: no access to `items[]` or `front`.

### Problem 43. A queue from two stacks *Easy*

Implement a FIFO queue using two stacks `in` and `out`: `tsqPush` adds at the rear, `tsqPop` removes from the front, `tsqPeek` reads the front.

```
typedef struct { stack in; stack out; } twoStackQueue;
void tsqInit(twoStackQueue *q);
void tsqPush(twoStackQueue *q, int x);
int tsqPop(twoStackQueue *q, int *x);    // 0 if empty
int tsqPeek(twoStackQueue *q, int *x);   // 0 if empty
int tsqEmpty(twoStackQueue *q);
```

Permitted operations. Stack operations only.

Required. Every operation $O(1)$ amortised.

Bonus (extra credit). Prove the amortised bound (Theorem 5.2).

| Input | Expected output |
|---|---|
| `push 1, push 2, peek, pop, empty` | `1, 1, 0` |
| `then push 3, pop, pop, pop` | `2, 3, fails` |

### Problem 44. A stack from a queue *Easy*

Implement a LIFO stack using *one* queue.

```
typedef struct { queue q; } queueStack;
void qsInit(queueStack *s);
void qsPush(queueStack *s, int x);
int qsPop(queueStack *s, int *x);    // 0 if empty
int qsTop(queueStack *s, int *x);    // 0 if empty
int qsEmpty(queueStack *s);
```

Permitted operations. Queue operations only.

Required. One of push and pop in $O(1)$, the other in $O(n)$.

| Input | Expected output |
|---|---|
| `push 1, push 2, top, pop, empty` | `2, 2, 0` |
| `then pop, empty` | `1, 1` |
| `push 1, push 2, push 3, pop, push 4, pop, pop` | `3, 4, 2` |

Juevesano D. 2026 All rights reserved. 

### Problem 45. The Josephus problem *Medium*

$n$ people stand in a circle, numbered $1..n$. Counting starts at person 1; every $k$-th person leaves the circle, and counting resumes with the next person. Return the number of the last person left.

```
int josephus(int n, int k);
```

Permitted operations. Queue operations only.

Constraints. $1 \le n \le 99$ (the queue has capacity 100), $1 \le k \le 1000$.

Required. $O(nk)$ time.

Bonus (extra credit). $O(n)$ time and $O(1)$ space without a queue (`josephusFast`), for any $n$.

| Input | Expected output |
|---|---|
| n=7, k=3 | 4 |
| n=5, k=2 | 3 |
| n=6, k=3 | 1 |
| n=41, k=3 | 31 |
| n=1, k=5 | 1 |

### Problem 46. Reverse the first $k$ items of a queue *Easy*

Reverse the order of the first $k$ items of the queue `q`; the other items keep their order and stay behind them.

```
void reverseFirstK(queue *q, int k);   // 0 <= k <= size
```

Permitted operations. Queue operations on `q`; one stack.

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| [1,2,3,4,5], k=3 | [3,2,1,4,5] |
| [1,2,3,4,5], k=5 | [5,4,3,2,1] |
| [1,2,3,4,5], k=0 | [1,2,3,4,5] |
| [10,20,…,100], k=5 | [50,40,30,20,10,60,70,80,90,100] |

### Problem 47. Number of recent calls *Easy*

A counter receives pings at times `t` (in milliseconds), each strictly later than the previous one. `rcPing(r, t)` records the ping and returns how many pings happened in the inclusive range $[t - 3000, t]$.

```
typedef struct { queue times; } recentCounter;
void rcInit(recentCounter *r);
int rcPing(recentCounter *r, int t);
```

Permitted operations. Queue operations on `times`.

Constraints. At most 100 pings fall inside any window.

Required. $O(1)$ amortised per ping.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `ping 1, 100, 3001, 3002` | `1, 2, 3, 3` |
| `ping 1, 5000, 5001` | `1, 1, 2` |

### Problem 48. Interleave the two halves of a queue *Medium*

A queue of even length holds $x_1 \dots x_m\, y_1 \dots y_m$. Rearrange it into $x_1 y_1 x_2 y_2 \dots x_m y_m$.

```
void interleaveHalves(queue *q);
```

Permitted operations. Queue operations on `q`; one stack (no second queue, no array).

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| `[1,2,3,4,5,6,7,8]` | `[1,5,2,6,3,7,4,8]` |
| `[11,12,…,20]` | `[11,16,12,17,13,18,14,19,15,20]` |
| `[1,2]` | `[1,2]` |

### Problem 49. First negative number in every window *Medium*

For every window `a[i..i+k-1]` of $k$ consecutive items, write into `out` the first negative number of the window, or 0 if it has none. Return the number of windows, $n - k + 1$.

```
int firstNegatives(int a[], int n, int k, int out[]);
```

Permitted operations. Array access to `a` and `out`; one queue.

Constraints. $1 \le k \le n \le 10^5$.

Required. $O(n)$ time ($O(nk)$ earns partial credit).

| Input | Expected output |
|---|---|
| `[12,-1,-7,8,-15,30,16,28], k=3` | `[-1,-1,-7,-15,-15,0]` |
| `[-8,2,3,-6,10], k=2` | `[-8,0,-6,-6]` |
| `[1,2,3], k=3` | `[0]` |

## 10.5 Deques

"Deque operations only" means `dqInit`, `dqPushFront`, `dqPushBack`, `dqPopFront`, `dqPopBack`, `dqFront`, `dqBack`, `dqEmpty`, `dqSize` of Listing 6.1. The deque has capacity 100; in the window problems it holds at most $k \le 100$ indices.

Juevesano D. 2026 All rights reserved. 

**Problem 50.** Design a circular deque *Medium*

Implement a deque of capacity $k$ in a circular array. The insert and delete operations return 1 on success and 0 if the deque is full (insert) or empty (delete); `cdGetFront` and `cdGetRear` return $-1$ if the deque is empty.

```
typedef struct { int items[1000]; int cap; int front; int size; } circularDeque;
void cdInit(circularDeque *d, int k);
int cdInsertFront(circularDeque *d, int x);
int cdInsertLast(circularDeque *d, int x);
int cdDeleteFront(circularDeque *d);
int cdDeleteLast(circularDeque *d);
int cdGetFront(circularDeque *d);
int cdGetRear(circularDeque *d);
int cdIsEmpty(circularDeque *d);
int cdIsFull(circularDeque *d);
```

Permitted operations. Array access to the fields.

Constraints. $1 \le k \le 1000$.

Required. Every operation $O(1)$; no shifting.

| Input | Expected output |
|---|---|
| `k=3: insertLast 1, insertLast 2, insertFront 3, insertFront 4` | `1, 1, 1, 0` |
| `getRear, isFull, deleteLast, insertFront 4, getFront` | `2, 1, 1, 1, 4` |
| `k=1: getFront, insertFront 7, insertLast 8, getRear` | `-1, 1, 0, 7` |

Juevesano D. 2026 All rights reserved. 

## Problem 51. A linked deque with a sentinel *Medium*

Implement a deque on a circular doubly linked list with one sentinel node stored inside the struct: `sentinel.next` is the first item and `sentinel.prev` the last; when the deque is empty both point to the sentinel. Write the two helpers first and implement the six operations with them, without any `if` that tests for `NULL`.

```
typedef struct dqnode { int item; struct dqnode *prev, *next; } dqnode;
typedef struct { dqnode sentinel; int size; } linkedDeque;
void ldInit(linkedDeque *d);
int ldLinkBetween(linkedDeque *d, dqnode *a, dqnode *b, int x);
void ldUnlink(linkedDeque *d, dqnode *n, int *x);
int ldPushFront(linkedDeque *d, int x);   int ldPushBack(linkedDeque *d, int x);
int ldPopFront(linkedDeque *d, int *x);   int ldPopBack(linkedDeque *d, int *x);
int ldFront(linkedDeque *d, int *x);      int ldBack(linkedDeque *d, int *x);
void ldFree(linkedDeque *d);
```

Permitted operations. `malloc`, `free`, pointer manipulation.

Required. Every operation $O(1)$ (`ldFree`: $O(n)$); no leaks.

Bonus (extra credit). Explain why `linkedDeque e = d;` produces a broken deque.

| Input | Expected output |
|---|---|
| `pushBack 1, pushBack 2, pushFront 0` | `0 1 2` |
| `popBack, popFront` | `2, 0` |
| `front, back` | `1, 1` |
| `popBack, popBack` | `1, fails` |

## Problem 52. Valid palindrome *Easy*

Return 1 if the string `s` reads the same forwards and backwards when only its letters and digits are kept and upper case is treated as lower case; else 0.

```
int isPalindromeText(char s[]);
```

Permitted operations. Character reads of `s`; deque operations. No `ctype.h` (write the tests yourself).

Constraints. $1 \le |s| \le 100$ letters and digits after filtering (the deque has capacity 100).

Required. $O(n)$ time.

Bonus (extra credit). $O(1)$ extra space with two indices instead of a deque.

| Input | Expected output |
|---|---|
| `A man, a plan, a canal: Panama` | `1` |
| `race a car` | `0` |
| `" " (one space)` | `1` |
| `0P` | `0` |
| `Was it a car or a cat I saw?` | `1` |

Juevesano D. 2026 All rights reserved. 

### Problem 53. Sliding window maximum *Medium*

For every window `a[i..i+k-1]` write its maximum into `out`; return the number of windows, $n - k + 1$.

```
int maxSlidingWindow(int a[], int n, int k, int out[]);
```

Permitted operations. Array access to `a` and `out`; one deque.

Constraints. $1 \le k \le n \le 10^5$, $k \le 100$.

Required. $O(n)$ time ($O(nk)$ earns partial credit).

| Input | Expected output |
|---|---|
| `[1,3,-1,-3,5,3,6,7], k=3` | `[3,3,5,5,6,7]` |
| `[1], k=1` | `[1]` |
| `[9,1,8,2,7,3], k=2` | `[9,8,8,7,7]` |
| `[4,2,12,3,8,8,1,5], k=3` | `[12,12,12,8,8,8]` |

### Problem 54. Longest subarray with absolute difference at most a limit *Medium*

Return the length of the longest non-empty contiguous subarray in which the difference between the largest and the smallest item is at most `limit`.

```
int longestSubarray(int a[], int n, int limit);
```

Permitted operations. Array reads; two deques.

Constraints. $1 \le n \le 10^5$, $0 \le$ *limit*; no valid window is longer than 100.

Required. $O(n)$ time.

| Input | Expected output |
|---|---|
| `[8,2,4,7], limit 4` | `2` |
| `[10,1,2,4,7,2], limit 5` | `4 ([2,4,7,2])` |
| `[4,2,2,2,4,4,2,2], limit 0` | `3` |

### Problem 55. Front-middle-back queue *Medium*

Design a queue that supports `push` and `pop` at the front, in the middle and at the back. When there are two middle positions, the *front-most* one is used: `pushMiddle` inserts so that the new item becomes item $\lfloor n/2 \rfloor$ (0-based, $n$ the size before), and `popMiddle` removes item $\lfloor (n-1)/2 \rfloor$. The pops return the removed item, or $-1$ if the queue is empty.

```
typedef struct { deque left; deque right; } fmbQueue;
void fmbInit(fmbQueue *q);
void fmbPushFront(fmbQueue *q, int x);
void fmbPushMiddle(fmbQueue *q, int x);
void fmbPushBack(fmbQueue *q, int x);
int fmbPopFront(fmbQueue *q);
int fmbPopMiddle(fmbQueue *q);
int fmbPopBack(fmbQueue *q);
```

Permitted operations. Deque operations on the two member deques.

Required. Every operation $O(1)$.

Juevesano D. 2026 All rights reserved. 

| Input | Expected output |
|---|---|
| `pushFront 1, pushBack 2, pushMiddle 3, pushMiddle 4` | `queue [1,4,3,2]` |
| `popFront, popMiddle, popMiddle, popBack, popFront` | `1, 3, 4, 2, -1` |

### Problem 56. ArrayDeque: positional insertion and removal *Medium*

In the ArrayDeque of the course notes, item $i$ ($0 \le i < n$) lives at `a[(j+i) % 100]`. Implement `adAdd(d, i, x)`, which inserts $x$ so that it becomes item $i$, and `adRemove(d, i, &x)`, which removes item $i$. Each must move the *shorter* side of the deque: the items before position $i$ (moving `j`) or the items after it.

```
typedef struct { int a[100]; int j; int n; } arrayDeque;
void adInit(arrayDeque *d);
int adGet(arrayDeque *d, int i);
int adAdd(arrayDeque *d, int i, int x);       // 0 on failure
int adRemove(arrayDeque *d, int i, int *x);   // 0 on failure
```

Permitted operations. Array access to the fields.

Required. $O(1 + \min(i, n - i))$ time per operation.

| Input | Expected output |
|---|---|
| `add(0,1), add(1,2), add(2,3), add(0,0)` | `[0,1,2,3]` |
| `add(2,9)` | `[0,1,9,2,3]` |
| `remove(1); get(2)` | `returns 1, [0,9,2,3]; 2` |
| `add(5,7); remove(3)` | `fails; returns 3, [0,9,2]` |

### Problem 57. Jump game VI *Medium*

You start on index 0 of the array `a` and must reach index $n - 1$. From index $i$ you may jump to any index $i + 1, \dots, i + k$ (inside the array). Your score is the sum of `a[j]` over all indices $j$ you land on, including the first and the last. Return the maximum score.

```
int maxResult(int a[], int n, int k);
```

Permitted operations. Array access; one deque.

Constraints. $1 \le n \le 10^4$, $1 \le k \le 98$, $|a[i]| \le 10^4$.

Required. $O(n)$ time ($O(nk)$ earns partial credit).

| Input | Expected output |
|---|---|
| `[1,-1,-2,4,-7,3], k=2` | `7 (path 1, -1, 4, 3)` |
| `[10,-5,-2,4,0,3], k=3` | `17` |
| `[1,-5,-20,4,-1,3,-6,-3], k=2` | `0` |

Juevesano D. 2026 All rights reserved. 

Part III

# Solutions and Explanations

Juevesano D. 2026 All rights reserved. 

Chapter 11

# Solutions to Section A

Model answers to the concept questions of Chapter 8; the numbers agree.

**1.** An *abstract data type* is a mathematical model consisting of a set of values and a set of operations on them; the representation is hidden. A *data structure* is a concrete way to realise an ADT in memory (an array, linked nodes), and it determines the costs of the operations. Relatives of the List ADT: the stack, the queue and the deque.

**2.** (a) True. For $n \geq 2$, $\lg n \geq 1$, so $23 \leq 23 \lg n$; take $c = 23$, $n_0 = 2$. (b) False. Suppose $5 \lg n \leq c$ for all $n \geq n_0$; for $n > \max(n_0, 2^{c/5})$ we get $5 \lg n > c$, a contradiction; the function is unbounded. (c) True. For $n \geq 1$, $5n - 7 \leq 5n$; take $c = 5$, $n_0 = 1$.

**3.** (a) $\Theta(1)$. (b) $\Theta(n)$ (every item moves). (c) $\Theta(1)$. (d) $\Theta(n)$ (the predecessor of the tail must be found from the head). (e) $\Theta(1)$. (f) $\Theta(n)$ in the worst case: the cost is $\Theta(\min(i, n - i))$, maximal near the middle. (g) $\Theta(n)$. (h) $\Theta(1)$: one modular step of `front` and one write.

**4.** A struct containing an array is copied with the array, so the callee changes its own copy: the caller sees no change. For a struct holding a pointer `head`, the pointer is copied but the nodes are shared: `l.head->item=100` is visible to the caller, while an assignment such as `l.head=NULL` is not. A raw array parameter `int a[]` is really a pointer to the first element, so the change is visible. (Tracing 2.)

**5.** It is not a bug. The valid items are exactly `items[0..size-1]`; `size` is the boundary, and cells beyond it are unspecified. The stale value matters only when some code reads beyond `size` without checking (for instance an `itemAt` that indexes without testing $p \leq$ `size`), or when the cell owns a resource (e.g. a pointer that was not released). The next `push_back` simply overwrites it. (Tracing 4.)

**6.** `malloc/free` are C library functions that allocate raw bytes (returning `void *`) and run no constructor; `new/delete` are C++ operators that are type-aware and call constructors and destructors (`new[]` pairs with `delete[]`); the two families must not be mixed. A *dangling pointer* points to memory that has been freed or has gone out of scope: `int *p = (int*)malloc(4); free(p); *p = 1;`. A *memory leak* is allocated memory that is no longer reachable: `int *p = (int*)malloc(4); p = NULL;`.

**7.** The parameter `h` is a copy of the caller's head pointer. For a non-empty list the loop follows the copy to the real nodes and modifies them, which works; for an empty list the function assigns the new node to the local copy only, so the caller's `head` stays `NULL` and the node leaks. Repairs: (1) pass the address, `void append(node **h, int x);` (2) return the (possibly new) head and write `l.head = append(l.head, x)`. (Tracing 7, Problem 15.)

**8.** `insert(p,x)` moves the $s - p$ items at positions $p..s - 1$ one place to the right; `erase(p)`

Juevesano D. 2026 All rights reserved. 

moves the $s - p - 1$ items at positions $p + 1..s - 1$ one place to the left. Both are worst at $p = 0$: $s$ and $s - 1$ moves, i.e. $\Theta(n)$; at the right end they need 0 moves.

**9.** With doubling, the copying done at sizes $1, 2, 4, \ldots, 2^k < n$ totals less than $2n$, plus $n$ writes: at most $3n$ operations, hence $O(1)$ amortised (Theorem 2.3). Growing by a constant $c = 10$ copies $10, 20, 30, \ldots$ items, a total of $10(1 + 2 + \cdots + n/10) = \Theta(n^2)$, i.e. $\Theta(n)$ amortised per `push_back`.

**10.** Suppose the array has capacity $2m$ and holds $m$ items. Halving gives capacity $m$, completely full: the next push must double again, and the next pop halves again. Alternating push/pop costs $\Theta(n)$ per operation (*thrashing*). The safe rule is to shrink only when $n \leq cap/4$ (or, as in the course, when $cap \geq 3n$): after the shrink the array is about half full, and $\Omega(n)$ cheap operations must occur before the next resize.

**11.** The array must be sorted (non-decreasing); the search takes $\Theta(\log n)$ comparisons. `insert` still needs to open a gap in a contiguous array: the items after the position are shifted, up to $n$ of them, so the worst case is $\Theta(n)$. The $\log n$ saving is for comparisons only.

**12.** Front: `items[front]`. Back: `items[(front+size-1) % CAP]`. Next free cell: `items[(front+size) % CAP]`. A full queue has `size` $= 4$; the back index is `(front` $+ 3)$ mod $4 = 2$ exactly when `front` $= 3$: so `(front, size)` $= (3, 4)$. (Tracing 5.)

**13.** `push_back` improves from $\Theta(n)$ (walk to the end) to $\Theta(1)$; `push_front` and `pop_front` are $\Theta(1)$ in both. `pop_back` stays $\Theta(n)$ even with a tail pointer.

**14.** After removing the tail, the new tail is the node before it. A node in an SLL has no `prev` link, so that node can only be found by walking from `head` until `cur->next==tail`, $n - 2$ steps. The tail pointer helps only to reach the old tail, not its predecessor.

**15.** `c->prev=a; c->next=b; a->next=c; b->prev=c;` and `size++`. If `a` is `NULL` (insertion at the front) then `head=c` instead of `a->next=c`; if `b` is `NULL` (at the end) then `tail=c` instead of `b->prev=c`.

**16.** (1) Only node: `head = tail = NULL`. (2) Head, with successor: `head = cur->next; head->prev = NULL`. (3) Tail, with predecessor: `tail = cur->prev; tail->next = NULL`. (4) Middle: `cur->prev->next = cur->next; cur->next->prev = cur->prev`. In all cases then `free(cur); size--`. The four cases collapse into two `if/else` statements: predecessor exists? set its `next`, else set `head`; successor exists? set its `prev`, else set `tail`.

**17.** A sentinel (dummy) node holds no item and is linked into a circular list so that no pointer is ever `NULL`. It removes (i) the special case of an empty list and (ii) the separate updates of `head` and `tail`: insertion before a node and removal of a node are the same few assignments everywhere. Costs: one extra node, and traversals must stop at the sentinel rather than at `NULL`.

**18.** Two pointers start at the head; per iteration `slow` moves one node, `fast` two. When `fast` cannot make another full step (`fast==NULL` or `fast->next==NULL`) `slow` has covered half the distance, so it is at the middle. With the loop condition `fast!=NULL && fast->next!=NULL`, odd length gives the exact middle, even length gives the *second* of the two middle nodes (for $1, 2, 3, 4$ it stops on 3).

Juevesano D. 2026 All rights reserved. 

**19.** At the top of each iteration the nodes before `cur` form a list in reversed order whose head is `prev`, and the nodes from `cur` on are still in the original order. Time $\Theta(n)$, extra space $O(1)$.

**20.** A *shallow copy* copies only the references (the struct with `head`, `tail`, `size`); both variables share the same nodes. A *deep copy* allocates new nodes with copies of the items. After `l = m` and the destruction of `m`, the nodes seen by `l` have been freed: `l.head` and `l.tail` are dangling, any use of `l` is undefined behaviour, and destroying `l` again is a double free. (Tracing 20.)

**21.** Splicing moves a chain of nodes from one list into another by changing pointers; nothing is allocated or copied. To place all of `m` after node `p` of `l`: let `q = p->next`; set `p->next = m.head` and `m.tail->next = q`; if `q` was `NULL` then `l.tail = m.tail`; add the sizes and empty `m`. The cost is $\Theta(1)$ only if `p` is already known (or is the tail) and `m` has a tail pointer; locating the $i$-th node costs $\Theta(i)$. (Tracing 19.)

**22.** Invariant: `mid` points to the node at index $\lfloor(\texttt{size}-1)/2\rfloor$. In an SLL, `middle`, `pushBack` and `popFront` are always $O(1)$; `pushFront` is $O(1)$ for even size but needs a backward step (a walk from the head, $\Theta(n)$) for odd size; `popBack` is $\Theta(n)$. Giving each node a `prev` pointer (a doubly linked list) makes every backward step $O(1)$ and hence all four end operations $O(1)$. (Tracing 16, Problem 24.)

**23.** A stack is a sequence accessed at one end only: `push`, `pop`, `top`, `empty`, `size`; last in, first out. Applications: bracket matching, infix-to-postfix conversion and postfix evaluation, undo/backtracking, the call stack (recursion), depth-first search. In a singly linked list the head should be the top, since `push_front` and `pop_front` are $\Theta(1)$ there, whereas `pop_back` is $\Theta(n)$.

**24.** Every item is moved from `in` to `out` at most once, because after the transfer it leaves from `out`. Charge 4 units to an enqueue (push onto `in`, and in advance: pop from `in`, push onto `out`, pop from `out`); the later transfer and the dequeue are then prepaid. $m$ operations therefore cost at most $4m$ stack operations plus $O(1)$ each for the tests, so $\Theta(1)$ amortised, although one dequeue that triggers a transfer of $n$ items costs $\Theta(n)$ (Theorem 5.2, Tracing 12).

**25.** In `([)]` each kind of bracket occurs equally often as opener and closer, yet the nesting is wrong: `)` arrives while `[` is the most recent unclosed opener, so the `(` is closed before the `[` opened inside it. A stack is the right tool: the closer must match the most recently opened bracket.

**26.** Postfix: `ab+c*de/-`. Prefix: `-*+abc/de`.

**27.** Tokens `5 1 2 + 4 * + 3 -`: push 5 → [5]; push 1 → [5, 1]; push 2 → [5, 1, 2]; `+`: $1 + 2 = 3$ → [5, 3]; push 4 → [5, 3, 4]; `*`: $3 \cdot 4 = 12$ → [5, 12]; `+`: $5 + 12 = 17$ → [17]; push 3 → [17, 3]; `-`: $17 - 3 = 14$ → [14]. The value is 14.

**28.** Operator stack (bottom to top) after each token: `a`: empty; `-`: `-`; `b`: `-`; `*`: `- *` (`*` binds tighter, nothing popped); `c`: `- *`; `+`: pop `*`, then `-` (equal precedence), push: `+`; `d`: `+`; `/`: `+ /`; `(`: `+ / (`; `e`; `-`: `+ / ( -`; `f`; `)`: pop `-`, discard `(`: `+ /`; end: pop `/`, `+`. Postfix: `abc*-def-/+`. With "strictly higher" the `-` of `8-3-2` is not popped when the second `-` arrives, giving `832--` $= 8 - (3 - 2) = 7$ instead of `83-2-` $= (8 - 3) - 2 = 3$: subtraction would become right-associative.

Juevesano D. 2026 All rights reserved. 

**29.** (a) Possible: `uuuoouuooo` (push 1,2,3; pop 3,2; push 4,5; pop 5,4,1). (b) Impossible: after 4 is printed the stack holds $1, 2, 3$ with 3 on top, so 1 cannot come next; the pattern *high, low, middle* is $4, 1, 3$ (or $4, 1, 2$). (c) Possible: `uououououo`. (d) Impossible: after `4 3 5` the stack holds $1, 2$ with 2 on top; pattern $4, 1, 2$. The number of stack permutations of 1..4 is the Catalan number $C_4 = \frac{1}{5}\binom{8}{4} = 14$ (Section 4.6, Tracing 14).

**30.** Answers: $[5, 25, 25, -1, 18, -1]$. Stack of waiting values after each item: $[4]$; $[5]$ (4 answered by 5); $[5, 2]$; $[25]$ (2 and 5 answered by 25); $[25, 7]$; $[25, 18]$ (7 answered by 18); 25 and 18 are left and get $-1$. Every index is pushed once and popped at most once, so the inner loop runs at most $n$ times *in total* over all rounds: $\Theta(n)$ (Theorem 4.5).

**31.** After the pushes: `all` $= [5, 3, 7, 3, 1]$, `mins` $= [5, 3, 3, 3, 1]$. After two pops: `all` $= [5, 3, 7]$, `mins` $= [5, 3, 3]$, and `min()` $= 3$. A single variable cannot be restored when the minimum is popped: the previous minimum is no longer known without scanning the whole stack, $\Theta(n)$.

**32.** (a) `(rear − front + CAP)` mod `CAP` $= (1 - 4 + 6) \bmod 6 = 3$ items (cells 4, 5, 0). (b) With only `front` and `rear`, an empty and a full queue would both have `front` $=$ `rear`. Leaving one cell unused separates them: empty iff `front` $=$ `rear`, full iff `(rear + 1)` mod `CAP` $=$ `front`; the capacity is `CAP` $- 1 = 5$.

**33.** Removal is $\Theta(1)$ only at the head of an SLL; insertion is $\Theta(1)$ at both ends (with a tail pointer). Dequeue removes, so it must happen at the head. When `dequeue` removes the last item it must set `tail = NULL` as well as `head`; otherwise `tail` dangles, and the next `enqueue` executes `tail->next = n` on freed memory and also fails to set `head`.

**34.** `-7/2` $= -3$ (division truncates toward zero); `-7%2` $= -1$; `7%-2` $= 1$ (the sign follows the left operand); `(0-1)%5` $= -1$; `(0-1+5)%5` $= 4$. With `front = (front - 1) % CAP`, a deque whose front is cell 0 gets `front` $= -1$ and the next write goes to `items[-1]`, outside the array. Adding `CAP` first keeps the left operand non-negative.

**35.** $[1]$, $[1, 2]$, $[3, 1, 2]$; `pop_back()` returns 2: $[3, 1]$; $[3, 1, 4]$; $[5, 3, 1, 4]$; `pop_front()` returns 5. Final contents: `3 1 4`.

**36.** When the maximum leaves the window from the front, a queue cannot tell which of the remaining items is now the largest without rescanning them. A deque lets the algorithm also discard from the *back* the items that can never be a maximum, so the front is always the answer. Maxima: $9, 8, 8, 7, 7$. Each index is pushed at the back exactly once and popped (from the front or the back) at most once: at most $n + n = 2n$ deque operations. (Section 6.5, Tracing 23.)

**37.** (a) $i = 2 < n/2 = 5$: the 2 items before position 2 move left: **2** moves. (b) $i = 7 \geq 5$: the $n - i = 3$ items from position 7 on move right: **3** moves. (c) $i = 5$ is not $< 5$: the $n-1-i = 4$ items after position 5 move left: **4** moves. The worst case is near the middle, about $n/2$ moves, because the shorter side is never longer than half the deque; an array list always moves the whole right side, up to $n$. (Tracing 22.)

**38.** Four in both cases: `n->prev`, `n->next`, `a->next`, `b->prev`, where `a` is the sentinel and `b` is the old first node — or, for an empty deque, the sentinel again. There is no special case. After `linkedDeque e = d;` the struct `e` contains a second sentinel at a different address; its `next`/`prev` point into `d`'s nodes, but those nodes point back to `d`'s sentinel, so a walk in `e` never returns to `e`'s sentinel, and an operation on `e` corrupts `d`. (Tracing 25.)

Juevesano D. 2026 All rights reserved. 

**39.** Removal order $3, 6, 2, 7, 5, 1$; the survivor is **4**. Recurrence with $k = 3$: $J(1) = 0, J(2) = 1, J(3) = 1, J(4) = 0, J(5) = 3, J(6) = 0, J(7) = 3$; the survivor's number is $J(7) + 1 = 4$. (Problem 45, Tracing 18.)

**40.** (a) Stack: undo the most recent action first. (b) Queue: print in order of arrival. (c) Deque: *back* pops the most recent page (stack end), the limit discards the oldest page (queue end). (d) Stack: a closing tag must match the most recently opened tag. (e) Deque: the monotonic deque of Section 6.5. (f) Queue.

Juevesano D. 2026 All rights reserved. 

Chapter 12

# Solutions to Section B

Each box is the real output of the program, captured when the book was built.

### Solution to Tracing 1.

```
3
11
0
23 16 4 8 15
16 4 8
1 0
8
16 4 99 5
5
```

The three appends succeed: $r = 3$. Both front insertions succeed, $r = 1{\cdot}10{+}1 = 11$. The list is now full, so `append(&l,42)` returns 0. `removeLast` only decrements `size`; `removeFirst` shifts left, leaving the raw array `[16 4 8 8 15]` with size 3. `setItem(&l,99,3)` is valid; position 4 exceeds the size and fails. *Twist:* `itemAt` does not check its position, so `itemAt(l,4)` reads the stale cell `items[3]`, which still holds 8. After `append(&l,5)` the same cell holds 5.

### Solution to Tracing 2.

```
A: 1 3
B: 100 9
C: 100 3
D: 100
E: 0 2 100
```

A: the struct (with its array) was copied; the caller sees nothing. B: through the pointer, both fields change. C: `a.items` decays to a pointer to the caller's array, so `items[0]` changes but `size` cannot. D: the same for a plain array. E: `llByValue` receives a copy of `head` and `size`; resetting them affects only the copy, but `l.head->item = 100` writes into the *shared* first node.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 3.

```
256 136 64 5 40 15
```

$c_1 = n^2 = 256$. $c_2 = 1 + 2 + \cdots + n = n(n+1)/2 = 136$. $c_3$: the outer loop runs for $i = 1, 2, 4, 8$, that is $\lg n = 4$ times, so $4 \cdot 16 = 64 = n \lg n$. $c_4$: $i = 16, 8, 4, 2, 1$, five times $= \lg n + 1$. $c_5$: for each $i$ the inner loop runs $\lceil (16 - i)/4 \rceil$ times; $4(4+3+2+1) = 40$, which is $\Theta(n^2)$ (about $n^2/8$). $c_6 = 1 + 2 + 4 + 8 = 15 = n - 1$: *twist* — a doubling outer loop whose inner loop runs $i$ times is $\Theta(n)$, not $\Theta(n \log n)$.

### Solution to Tracing 4.

```
size=6  [10 20 30 40 50 60 0 0]
size=5  [10 30 40 50 60 60 0 0]
size=6  [10 30 40 99 50 60 0 0]
1
size=5  [10 30 40 99 50 60 0 0]
size=7  [8 7 10 30 40 99 50 0]
0 0
size=7  [8 7 10 30 40 99 50 0]
```

`erase(1)` copies each item one cell left but does not clear the old last cell, so `60` appears twice in the raw array. `insert(3,99)` overwrites that stale copy when shifting right. `erase(5)` removes the real 60 ($r = 1$); the second `erase(5)` fails because now `size` $= 5$ ($r = 1 + 0 = 1$). The raw array is unchanged by the size decrement. Two front insertions shift everything. *Twist:* both later inserts fail — `insert(9,1)` because $9 >$ `size`, and `insert(8,1)` because $8 > 7 =$ `size`, even though the array still has one free cell.

### Solution to Tracing 5.

```
j=0 n=3 [1 2 3 0]
removed 1
j=1 n=2 [1 2 3 0]
j=1 n=4 [5 2 3 4]
0
j=3 n=2 [5 2 3 4]
j=3 n=4 [5 7 8 4]
4 5 7 8
j=3 n=0 [5 7 8 4]
```

The removal only advances `j`; the 1 stays in cell 0. Adding 4 and 5 fills cells 3 and then 0 (wrap-around). The queue is then full and `add(&q,6)` returns 0. Two removals take 2 and 3; `j` becomes 3. Then 7 and 8 go to cells $(3 + 2) \bmod 4 = 1$ and 2. The final loop prints the queue in queue order (4 5 7 8), not array order. *Twist:* after the queue is emptied, `show` still prints the old values; only `n` says the queue is empty.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 6.

```
pushBack 5  size=1 head=5 tail=5  [5]
pushFront 3 size=2 head=3 tail=5  [3 5]
pushBack 8  size=3 head=3 tail=8  [3 5 8]
popBack     size=2 head=3 tail=5  [3 5]
popBack     size=1 head=3 tail=3  [3]
popFront    size=0 head=NULL tail=NULL  []
0 3
pushFront 1 size=1 head=1 tail=1  [1]
pushBack 2  size=2 head=1 tail=2  [1 2]
```

`popBack` walks from the head to the node before the tail; the second `popBack` leaves one node, which is both head and tail. `popFront` on the one-node list must reset *both* pointers to `NULL`. *Twist:* the failed `popFront` returns 0 and leaves `x` at the last value successfully popped, 3.

### Solution to Tracing 7.

```
0 3
4 4
```

`appendA` receives a copy of the head pointer. On an empty list it stores the new node in that copy only: `a` stays `NULL` and the node leaks, three times. `appendB` receives the address of the caller's pointer and works. *Twist:* once the list is non-empty (`c` starts with 7), `appendA` works as well, since it modifies the `next` field of a shared node: both lists have length 4.

### Solution to Tracing 8.

```
5 7
size=2 head=5 tail=7
7 5
size=2
size=1 head=5 tail=5 prev=1
size=0 headIsNULL=1 tailIsNULL=1
```

Removing all 2's removes the first two nodes (the head changes twice), the one in the middle, and the last two (the tail changes twice); `5 7` remain and the backward display shows that the `prev` links are right. Erasing a missing value changes nothing. After removing 7, the single node 5 is both head and tail and its `prev` is `NULL`. Removing it empties the list: both pointers are `NULL`.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 9.

```
1 2 3 4 # 4 3 2 1
4 13
3 3
```

`mystery` prints each item on the way down, `#` at the end, and each item again on the way back: the list forwards and then backwards. `g` adds the even items and subtracts 1 for every odd item: $2+4-2=4$ and $2+8+6-3=13$. `h` counts the positions where an item is smaller than its successor: $1<2<3<4$ gives 3; in $3,5,2,8,6,9$ the ascents are $3<5$, $2<8$, $6<9$: also 3.

### Solution to Tracing 10.

```
3 10
3 1
2 3
4 5
```

Line 1: the stack gives its newest item 3, the queue its oldest 10. Then 3 joins the queue (`[20 3]`) and 10 the stack (`[1 2 10]`). Line 2: three pops succeed (10, 2, 1) and the fourth fails: $r=3$ and `x` keeps 1. Line 3: two dequeues succeed (20, 3): $r=2$, `y`$=3$. Last line: 4 and 5 go stack → queue (`5 4`) → stack (top 4). *Twist:* two reversals cancel, so the items come out in push order.

### Solution to Tracing 11.

```
abcd-*e/+ 4
```

The stack grows to `+ * (  -` (depth 4) inside the parentheses. At `)` the `-` is output and `(` discarded. At `/` the `*` has equal precedence and leaves first. *Twist:* `prec('(')` is 0, so the operator loop stops at a parenthesis without an explicit test.

### Solution to Tracing 12.

```
1 2 3 4 5 6 -1
in=0 out=0
```

The first dequeue moves 1, 2, 3 to `out`. The dequeues of 2 and 3 need no transfer, although 4, 5, 6 are waiting in `in`. Only when `out` is empty again are 4, 5, 6 moved. In total each item is moved once: *6* moves for 6 items. The last dequeue finds both stacks empty and returns $-1$.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 13.

```
34+      7   7
93-      6  -6
82/      4   0
27-3/   -1   0
234*-  -10  10
92-3-    4  10
52-4*   12 -12
```

`evalB` computes $b$ op $a$ with the operands swapped. For `+` and `*` the error is invisible. For `93-` it gives $3 - 9 = -6$; for `82/` it gives $2/8 = 0$. In `27-3/`, `evalA` computes $(2 - 7)/3 = -5/3 = -1$ (truncation toward zero, not $-2$), while `evalB` computes $3/(7 - 2) = 0$. The errors compound: for `92-3-`, `evalB` gives $3 - (2 - 9) = 10$.

### Solution to Tracing 14.

```
uuuooouo ok
uouououo ok
uuuou stuck: top 4, want 1
uuouuooo ok
uuuuo stuck: top 3, want 1
3
```

`3 2 1 4`: push three, pop three, push, pop. `1 2 3 4`: alternate. `3 1 2 4`: after 3 is printed, 2 is on top and 1 is below it, so the function pushes 4 (one more `u`) and then has nothing left to push: stuck with top 4. This is the forbidden pattern $3, 1, 2$. `2 4 3 1` works. `4 1 2 3`: after 4 the top is 3 and 1 is buried. Three of the five are stack permutations.

### Solution to Tracing 15.

```
100 -> 1  [100:1]
 80 -> 1  [100:1 80:1]
 60 -> 1  [100:1 80:1 60:1]
 70 -> 2  [100:1 80:1 70:2]
 60 -> 1  [100:1 80:1 70:2 60:1]
 75 -> 4  [100:1 80:1 75:4]
 85 -> 6  [100:1 85:6]
 85 -> 7  [100:1 85:7]
```

70 absorbs the 60 before it (span $1 + 1$). 75 absorbs `60:1` and `70:2`: span $1 + 1 + 2 = 4$. 85 absorbs `75:4` and `80:1`: span 6. The second 85 absorbs the first (`<=`): span 7. *Twist:* the stack keeps only 2 pairs at the end although 8 days were seen; the absorbed days are represented by the spans.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 16.

```
mPushBack 10  n=1 mid=10  [10]
mPushBack 20  n=2 mid=10  [10 20]
mPushBack 30  n=3 mid=20  [10 20 30]
mPushBack 40  n=4 mid=20  [10 20 30 40]
mPushFront 5  n=5 mid=20  [5 10 20 30 40]
mPushFront 1  n=6 mid=10  [1 5 10 20 30 40]
mPopFront     n=5 mid=20  [5 10 20 30 40]
mPopBack      n=4 mid=10  [5 10 20 30]
mPushBack 50  n=5 mid=20  [5 10 20 30 50]
mPopFront     n=4 mid=20  [10 20 30 50]
```

The middle is index $\lfloor (n-1)/2 \rfloor$: for $n = 1..4$ that is $0, 0, 1, 1$. A push at the back moves `mid` forward when $n$ becomes odd; a push at the front moves it *backward* when $n$ becomes even (from 20 to 10 at $n = 6$). Pops do the reverse.

### Solution to Tracing 17.

```
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=0 hi=4 mid=2 a[mid]=4
step 3: lo=0 hi=2 mid=1 a[mid]=4
step 4: lo=0 hi=1 mid=0 a[mid]=2
answer 1
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=5 hi=9 mid=7 a[mid]=15
step 3: lo=5 hi=7 mid=6 a[mid]=12
step 4: lo=5 hi=6 mid=5 a[mid]=9
answer 6
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=5 hi=9 mid=7 a[mid]=15
step 3: lo=8 hi=9 mid=8 a[mid]=15
answer 9
```

For $x = 4$ the search keeps going left after finding a 4, ending at the *first* 4 (index 1). For $x = 10$ the answer 6 is where 10 would be inserted. For $x = 20$ all items are smaller and the answer is $n = 9$, which is not a valid index: a caller must test `answer` $< n$ before reading `a[answer]`. Each search takes at most $\lceil \lg(n+1) \rceil = 4$ steps.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 18.

```
out:3 out:6 out:4 out:2 out:5
winner:1
```

Queue after each round: `[4 5 6 1 2]` (3 out), `[1 2 4 5]` (6 out), `[5 1 2]` (4 out), `[5 1]` (2 out); in the last round 5 and 1 both go to the back and the third person counted is 5 again, so 5 leaves. The winner is 1. Recurrence with $k = 3$: $J = 0, 1, 1, 0, 3, 0$ for $i = 1..6$, so the winner is $J(6) + 1 = 1$. *Twist:* when fewer than $k$ people remain, the count wraps around the circle more than once.

### Solution to Tracing 19.

```
[1 2 10 20 30 3 4 5]
[]
8 0
[1 2 10 7 8 9 20 30 3 4 5]
[]
11 0 tail=5
0 1
[1 2 10 7 8 9 20 30 3 4 5 100]
1 12 tail=100
```

`splice(&a,2,&b)` makes 10 the item at index 2. `a.head->next->next` is the node 10, so `c` goes after it. Splicing at the end must update `a.tail` (the tail stays 5 in the middle splices). An invalid position changes nothing, and `d` keeps its node until the valid splice at `a.size` appends it and moves the tail.

### Solution to Tracing 20.

```
99 2 3 | 3
0 99 2 3 | 4
1 77 3 | 3
```

`sh` shares its nodes with `orig`: changing `sh.head->item` changes `orig`'s first item to 99. `dp` has its own nodes. *Twist:* `pushFront(&sh,0)` changes only `sh.head` and `sh.size`: `orig` does not see the new node and still has size 3, although the two lists share their last three nodes.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 21.

```
f=3 n=4 raw=[1 4 0 3 2] deque=3 2 1 4
4 3
f=4 n=2 raw=[1 4 0 3 2] deque=2 1
f=3 n=5 raw=[1 5 6 7 2] deque=7 2 1 5 6
0
-1 4 -3
```

`pushBack(1)` writes cell 0. `pushFront(2)`: $f = (0 + 4) \bmod 5 = 4$. `pushFront(3)`: $f = 3$. `pushBack(4)` writes cell $(3 + 3) \bmod 5 = 1$. `popBack` returns cell $(3 + 3) \bmod 5 = 1$, i.e. 4; `popFront` returns 3. Then 5 and 6 fill cells 1 and 2 and 7 goes to cell 3; the deque is full and both further pushes fail ($r = 0$). The last line shows why the code adds 4: `(0-1)%5` is $-1$ in C, `(0-1+5)%5` is 4, and `-7/2` is $-3$.

### Solution to Tracing 22.

```
moved 0  j=0 raw=[10 20 30 40 0 0 0 0] list=10 20 30 40
moved 1  j=7 raw=[15 20 30 40 0 0 0 10] list=10 15 20 30 40
moved 2  j=7 raw=[15 20 25 30 40 0 0 10] list=10 15 20 25 30 40
moved 0  j=6 raw=[15 20 25 30 40 0 5 10] list=5 10 15 20 25 30 40
moved 1  j=6 raw=[15 20 25 30 45 40 5 10] list=5 10 15 20 25 30 45 40
```

`add(1,15)` with $n = 4$: $1 < 2$, so the left side (item 10) moves one cell left, to cell 7, and $j = 7$. `add(3,25)` with $n = 5$: $3 \geq 2$, so items 3 and 4 (30, 40) move right. `add(0,5)`: nothing moves, $j$ steps back to 6. `add(6,45)` with $n = 7$: only the last item moves. Total moves 4, against $3 + 2 + 6 + 1 = 12$ for an array list doing the same insertions.

### Solution to Tracing 23.

```
i=0 a[i]= 4 dq=[0]
i=1 a[i]= 2 dq=[0 1]
i=2 a[i]=12 dq=[2] max=12
i=3 a[i]= 3 dq=[2 3] max=12
i=4 a[i]= 8 dq=[2 4] max=12
i=5 a[i]= 8 dq=[5] max=8
i=6 a[i]= 1 dq=[5 6] max=8
i=7 a[i]= 5 dq=[5 7] max=8
```

At $i = 2$ the 12 pops both earlier indices. At $i = 4$ the 8 pops the 3. At $i = 5$ the front index 2 has left the window ($2 \leq 5 - 3$) and is popped from the front; then the new 8 pops the old 8 because the test is `<=`, so the deque keeps the *newer* of two equal values, which stays in the window longer. The maxima are 12, 12, 12, 8, 8, 8.

Juevesano D. 2026 All rights reserved. 

### Solution to Tracing 24.

```
1 2 3 | last=3
2 3 1 | last=1
1 2 3 | last=3
got 1  2 3 | last=3
2 3 4 | last=4
2 3 4
(empty)
0 4
```

The list is a circle, so moving `last` one step *rotates* the queue: the old front 1 becomes the rear. Two more steps bring it back. `deq` removes the front 1. After the loop empties the queue, `last` is `NULL`. *Twist:* the final failed `deq` returns 0 and `x` keeps 4, the last item removed.

### Solution to Tracing 25.

```
1 0 -1
0 1 2 3 | size 4
3 0
1 2 | size 2
2 1
1 0
9 1 2
0 1
```

The empty deque's sentinel points to itself; `ldPopBack` fails and leaves $x = -1$. After `0 1 2 3`, popping both ends gives 3 and 0. The last item is 2 and the first node's `prev` is the sentinel. *Twist:* the copy `e` has its own sentinel, but `e.sentinel.next` is `d`'s first node, whose `prev` still points to `d`'s sentinel $(0)$. After `ldPushFront(&d,9)` the copy still thinks its first item is 1 and its size is 2 — it is out of date and, once `d` is freed, dangling. The deque `d` is empty again after `ldFree`.

Juevesano D. 2026 All rights reserved. 

Chapter 13

# Solutions to Section C

Every solution below was compiled and tested: against the sample tests of Chapter 10, against a brute-force model on thousands of random inputs where one exists, and, for the linked structures, with a count of the live heap blocks (no leaks, no double frees). Solutions that already appear as listings in Part I are referred to rather than printed twice.

## 13.1 Arrays and array lists

### Solution to Problem 1.

Idea. Read the string run by run: remember the current character, count how far it repeats, then write the character and the count. A count can have several digits; find how many by repeated division by 10, then fill the digits *from the right*, because `run % 10` produces the last digit first. `compress` works in place because the write position never overtakes the read position: a run of length $r \geq 2$ occupies $r$ cells and is replaced by $1 + \lfloor \log_{10} r \rfloor + 1 \leq r$ cells.

**Listing 13.1:** Run-length encoding, in-place compression and decoding.

```c
// Writes the encoding of s into out (a letter followed by its run length).
// Returns the length of the encoding.
int rleEncode(char *s, char *out) {
    int i = 0;                      // read position in s
    int w = 0;                      // write position in out

    while (s[i] != '\0') {
        char c = s[i];
        int run = 0;
        while (s[i] == c) {         // count the run of c
            run++;
            i++;
        }
        out[w] = c;
        w++;

        int digits = 0;             // how many digits does run have?
        int t = run;
        while (t > 0) {
            digits++;
            t = t / 10;
        }
        for (int d = digits - 1; d >= 0; d--) {
            out[w + d] = '0' + run % 10;   // fill the digits from the right
            run = run / 10;
        }
```

Juevesano D. 2026 All rights reserved. 

```
        w = w + digits;
    }
    out[w] = '\0';
    return w;
}

// Compresses a[0..n-1] in place; a run of length 1 keeps no number.
// Returns the new length.
int compress(char *a, int n) {
    int r = 0;                      // read position
    int w = 0;                      // write position (never passes r)

    while (r < n) {
        char c = a[r];
        int run = 0;
        while (r < n && a[r] == c) {
            run++;
            r++;
        }
        a[w] = c;
        w++;

        if (run > 1) {
            int digits = 0;
            int t = run;
            while (t > 0) {
                digits++;
                t = t / 10;
            }
            for (int d = digits - 1; d >= 0; d--) {
                a[w + d] = '0' + run % 10;
                run = run / 10;
            }
            w = w + digits;
        }
    }
    return w;
}

// Bonus: decodes "a3b1" into "aaab". Returns the decoded length.
int rleDecode(char *e, char *out) {
    int i = 0;
    int w = 0;

    while (e[i] != '\0') {
        char c = e[i];
        i++;
        int count = 0;
        while (e[i] >= '0' && e[i] <= '9') {
            count = count * 10 + (e[i] - '0');
            i++;
        }
        for (int k = 0; k < count; k++) {
            out[w] = c;
            w++;
```

Juevesano D. 2026 All rights reserved. 

```
        }
    }
    out[w] = '\0';
    return w;
}
```

Complexity. $\Theta(n)$ time: each input character is read once and each output character written once. `compress` uses $O(1)$ extra space.

Common mistakes. Writing the count as a single character (`'0'+run`), which breaks for runs of 10 or more; writing the digits in reverse; forgetting the terminating `'\0'` in `rleEncode`; in `compress`, writing the count `1` for single characters, or testing `a[r]==c` without `r<n`.

## Solution to Problem 2.

Idea. $t_{ji} = a_{ij}$ is a direct double loop. In place, swap $a_{ij}$ and $a_{ji}$ for the cells *above the diagonal* only ($j > i$): visiting every pair twice swaps each pair back and leaves the matrix unchanged.

**Listing 13.2:** Transpose.

```
// t becomes the m x n transpose of the n x m matrix a.
void transpose(int a[][10], int n, int m, int t[][10]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            t[j][i] = a[i][j];
        }
    }
}

// Transposes a square n x n matrix in place.
void transposeInPlace(int a[][10], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {    // only above the diagonal
            int tmp = a[i][j];
            a[i][j] = a[j][i];
            a[j][i] = tmp;
        }
    }
}
```

Complexity. $\Theta(nm)$ and $\Theta(n^2)$ time; $O(1)$ extra space in place.

Common mistakes. Looping $j$ from 0 in the in-place version (double swap); mixing up $n$ and $m$ in the bounds of the rectangular case.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 3.

Idea. The sum $1 + 2 + \cdots + (m + 1) = (m + 1)(m + 2)/2$ is known; subtracting the sum of the array leaves the missing number. For the bonus, note that before the gap $a[i] = i + 1$ and from the gap on $a[i] = i + 2$: the predicate "$a[i] \neq i + 1$" is false...false true...true, so bisection finds the first index where it holds.

**Listing 13.3:** The missing number, linear and logarithmic.

```
// O(m): the sum 1 + 2 + ... + (m + 1) is known, so subtract the array sum.
int missingNumber(int a[], int m) {
    int n = m + 1;
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int i = 0; i < m; i++) {
        actual = actual + a[i];
    }
    return expected - actual;
}

// Bonus, O(log m): before the gap a[i] == i + 1, after it a[i] == i + 2.
// Binary search for the first index where a[i] != i + 1.
int missingNumberFast(int a[], int m) {
    int lo = 0;
    int hi = m;                          // the answer index is in [0, m]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == mid + 1) lo = mid + 1;   // gap is to the right
        else hi = mid;                         // gap is here or to the left
    }
    return lo + 1;
}
```

Complexity. $\Theta(m)$ and $\Theta(\log m)$ time; $O(1)$ space.

Common mistakes. Using $m(m + 1)/2$ (the range has $m + 1$ numbers); in the bisection, initialising `hi=m-1`, which misses the case where the missing number is $m + 1$.

## Solution to Problem 4.

Idea. `lowerBound` and `upperBound` are the bisection of Section 2.4 with `<` and `<=`. Then `search` is a lower bound plus one check, `count` is the difference of the two bounds, `insert` opens a gap at the upper bound (so equal items keep their arrival order), and `erase` closes the gap at the lower bound.

**Listing 13.4:** A sorted array list.

```
typedef struct { int items[100]; int size; } slist;

// Index of the first item >= x (size if every item is smaller).
int lowerBound(slist *l, int x) {
    int lo = 0;
    int hi = l->size;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
```

```
        if (l->items[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// Index of the first item > x (size if no item is larger).
int upperBound(slist *l, int x) {
    int lo = 0;
    int hi = l->size;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (l->items[mid] <= x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// Index of the first occurrence of x, or -1.
int search(slist *l, int x) {
    int p = lowerBound(l, x);
    if (p < l->size && l->items[p] == x) return p;
    return -1;
}

// Inserts x so the list stays sorted. Returns 0 if the list is full.
int insert(slist *l, int x) {
    if (l->size == 100) return 0;
    int p = upperBound(l, x);                 // slot after equal items
    for (int i = l->size; i > p; i--) {       // shift the tail right
        l->items[i] = l->items[i - 1];
    }
    l->items[p] = x;
    l->size++;
    return 1;
}

// Removes one occurrence of x. Returns 1 if x was found, else 0.
int erase(slist *l, int x) {
    int p = search(l, x);
    if (p == -1) return 0;
    for (int i = p; i < l->size - 1; i++) {   // shift the tail left
        l->items[i] = l->items[i + 1];
    }
    l->size--;
    return 1;
}

// Number of occurrences of x in O(log n).
int count(slist *l, int x) {
    return upperBound(l, x) - lowerBound(l, x);
}
```

Complexity. Bounds, `search`, `count`: $\Theta(\log n)$. `insert`, `erase`: $\Theta(\log n)$ comparisons plus up to $n$ moves, $\Theta(n)$.

Juevesano D. 2026 All rights reserved. 

Common mistakes. Reading `items[p]` when $p =$ `size` (not found and larger than every item); shifting in the wrong direction (left-to-right when opening a gap overwrites every item with the same value); counting by a linear scan from the lower bound, which is $\Theta(n)$ when all items are equal.

## Solution to Problem 5.

Idea. Reduce $k$ modulo $n$. Rotating right by $k$ moves the last $k$ items to the front. Reverse the whole array, then reverse the first $k$ items and the remaining $n - k$ items separately; each block is now in its original order but the blocks have swapped places.

**Listing 13.5:** Rotation by three reversals.

```
void reverse(int a[], int lo, int hi) {
    while (lo < hi) {
        int tmp = a[lo];
        a[lo] = a[hi];
        a[hi] = tmp;
        lo++;
        hi--;
    }
}

// Rotates a[0..n-1] to the right by k places.
void rotate(int a[], int n, int k) {
    if (n == 0) return;
    k = k % n;
    reverse(a, 0, n - 1);   // [1 2 3 4 5 6 7] -> [7 6 5 4 3 2 1]
    reverse(a, 0, k - 1);   // first k items   -> [5 6 7 4 3 2 1]
    reverse(a, k, n - 1);   // remaining items -> [5 6 7 1 2 3 4]
}
```

Complexity. $\Theta(n)$ time (each item is swapped at most twice), $O(1)$ space.

Common mistakes. Forgetting `k % n` ($k$ may be $10^9$); computing `k % n` when $n = 0$ (division by zero); rotating left instead of right (reversing the blocks in the other order).

## Solution to Problem 6.

Idea. Two indices walk through `a` and `b`; at each step the smaller front item is copied. Comparing with `<=` copies `a`'s item on ties (stability). When one array is exhausted, the rest of the other is copied.

**Listing 13.6:** Merging two sorted arrays.

```
// Merges sorted a[0..n-1] and b[0..m-1] into c. Returns n + m.
int mergeArrays(int a[], int n, int b[], int m, int c[]) {
    int i = 0;
    int j = 0;
    int k = 0;

    while (i < n && j < m) {
        if (a[i] <= b[j]) {          // <= keeps equal items of a first
            c[k] = a[i];
```

```
            i++;
        } else {
            c[k] = b[j];
            j++;
        }
        k++;
    }
    while (i < n) {                 // one of these loops does nothing
        c[k] = a[i];
        i++;
        k++;
    }
    while (j < m) {
        c[k] = b[j];
        j++;
        k++;
    }
    return k;
}
```

Complexity. $\Theta(n+m)$: each item is copied once.

Common mistakes. Forgetting the two tail loops; using `<` (the order of equal items is then `b` first, which the statement forbids).

## Solution to Problem 7.

Idea. Keep a write index $w$: `a[0..w-1]` holds the distinct values seen so far. Since the array is sorted, a value is new exactly when it differs from `a[w-1]`.

**Listing 13.7:** Remove duplicates from a sorted array.

```
// Keeps one copy of each value of the sorted array. Returns the new length.
int dedupSorted(int a[], int n) {
    if (n == 0) return 0;
    int w = 1;                      // a[0..w-1] holds the unique items
    for (int r = 1; r < n; r++) {
        if (a[r] != a[w - 1]) {     // a new value
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Comparing with `a[r-1]` instead of `a[w-1]` (works here, but not for the "at most two copies" variant); not handling $n = 0$.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 8.

Idea. Two pointers from both ends. If the sum is too small, only a larger left item can help: `lo++`. If it is too large, `hi--`. No pair is skipped: when `lo` moves, `a[lo]` could only pair with indices $\leq$ `hi`, all of which give too small a sum.

**Listing 13.8:** Two-sum in a sorted array.

```
// Finds i < j with a[i] + a[j] == target in a sorted array.
// Returns 1 and sets *pi, *pj, or returns 0 if there is no such pair.
int twoSum(int a[], int n, int target, int *pi, int *pj) {
    int lo = 0;
    int hi = n - 1;
    while (lo < hi) {
        int sum = a[lo] + a[hi];
        if (sum == target) {
            *pi = lo;
            *pj = hi;
            return 1;
        }
        if (sum < target) lo++;      // need a bigger sum
        else hi--;                   // need a smaller sum
    }
    return 0;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space (the double loop is $\Theta(n^2)$).

Common mistakes. The condition `lo<=hi`, which may use the same index twice; moving both pointers at once.

## Solution to Problem 9.

Idea. Kadane's algorithm. Let $E_i$ be the best sum of a subarray *ending* at $i$. Either it extends the best one ending at $i-1$, or it starts afresh at $i$: $E_i = a_i + \max(E_{i-1}, 0)$. The answer is $\max_i E_i$.

**Listing 13.9:** Maximum subarray sum, $\Theta(n)$ and $\Theta(n^2)$.

```
// Largest sum of a non-empty contiguous subarray (n >= 1).
int maxSubarray(int a[], int n) {
    int endingHere = a[0];           // best sum of a subarray ending at i
    int best = a[0];
    for (int i = 1; i < n; i++) {
        if (endingHere > 0) endingHere = endingHere + a[i];   // extend
        else endingHere = a[i];                               // start over
        if (endingHere > best) best = endingHere;
    }
    return best;
}

// Slower version, O(n^2): try every starting index with a running sum.
int maxSubarraySlow(int a[], int n) {
    int best = a[0];
    for (int i = 0; i < n; i++) {
```

```
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum = sum + a[j];
            if (sum > best) best = sum;
        }
    }
    return best;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Initialising `best = 0`, which returns 0 for an all-negative array (the sub-array must be non-empty); resetting to 0 instead of to `a[i]`.

## Solution to Problem 10.

Idea. Copy every non-zero item to the next write position (a stable compaction), then fill the rest with zeros.

**Listing 13.10:** Move zeroes.

```
// Moves all zeros to the end; the other items keep their order.
void moveZeroes(int a[], int n) {
    int w = 0;                      // next place for a non-zero item
    for (int r = 0; r < n; r++) {
        if (a[r] != 0) {
            a[w] = a[r];
            w++;
        }
    }
    while (w < n) {
        a[w] = 0;
        w++;
    }
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Swapping a zero with the *last* item, which destroys the order; using a second array.

## Solution to Problem 11.

Idea. Boyer–Moore voting. Pair off two different items and discard both: the majority item, which occurs more than half the time, cannot be paired off completely, so it survives. The variable `votes` is the number of unpaired copies of `candidate`.

**Listing 13.11:** Majority element.

```
// Returns the element that appears more than n/2 times (it always exists).
// Boyer-Moore voting: different items cancel each other in pairs.
int majority(int a[], int n) {
    int candidate = a[0];
    int votes = 0;
    for (int i = 0; i < n; i++) {
```

Juevesano D. 2026 All rights reserved. 

```
        if (votes == 0) candidate = a[i];
        if (a[i] == candidate) votes++;
        else votes--;
    }
    return candidate;
}

// Slower version, O(n^2): count every item.
int majoritySlow(int a[], int n) {
    for (int i = 0; i < n; i++) {
        int c = 0;
        for (int j = 0; j < n; j++) {
            if (a[j] == a[i]) c++;
        }
        if (c > n / 2) return a[i];
    }
    return -1;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space; the counting version is $\Theta(n^2)$.

Common mistakes. Assuming the most frequent item is the answer when no majority is guaranteed (then a second pass must verify the candidate).

## Solution to Problem 12.

Idea. A clockwise rotation sends $(i, j)$ to $(j, n-1-i)$. That is the transpose $(i, j) \mapsto (j, i)$ followed by the reflection of each row $(j, i) \mapsto (j, n-1-i)$; both are in-place swaps.

**Listing 13.12:** Rotating a matrix by 90° clockwise.

```
// Rotates an n x n matrix 90 degrees clockwise, in place.
// Clockwise rotation = transpose, then reverse every row.
void rotate90(int a[][10], int n) {
    for (int i = 0; i < n; i++) {             // transpose
        for (int j = i + 1; j < n; j++) {
            int tmp = a[i][j];
            a[i][j] = a[j][i];
            a[j][i] = tmp;
        }
    }
    for (int i = 0; i < n; i++) {             // reverse each row
        int lo = 0;
        int hi = n - 1;
        while (lo < hi) {
            int tmp = a[i][lo];
            a[i][lo] = a[i][hi];
            a[i][hi] = tmp;
            lo++;
            hi--;
        }
    }
}
```

Complexity. $\Theta(n^2)$ time, $O(1)$ space.

Juevesano D. 2026 All rights reserved. 

Common mistakes. Reversing the columns instead of the rows (counter-clockwise rotation); transposing with $j$ from 0.

## 13.2 Linked lists

### Solution to Problem 13.

Idea. Walk the list once. For a node `cur` holding $x$, make its predecessor skip it (or, if it has none, move `head`), make its successor skip it backwards (or move `tail`), free it and decrease `size`. The successor must be saved *before* the node is freed. The two `if/else` statements cover the four cases of Question 16 of Section A: only node, head, tail, middle.

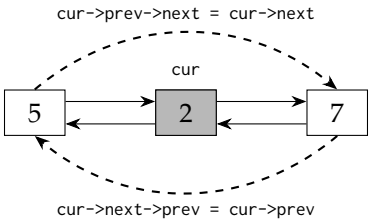



**Figure 13.1:** Unlinking a middle node of a DLL: two pointer updates, then `free(cur)`.

**Listing 13.13:** eraseItems.

```
// Removes every node whose item equals x from the doubly linked list.
void eraseItems(list *l, int x) {
    node *cur = l->head;

    while (cur != NULL) {
        node *nxt = cur->next;       // save it: cur may be freed below

        if (cur->item == x) {
            // make the node before cur skip it (or move the head)
            if (cur->prev != NULL) cur->prev->next = cur->next;
            else l->head = cur->next;

            // make the node after cur skip it (or move the tail)
            if (cur->next != NULL) cur->next->prev = cur->prev;
            else l->tail = cur->prev;

            free(cur);
            l->size--;
        }
        cur = nxt;
    }
}
```

The departmental test program, reconstructed from the sample output, is shown below; it reproduces the sample output of the statement character for character.

```
// The test program around eraseItems (reconstructed from the sample output).
// It processes test cases until the end of the input.
```

```
int main() {
    int n, x, item;
    list l;
    while (scanf("%d", &n) == 1) {
        init(&l);
        for (int i = 0; i < n; i++) {
            scanf("%d", &item);
            append(&l, item);
        }
        scanf("%d", &x);
        display(&l);
        eraseItems(&l, x);
        display(&l);
        printf("New size: %d\n", l.size);
        printf("Displaying from tail down to head:\n");
        displayBackward(&l);
        if (l.size == 0) {
            if (l.head == NULL && l.tail == NULL)
                printf("Yes, correct. Both the head and tail are NULL.\n");
            else
                printf("Wrong! The head or tail is not NULL.\n");
        }
        destroy(&l);
        printf("----\n");
    }
    return 0;
}
```

Complexity. $\Theta(n)$ time, one pass; $O(1)$ extra space.

Common mistakes. Reading `cur->next` after `free(cur)`; updating `head` but not `head->prev` (here done by the second `if`); forgetting `tail` when the last node is removed (the backward display then starts from a freed node); calling `search` and `erase` in a loop. Such a loop restarts from the head after every removal. On `3 3 3 3 3` that costs nothing, since every match is at the head, but on `1 1 1 1 1 3 3 3 3 3` each of the $n/2$ searches walks past the same $n/2$ ones: $\Theta(n^2)$ in total (bonus).

## Solution to Problem 14.

Idea. `m` is passed by value, but only the struct is copied: `m.head` points to the caller's nodes. Appending those nodes to `l` would make `l` share them, and destroying `m` would free them. So each item is *copied* into a new node linked at the tail of `l`. The loop is counted with `m.size`, not run until `NULL`: in `appendList(&x, x)` the list grows while it is being read, and a loop until `NULL` would never end; the counted loop stops after the original $|m|$ items (bonus).

**Listing 13.14:** appendList.

```
// Appends a copy of every item of m to the end of l (without append).
// m is a copy of the caller's struct, but its nodes are the caller's nodes,
// so we must create NEW nodes: otherwise destroying m would free l's nodes.
void appendList(list *l, list m) {
    node *src = m.head;

```

```
    for (int k = 0; k < m.size; k++) {        // count; do not stop at NULL
        node *copy = (node *)malloc(sizeof(node));
        copy->item = src->item;
        copy->next = NULL;
        copy->prev = l->tail;

        if (l->tail == NULL) l->head = copy;  // l was empty
        else l->tail->next = copy;
        l->tail = copy;
        l->size++;

        src = src->next;
    }
}
```

```
// The test program around appendList (reconstructed from the sample output).
int main() {
    int n, item;
    list l, m;
    while (scanf("%d", &n) == 1) {
        init(&l);
        init(&m);
        for (int i = 0; i < n; i++) { scanf("%d", &item); append(&l, item); }
        scanf("%d", &n);
        for (int i = 0; i < n; i++) { scanf("%d", &item); append(&m, item); }
        printf("list l:\n");
        display(&l);
        printf("list m:\n");
        display(&m);
        appendList(&l, m);
        destroy(&m);              // l must not be affected
        display(&l);
        destroy(&l);
        printf("----\n");
    }
    return 0;
}
```

Complexity. $\Theta(|m|)$ time.

Common mistakes. Linking `m`'s nodes directly (shallow copy); forgetting the empty `l` (`head` must be set); forgetting `prev`; empty `m` must leave `l` unchanged.

## Solution to Problem 15.

Idea. `*h` is the pointer that leads to the rest of the list: `head` at the top level, then `head->next`, and so on. If it is `NULL`, the new node is stored *there*: this changes `head` for an empty list and the last `next` field otherwise. Else recurse on the address of the next field. The loop version walks the same pointer-to-pointer.

**Listing 13.15:** append with a double pointer, recursive and iterative.

```
// Appends x at the end of a list that has no tail pointer.
// h is the address of the pointer that points to the rest of the list.
void append(snode **h, int x) {
```

```
    if (*h == NULL) {               // reached the end: put the node here
        snode *n = (snode *)malloc(sizeof(snode));
        n->item = x;
        n->next = NULL;
        *h = n;
    } else {                        // otherwise append to the rest
        append(&((*h)->next), x);
    }
}

// Bonus: the same with a loop. h walks from &head to the last next field.
void appendLoop(snode **h, int x) {
    while (*h != NULL) h = &((*h)->next);
    *h = (snode *)malloc(sizeof(snode));
    (*h)->item = x;
    (*h)->next = NULL;
}
```

Complexity. $\Theta(n)$ time. The recursion depth is $n + 1$, so the recursive version uses $\Theta(n)$ stack and can overflow for very long lists; the loop uses $O(1)$.

Common mistakes. Passing `l.head` instead of `&l.head`; writing `append((*h)->next, x)` (type error: that is a `snode *`); allocating the node before the recursion (one node per level would leak).

## Solution to Problem 16.

Idea. Three pointers: `prev` (the reversed part), `cur` (the next node to turn around) and `nxt` (saved before `cur->next` is overwritten). See Question 19 of Section A for the invariant. Afterwards the old head is the tail.

**Listing 13.16:** Reversing an SLL, iteratively and recursively.

```
// Reverses the list in place by changing the links.
void reverseList(sll *l) {
    snode *prev = NULL;
    snode *cur = l->head;

    while (cur != NULL) {
        snode *nxt = cur->next;     // remember the rest
        cur->next = prev;           // point backwards
        prev = cur;
        cur = nxt;
    }
    l->tail = l->head;              // the old head is the new tail
    l->head = prev;
}

// Bonus: recursive version. Returns the new head of the reversed list.
snode *reverseRec(snode *h) {
    if (h == NULL || h->next == NULL) return h;
    snode *newHead = reverseRec(h->next);
    h->next->next = h;              // the next node now points back to h
    h->next = NULL;
    return newHead;
```

```
}
```

Complexity. $\Theta(n)$ time; $O(1)$ space iteratively, $\Theta(n)$ stack recursively.

Common mistakes. Losing the rest of the list by not saving `nxt`; forgetting to update `tail`; in the recursive version, forgetting `h->next = NULL`, which leaves a two-node cycle at the end.

## Solution to Problem 17.

Idea. Middle: slow and fast pointers (Question 18 of Section A). $k$-th from the end: send `lead` $k$ nodes ahead; then move both until `lead` falls off the end; `lag` is $k$ nodes behind the end. If `lead` falls off during the head start, $k > n$.

**Listing 13.17:** Middle and $k$-th from the end.

```c
// Returns the middle item (the second one if the length is even).
// slow moves 1 node per step, fast moves 2, so slow is halfway when fast ends.
int middleItem(sll *l) {
    snode *slow = l->head;
    snode *fast = l->head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->item;
}

// Stores the k-th item from the end (k = 1 is the last) in *out.
// Returns 1 on success, 0 if k < 1 or k > the length.
int kthFromEnd(sll *l, int k, int *out) {
    if (k < 1) return 0;
    snode *lead = l->head;
    for (int i = 0; i < k; i++) {    // lead goes k nodes ahead
        if (lead == NULL) return 0;
        lead = lead->next;
    }
    snode *lag = l->head;
    while (lead != NULL) {           // both move until lead falls off
        lead = lead->next;
        lag = lag->next;
    }
    *out = lag->item;
    return 1;
}
```

Complexity. $\Theta(n)$ time, one traversal, $O(1)$ space. For the first middle, use the condition `fast->next != NULL && fast->next->next != NULL` (the fast pointer stops one step earlier).

Common mistakes. Off-by-one in the head start (moving $k - 1$ instead of $k$); not checking $k < 1$; dereferencing `lead` after it became `NULL`.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 18.

Idea. Splicing changes three pointers: the anchor's `next` (or `head`), the `next` of `m`'s tail, and `l`'s `tail` if the splice is at the end. `splice` walks to the node at index `pos` − 1 and calls `spliceAfter`.

**Listing 13.18:** Splicing.

```
// Moves all nodes of m into l, right after node p (p == NULL: at the front).
// No node is created or freed; only pointers change. m becomes empty.
void spliceAfter(sll *l, snode *p, sll *m) {
    if (m->head == NULL) return;                // nothing to move

    snode *after = (p == NULL) ? l->head : p->next;
    if (p == NULL) l->head = m->head;           // spliced at the front
    else p->next = m->head;
    m->tail->next = after;
    if (after == NULL) l->tail = m->tail;       // spliced at the end

    l->size = l->size + m->size;
    m->head = NULL;
    m->tail = NULL;
    m->size = 0;
}

// Same, but pos nodes of l stay in front (0 <= pos <= size).
// Returns 0 if pos is invalid.
int splice(sll *l, int pos, sll *m) {
    if (pos < 0 || pos > l->size) return 0;
    snode *p = NULL;                            // the node at index pos - 1
    if (pos > 0) {
        p = l->head;
        for (int i = 1; i < pos; i++) p = p->next;
    }
    spliceAfter(l, p, m);
    return 1;
}
```

Complexity. `spliceAfter`: $\Theta(1)$. `splice`: $\Theta(\texttt{pos})$. An array list would have to copy all $|m|$ items (bonus), $\Theta(|m|)$ even at the end.

Common mistakes. Forgetting `l->tail` when splicing at the end; not emptying `m`, so that both lists own the nodes (a later destroy frees them twice); an empty `m` has `m->tail == NULL`, so it must be handled first.

## Solution to Problem 19.

Idea. Build the merged chain with `head`/`last`, taking the smaller front node from `x` or `y` each time (`<=` keeps `a`'s node first on ties) and setting its `prev` to `last`. Nothing is allocated; at the end `a` receives the chain and `b` is emptied.

**Listing 13.19:** Merging two sorted DLLs.

```
// Merges the sorted lists a and b by relinking nodes. a gets all nodes, b is
// left empty. On equal items the node of a goes first.
```

```c
void mergeSorted(list *a, list *b) {
    node *x = a->head;
    node *y = b->head;
    node *head = NULL;              // head and last build the merged chain
    node *last = NULL;

    while (x != NULL || y != NULL) {
        node *take;
        if (y == NULL || (x != NULL && x->item <= y->item)) {
            take = x;
            x = x->next;
        } else {
            take = y;
            y = y->next;
        }
        take->prev = last;
        take->next = NULL;
        if (last == NULL) head = take;
        else last->next = take;
        last = take;
    }
    a->head = head;
    a->tail = last;
    a->size = a->size + b->size;
    b->head = NULL;
    b->tail = NULL;
    b->size = 0;
}
```

Complexity. $\Theta(|a| + |b|)$ time, $O(1)$ space. For the bonus, when one list is exhausted, link the remainder of the other with one `next` and one `prev` assignment and take its tail.

Common mistakes. Forgetting the `prev` links (the forward display is right, the backward one is not); not resetting `b`; taking `y` on ties.

## Solution to Problem 20.

Idea. Two pointers from both ends move toward each other. They stop when they meet (odd length) or cross (even length: `left->prev == right`). An empty list has `left == right == NULL` and skips the loop.

**Listing 13.20:** Palindromic DLL.

```c
// Returns 1 if the list reads the same forwards and backwards.
int isPalindrome(list *l) {
    node *left = l->head;
    node *right = l->tail;

    // stop when the two pointers meet or cross (an empty list skips the loop)
    while (left != right && left->prev != right) {
        if (left->item != right->item) return 0;
        left = left->next;
        right = right->prev;
    }
    return 1;
```

Juevesano D. 2026 All rights reserved. 

```
}
```

Complexity. $\Theta(n)$ time, at most $\lfloor n/2 \rfloor$ comparisons, $O(1)$ space.

Common mistakes. With the condition `left != right` alone, on an even length the pointers cross without ever being equal, and the loop runs until one of them is `NULL`: a crash. Using a stack contradicts the permitted operations.

## Solution to Problem 21.

Idea. In every node swap `next` and `prev`; then swap `head` and `tail`. The walk must use the *saved* old `next`.

**Listing 13.21:** Reversing a DLL.

```c
// Reverses the list in place: every node swaps its next and prev.
void reverseDLL(list *l) {
    node *cur = l->head;
    while (cur != NULL) {
        node *nxt = cur->next;       // read before swapping
        cur->next = cur->prev;
        cur->prev = nxt;
        cur = nxt;
    }
    node *tmp = l->head;             // head and tail trade places
    l->head = l->tail;
    l->tail = tmp;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Advancing with `cur = cur->next` after the swap (that is the old `prev`: the loop goes backwards and stops at once).

## Solution to Problem 22.

Idea. Compare each node with its successor; if equal, unlink and free the successor and *stay* on the current node (the next one may be equal too).

**Listing 13.22:** Removing duplicates from a sorted DLL.

```c
// Keeps one node for each value of a sorted list.
void removeDuplicates(list *l) {
    node *cur = l->head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->next->item == cur->item) {
            node *dup = cur->next;           // unlink the duplicate
            cur->next = dup->next;
            if (dup->next != NULL) dup->next->prev = cur;
            else l->tail = cur;
            free(dup);
            l->size--;
        } else {
            cur = cur->next;                 // advance only when no removal
        }
```

```
    }
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Advancing after a removal, which skips the third copy of `2 2 2`; forgetting `tail` when the last node is removed.

## Solution to Problem 23.

Idea. Find the first node greater than $x$ (`cur`); the new node goes between `cur->prev` and `cur`. If `cur` is `NULL`, the node becomes the new tail; if its predecessor is `NULL`, it becomes the new head. Both can happen (empty list).

**Listing 13.23:** Insertion into a sorted DLL.

```
// Inserts x into the sorted list, after any equal items.
void insertSorted(list *l, int x) {
    node *n = (node *)malloc(sizeof(node));
    n->item = x;

    node *cur = l->head;
    while (cur != NULL && cur->item <= x) {   // first node greater than x
        cur = cur->next;
    }
    // n goes between cur->prev and cur
    n->next = cur;
    if (cur != NULL) {
        n->prev = cur->prev;
        cur->prev = n;
    } else {                                  // x is the new maximum
        n->prev = l->tail;
        l->tail = n;
    }
    if (n->prev != NULL) n->prev->next = n;
    else l->head = n;                         // x is the new minimum
    l->size++;
}
```

Complexity. $\Theta(n)$ time in the worst case. Bonus: if $x \geq$ `tail->item`, link after the tail directly, $O(1)$.

Common mistakes. Stopping at the first node $\geq x$ (equal items then end up after $x$, which is still sorted but violates the statement); forgetting `prev` of the old neighbour.

## Solution to Problem 24.

Idea. The middle index $\lfloor (\texttt{size} - 1)/2 \rfloor$ moves right by one when `size` goes from even to odd at the *back* (a push at the back when the size was even) or when an item leaves at the *front* with even size. It must move *left* when an item is pushed at the front with odd size or popped at the back with odd size: an SLL cannot step back, so `mid` is recomputed by a walk from the head.

**Listing 13.24:** An SLL with a middle pointer.

Juevesano D. 2026 All rights reserved. 

```c
typedef struct mnode {
    int item;
    struct mnode *next;
} mnode;

typedef struct {
    mnode *head;
    mnode *mid;          // the node at index (size - 1) / 2
    mnode *tail;
    int size;
} mlist;

void minit(mlist *l) {
    l->head = NULL;
    l->mid = NULL;
    l->tail = NULL;
    l->size = 0;
}

// O(1)
int middle(mlist *l) {
    return l->mid->item;
}

// O(1): the new node is after mid; mid moves right when size was even.
void mPushBack(mlist *l, int x) {
    mnode *n = (mnode *)malloc(sizeof(mnode));
    n->item = x;
    n->next = NULL;
    if (l->size == 0) {
        l->head = n;
        l->mid = n;
    } else {
        l->tail->next = n;
        if (l->size % 2 == 0) l->mid = l->mid->next;
    }
    l->tail = n;
    l->size++;
}

// O(1) if size is even; O(n) if odd, because mid must move one node LEFT.
void mPushFront(mlist *l, int x) {
    mnode *n = (mnode *)malloc(sizeof(mnode));
    n->item = x;
    n->next = l->head;
    l->head = n;
    if (l->size == 0) {
        l->mid = n;
        l->tail = n;
    } else if (l->size % 2 == 1) {
        l->mid = l->head;                    // walk from the head
        for (int i = 0; i < l->size / 2; i++) l->mid = l->mid->next;
    }
    l->size++;
```

Juevesano D. 2026 All rights reserved. 

```c
}

// O(1): mid moves right when size was even.
int mPopFront(mlist *l, int *x) {
    if (l->size == 0) return 0;
    mnode *old = l->head;
    *x = old->item;
    l->head = old->next;
    if (l->size == 1) {
        l->mid = NULL;
        l->tail = NULL;
    } else if (l->size % 2 == 0) {
        l->mid = l->mid->next;
    }
    free(old);
    l->size--;
    return 1;
}

// O(n): the new tail is the node before the old tail, which needs a walk.
// If size was odd, mid also moves one node LEFT.
int mPopBack(mlist *l, int *x) {
    if (l->size == 0) return 0;
    *x = l->tail->item;
    if (l->size == 1) {
        free(l->head);
        l->head = NULL;
        l->mid = NULL;
        l->tail = NULL;
    } else {
        mnode *prev = l->head;              // will be the node before tail
        while (prev->next != l->tail) prev = prev->next;
        free(l->tail);
        prev->next = NULL;
        l->tail = prev;
        if (l->size % 2 == 1) {             // recompute mid from the head
            l->mid = l->head;
            for (int i = 0; i < (l->size - 2) / 2; i++) l->mid = l->mid->next;
        }
    }
    l->size--;
    return 1;
}
```

Complexity. `middle`, `mPushBack`, `mPopFront`: $\Theta(1)$. `mPushFront`: $\Theta(1)$ for even size, $\Theta(n)$ for odd. `mPopBack`: $\Theta(n)$ (it needs the node before the tail anyway). With a `prev` pointer every move of `mid` and the new tail are one step away, so all are $\Theta(1)$ (bonus). Tracing 16 shows `mid` after each operation.

Common mistakes. Moving `mid` in the wrong direction or on the wrong parity; forgetting the one-node case, where all three pointers become `NULL`.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 25.

Idea. Reduce $k$ modulo $n$. Close the list into a circle (`tail->next = head`), then cut it after the $k$-th node: the node after the cut is the new head, the $k$-th node the new tail.

**Listing 13.25:** Rotating a list to the left.

```
// Rotates the list left by k places: [1 2 3 4 5], k = 2 -> [3 4 5 1 2].
void rotateLeft(sll *l, int k) {
    if (l->size == 0) return;
    k = k % l->size;
    if (k == 0) return;

    snode *newTail = l->head;               // the k-th node
    for (int i = 1; i < k; i++) newTail = newTail->next;

    l->tail->next = l->head;                // old tail points to old head
    l->head = newTail->next;                // the node after becomes head
    newTail->next = NULL;
    l->tail = newTail;
}
```

Complexity. $\Theta(n)$ time; no allocation.

Common mistakes. Not reducing $k$ ($10^9$ steps); not handling $k$ mod $n = 0$, which would cut after the $n$-th node and make the head `NULL`; forgetting to update `tail`.

## Solution to Problem 26.

Idea. Two chains with dummy heads: one for the items $< x$, one for the others. Each node is appended to its chain in order; at the end the first chain is followed by the second, and the second is terminated with `NULL`. The dummies are local `snode` variables, so nothing is allocated.

**Listing 13.26:** Partitioning a list.

```
// Moves nodes with item < x in front of the others; order inside each group
// is kept. Two chains are built and joined; no node is created.
void partitionList(sll *l, int x) {
    snode smallDummy;               // dummy heads remove the empty cases
    snode bigDummy;
    snode *small = &smallDummy;     // last node of each chain
    snode *big = &bigDummy;
    smallDummy.next = NULL;
    bigDummy.next = NULL;

    snode *cur = l->head;
    while (cur != NULL) {
        if (cur->item < x) {
            small->next = cur;
            small = cur;
        } else {
            big->next = cur;
            big = cur;
        }
```

```
        cur = cur->next;
    }
    big->next = NULL;                // end of the second chain
    small->next = bigDummy.next;     // join the two chains

    l->head = smallDummy.next;
    if (big != &bigDummy) l->tail = big;
    else if (small != &smallDummy) l->tail = small;
    else l->tail = NULL;
}
```

Complexity. $\Theta(n)$ time, one pass, $O(1)$ space.

Common mistakes. Forgetting `big->next = NULL`, which can leave a cycle (the last big node may still point to a small node); updating `tail` wrongly when one chain is empty.

## Solution to Problem 27.

Idea. The lead/lag technique of Problem 17, started from a dummy node in front of the head, stops `lag` on the node *before* the target. Then one assignment unlinks the target, whether it is the head or not.

**Listing 13.27:** Removing the $n$-th node from the end.

```
// Removes the n-th node from the end (n = 1 is the last) in one pass.
// Returns 1 on success, 0 if n < 1 or n > the length.
int removeNthFromEnd(sll *l, int n) {
    if (n < 1) return 0;
    snode dummy;                     // a dummy in front of the head means
    dummy.next = l->head;            // the head needs no special case

    snode *lead = &dummy;
    for (int i = 0; i < n; i++) {    // lead goes n nodes ahead
        if (lead->next == NULL) return 0;
        lead = lead->next;
    }
    snode *lag = &dummy;
    while (lead->next != NULL) {     // lag stops just before the target
        lead = lead->next;
        lag = lag->next;
    }
    snode *target = lag->next;
    int wasTail = (target == l->tail);
    lag->next = target->next;
    free(target);
    l->size--;

    l->head = dummy.next;
    if (l->head == NULL) l->tail = NULL;
    else if (wasTail) l->tail = lag;
    return 1;
}
```

Complexity. $\Theta(|l|)$ time, one pass.

Juevesano D. 2026 All rights reserved. 

Common mistakes. Stopping `lag` on the target itself (an SLL cannot unlink a node without its predecessor); forgetting `tail` when the last node is removed; not freeing the node.

## Solution to Problem 28.

Idea. With `prev` before the pair `a`, `b`: `a->next = b->next`, `b->next = a`, `prev->next = b`; then `a` is the node before the next pair. A dummy node makes the first pair like the others.

**Listing 13.28:** Swapping nodes in pairs.

```
// Swaps every two adjacent nodes: [1 2 3 4] -> [2 1 4 3].
// The nodes move; the items stay inside their nodes.
void swapPairs(sll *l) {
    snode dummy;
    dummy.next = l->head;
    snode *prev = &dummy;            // the node before the pair

    while (prev->next != NULL && prev->next->next != NULL) {
        snode *a = prev->next;
        snode *b = a->next;
        a->next = b->next;
        b->next = a;
        prev->next = b;
        if (a->next == NULL) l->tail = a;   // the pair was at the end
        prev = a;                    // a is now the node before the next pair
    }
    l->head = dummy.next;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. The order of the three assignments (doing `b->next = a` first loses the rest of the list); swapping the *items*, which the statement forbids; the tail is `a` when the last pair is swapped and unchanged when a single node is left over.

## Solution to Problem 29.

Idea. Floyd's tortoise and hare. If there is no cycle, `fast` reaches `NULL`. If there is one, both pointers end up inside it, and the distance between them shrinks by one per step, so they meet. For the start: if the tail before the cycle has length $\mu$, the cycle length $\lambda$, and they meet $t$ nodes into the cycle, then $2(\mu + t) \equiv \mu + t \pmod{\lambda}$, so $\mu \equiv -t \pmod{\lambda}$: walking $\mu$ more steps from the meeting point reaches the start, and so does walking $\mu$ steps from the head.

**Listing 13.29:** Cycle detection and the start of the cycle.

```
// Returns 1 if the list starting at head has a cycle, 0 otherwise.
// Floyd: if fast ever meets slow, there is a cycle.
int hasCycle(snode *head) {
    snode *slow = head;
    snode *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
```

Juevesano D. 2026 All rights reserved. 

```
        if (slow == fast) return 1;
    }
    return 0;
}

// Bonus: returns the first node of the cycle, or NULL if there is none.
snode *cycleStart(snode *head) {
    snode *slow = head;
    snode *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            snode *p = head;            // walk from the head and from the meeting
            while (p != slow) {         // point; they meet at the cycle start
                p = p->next;
                slow = slow->next;
            }
            return p;
        }
    }
    return NULL;
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Testing `slow == fast` before the first move (they start equal); checking only `fast != NULL` before `fast->next->next`.

## Solution to Problem 30.

Idea. Two chains threaded through the list: `odd` takes `even->next`, `even` takes `odd->next`. At the end the odd chain is followed by the even chain, whose last node is the new tail.

**Listing 13.30:** Grouping odd and even positions.

```
// Puts the nodes at odd positions (1st, 3rd, ...) before those at even
// positions: [1 2 3 4 5] -> [1 3 5 2 4]. Positions, not values.
void oddEvenList(sll *l) {
    if (l->head == NULL || l->head->next == NULL) return;

    snode *odd = l->head;               // last node of the odd chain
    snode *even = l->head->next;        // last node of the even chain
    snode *evenHead = even;
    while (even->next != NULL) {
        odd->next = even->next;         // odd chain takes the next odd node
        odd = odd->next;
        even->next = odd->next;         // even chain takes the next even node
        if (even->next != NULL) even = even->next;
    }
    odd->next = evenHead;               // odd chain, then even chain
    l->tail = even;                     // the even chain ends the whole list
}
```

Complexity. $\Theta(n)$ time, $O(1)$ space.

Juevesano D. 2026 All rights reserved. 

Common mistakes. Grouping by *value* parity; losing the head of the even chain (save `evenHead`); leaving the even chain unterminated.

### Solution to Problem 31.

Idea. Schoolbook addition from the least significant digit: add the two digits (missing digits count as 0) and the carry; push `sum % 10`; carry `sum / 10`. Continue while either list has digits *or* the carry is non-zero.

**Listing 13.31:** Adding numbers stored in lists.

```
// Adds two numbers stored as lists of digits, least significant digit first.
// [2 4 3] + [5 6 4] -> [7 0 8]   (342 + 465 = 807). The sum goes to *out.
void addNumbers(sll *a, sll *b, sll *out) {
    snode *x = a->head;
    snode *y = b->head;
    int carry = 0;
    sinit(out);

    while (x != NULL || y != NULL || carry > 0) {
        int sum = carry;
        if (x != NULL) {
            sum = sum + x->item;
            x = x->next;
        }
        if (y != NULL) {
            sum = sum + y->item;
            y = y->next;
        }
        pushBack(out, sum % 10);
        carry = sum / 10;
    }
}
```

Complexity. $\Theta(\max(|a|, |b|))$ time. For the bonus (most significant digit first), reverse both lists (Problem 16), add, and reverse the result; or push the digits on two stacks.

Common mistakes. Forgetting the final carry (99 + 1); stopping at the end of the shorter list; converting to `int` or `long`, which overflows.

## 13.3 Stacks

### Solution to Problem 32.

The solution is Listing 4.2 of Chapter 4.

Idea. Push openers; a closer must match the most recent unmatched opener, which is the top. Fail on a mismatch, on a closer with an empty stack, or on openers left at the end.

Complexity. $\Theta(n)$ time, $O(n)$ stack.

Common mistakes. Returning 1 at the end without checking that the stack is empty (`"(("`); popping without checking the return value, so that `"))"` compares against a stale variable; using counters, which accept `"([)]"`.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 33.

The solution is Listing 4.3 of Chapter 4; the table there traces `2*(3+4)-5`.

Idea. Shunting-yard for the conversion; one operand stack for the evaluation, where the *first* pop is the right operand.

Complexity. Both $\Theta(n)$: every token is pushed and popped at most once.

Common mistakes. Popping operators of equal precedence too late (`>` instead of `>=`: then `8-3-2` evaluates to 7); forgetting to discard the `(`; swapping the operands of `-` and `/` (Tracing 13). For the bonus: `^` has the highest precedence and is right-associative, so an incoming `^` pops only operators of *strictly* higher precedence.

## Solution to Problem 34.

Idea. Keep a second stack `mins` in step with `all`: its top is always the minimum of the whole stack. Each push pushes $\min(x, \text{top of mins})$; each pop pops both.

**Listing 13.32:** A min stack.

```
// A stack that also returns its minimum in O(1).
// "mins" holds, for every level of "all", the smallest value at or below it.
typedef struct {
    stack all;
    stack mins;
} minStack;

void msInit(minStack *m) {
    stInit(&m->all);
    stInit(&m->mins);
}

void msPush(minStack *m, int x) {
    int smallest;
    stPush(&m->all, x);
    if (stTop(&m->mins, &smallest) && smallest < x) {
        stPush(&m->mins, smallest);      // the old minimum still wins
    } else {
        stPush(&m->mins, x);             // x is the new minimum
    }
}

// Removes the top into *x. Returns 0 if empty.
int msPop(minStack *m, int *x) {
    int dummy;
    if (!stPop(&m->all, x)) return 0;
    stPop(&m->mins, &dummy);             // keep the two stacks in step
    return 1;
}

// Copies the minimum into *x. Returns 0 if empty.
int msMin(minStack *m, int *x) {
    return stTop(&m->mins, x);
}
```

Juevesano D. 2026 All rights reserved. 

Complexity. Every operation $\Theta(1)$; $2n$ cells. In the bonus version `mins` is pushed only when $x \leq$ its top and popped only when the popped item equals its top; the test must be $\leq$ so that a repeated minimum (pushes 3, 3, pop) is still on `mins` after one of its copies leaves.

Common mistakes. A single variable for the minimum (it cannot be restored after a pop); popping `all` without popping `mins`.

### Solution to Problem 35.

The solution is Listing 4.4 of Chapter 4, with Theorem 4.5 for the running time.

Idea. The stack holds the indices still waiting for a greater value; `a[i]` answers all smaller waiting values, which are on top.

Complexity. $\Theta(n)$: every index is pushed and popped at most once. Circular bonus: run the loop for $i = 0..2n-1$ on `a[i % n]` and push only in the first round.

Common mistakes. Storing values instead of indices (then the answer cannot be written to the right cell); using <=, which treats an equal value as greater (`[1,1]` would give `[1,-1]`).

### Solution to Problem 36.

Idea. The same monotonic stack: the waiting days, with non-increasing temperatures. When day $i$ is strictly warmer than the day $j$ on top, day $j$ waited $i - j$ days.

**Listing 13.33:** Daily temperatures.

```
// wait[i] = number of days after day i until a strictly warmer day,
// or 0 if there is none.   [73 74 75 71 69 72 76 73] -> [1 1 4 2 1 1 0 0]
void dailyTemperatures(int t[], int n, int wait[]) {
    stack cold;                     // days still waiting, temperatures
    stInit(&cold);                  // non-increasing from bottom to top

    for (int i = 0; i < n; i++) {
        int j;
        while (stTop(&cold, &j) && t[j] < t[i]) {
            wait[j] = i - j;        // day i is the first warmer day for j
            stPop(&cold, &j);
        }
        stPush(&cold, i);
    }
    int j;
    while (stPop(&cold, &j)) wait[j] = 0;   // never got warmer
}
```

Complexity. $\Theta(n)$ time.

Common mistakes. Using <= (equal temperatures are not warmer); forgetting to write 0 for the days left on the stack (here written explicitly, since `wait` is not initialised).

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 37.

Idea. The online "previous greater" problem. Keep pairs (price, span) on two stacks in step, prices strictly decreasing from bottom to top. A new price absorbs every pair with a price $\leq$ it, together with the days that pair already covered.

**Listing 13.34:** Online stock span.

```
// The span of today's price is the number of consecutive days, ending
// today, on which the price was <= today's price.
// Two stacks in step: a price, and how many days that price "covers".
typedef struct {
    stack prices;
    stack spans;
} stockSpanner;

void ssInit(stockSpanner *s) {
    stInit(&s->prices);
    stInit(&s->spans);
}

int ssNext(stockSpanner *s, int price) {
    int span = 1;                   // today itself
    int p, d;
    while (stTop(&s->prices, &p) && p <= price) {
        stPop(&s->prices, &p);      // a cheaper day is absorbed, together
        stPop(&s->spans, &d);       // with all the days it already covered
        span = span + d;
    }
    stPush(&s->prices, price);
    stPush(&s->spans, span);
    return span;
}
```

Complexity. $\Theta(1)$ amortised per call: every pair is pushed once and popped at most once (Tracing 15).

Common mistakes. Storing every price and scanning backwards ($\Theta(n)$ per call, $\Theta(n^2)$ in total for decreasing prices); using < (an equal price must be absorbed).

## Solution to Problem 38.

Idea. The stack holds the survivors so far, left to right. Only a left-mover arriving at a right-mover on top can collide; it destroys smaller right-movers one after another, dies with an equal one, or dies against a larger one. Left-movers with nothing to their left that moves right survive. At the end the stack, read from bottom to top, is the answer; popping it fills `out` from the right.

**Listing 13.35:** Asteroid collision.

```
// a[i] > 0 moves right, a[i] < 0 moves left; |a[i]| is the size. When two
// meet, the smaller explodes (both if equal). Writes the survivors, left to
// right, into out and returns how many there are.
int asteroidCollision(int a[], int n, int out[]) {
    stack st;                       // survivors so far, left to right
```

```
    stInit(&st);

    for (int i = 0; i < n; i++) {
        int x = a[i];
        int alive = 1;
        int top;
        // only a right-mover on the stack and a left-mover x can meet
        while (alive && x < 0 && stTop(&st, &top) && top > 0) {
            if (top < -x) {
                stPop(&st, &top);    // the top explodes, x keeps going
            } else if (top == -x) {
                stPop(&st, &top);    // both explode
                alive = 0;
            } else {
                alive = 0;           // x explodes
            }
        }
        if (alive) stPush(&st, x);
    }

    int m = stSize(&st);             // the stack holds the answer, last on top
    for (int k = m - 1; k >= 0; k--) stPop(&st, &out[k]);
    return m;
}
```

Complexity. $\Theta(n)$: each asteroid is pushed and popped at most once.

Common mistakes. Letting a right-mover collide with a left-mover on top (`[-2,-1,1,2]`: they move apart); stopping after one collision; returning the survivors in reverse order.

## Solution to Problem 39.

Idea. At each `[` remember, on two stacks, the repeat count and the position in `out` where the bracketed part starts. Characters are written straight to `out`. At `]` the part `out[start..len-1]` is already there once; append it $k-1$ more times. Nested brackets work because the inner part is expanded before the outer `]` copies it.

**Listing 13.36:** Decode string.

```
// Decodes k[...] (repeat the bracketed part k times), with nesting:
// "3[a]2[bc]" -> "aaabcbc", "3[a2[c]]" -> "accaccacc".
// The answer is built directly in out; the stacks remember, for every open
// bracket, its repeat count and where its part starts in out.
void decodeString(char s[], char out[]) {
    stack counts;
    stack starts;
    int len = 0;                     // current length of out
    int k = 0;                       // number being read
    stInit(&counts);
    stInit(&starts);

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c >= '0' && c <= '9') {
            k = k * 10 + (c - '0');
```

```
        } else if (c == '[') {
            stPush(&counts, k);
            stPush(&starts, len);
            k = 0;
        } else if (c == ']') {
            int times, start;
            stPop(&counts, &times);
            stPop(&starts, &start);
            int partLen = len - start;          // out[start..len-1] is the part
            for (int t = 1; t < times; t++) {   // it is already there once
                for (int j = 0; j < partLen; j++) {
                    out[len] = out[start + j];
                    len++;
                }
            }
        } else {
            out[len] = c;
            len++;
        }
    }
    out[len] = '\0';
}
```

Complexity. Linear in the length of the output (every output character is written once).

Common mistakes. Reading only one digit of the count (`10[a]`); not resetting the count after `[`; copying the part $k$ times instead of $k-1$.

## Solution to Problem 40.

Idea. The stack holds the letters that survived so far. A letter equal to the top cancels with it; otherwise it is pushed. Cancellations can cascade (`abba`) because after a pop the new top is the next older survivor.

**Listing 13.37:** Removing adjacent duplicates.

```
// Repeatedly removes two equal neighbouring letters. "abbaca" -> "ca".
// Result is written to out.
void removeAdjacent(char s[], char out[]) {
    stack st;
    int top;
    stInit(&st);

    for (int i = 0; s[i] != '\0'; i++) {
        if (stTop(&st, &top) && top == s[i]) {
            stPop(&st, &top);        // equal to the top: both vanish
        } else {
            stPush(&st, s[i]);
        }
    }
    int k = stSize(&st);             // the stack holds the answer
    out[k] = '\0';
    while (stPop(&st, &top)) out[--k] = top;
}
```

Juevesano D. 2026 All rights reserved. 

Complexity. $\Theta(n)$.

Common mistakes. Removing only one pass of pairs (`abbaca` becomes `aaca`, not `ca`); copying the stack to `out` in reverse order.

## Solution to Problem 41.

Idea. Insertion sort between two stacks. Pop $x$ from `s`; move back to `s` every item of `r` larger than $x$; push $x$ on `r`. Then `r` is sorted with the largest on top; pouring it into `s` puts the smallest on top.

**Listing 13.38:** Sorting a stack with one extra stack.

```
// Sorts the stack so that the smallest item is on top (popping gives
// 1, 2, 3, ...). Only one extra stack is used.
void sortStack(stack *s) {
    stack r;                        // sorted, largest on top
    int x, y;
    stInit(&r);

    while (stPop(s, &x)) {
        // items in r larger than x must go back to s so that x fits below them
        while (stTop(&r, &y) && y > x) {
            stPop(&r, &y);
            stPush(s, y);
        }
        stPush(&r, x);
    }
    while (stPop(&r, &x)) stPush(s, x);   // moving back reverses the order
}
```

Complexity. $\Theta(n^2)$ in the worst case (each item may be moved back and forth $O(n)$ times), $\Theta(n)$ if `s` is already ordered suitably.

Common mistakes. Comparing with `>=`, which still sorts but moves equal items needlessly; forgetting the final pour, which leaves the largest item on top.

## Solution to Problem 42.

Idea. Simulate: push the items of `pushed` in order, and after each push pop as long as the top is the next wanted item. The sequence is valid iff every item was popped. Bonus: an item must be pushed before it is popped, and once the top is the wanted item, popping now is never worse than pushing more (which would only bury it), so the greedy rule never misses a solution (Section 4.6).

**Listing 13.39:** Validate stack sequences.

```
// pushed[] is the order in which items are pushed; may some interleaving of
// pops produce popped[]?   pushed 1 2 3 4 5: popped 4 5 3 2 1 -> 1,
// popped 4 3 5 1 2 -> 0. Both arrays have n distinct items.
int validateStackSequences(int pushed[], int popped[], int n) {
    stack st;
    int j = 0;                      // next item of popped[] to produce
    int top;
    stInit(&st);
```

```
9
10      for (int i = 0; i < n; i++) {
11          stPush(&st, pushed[i]);
12          // pop greedily while the top is the item we need next
13          while (stTop(&st, &top) && top == popped[j]) {
14              stPop(&st, &top);
15              j++;
16          }
17      }
18      return j == n;                  // every item was produced
19  }
```

Complexity. $\Theta(n)$: $n$ pushes and at most $n$ pops.

Common mistakes. Popping only once after each push (`4 5 3 2 1` needs several pops at the end); reading `popped[j]` with $j = n$ (guarded here by the empty stack after the last pop).

## 13.4 Queues

### Solution to Problem 43.

The solution is Listing 5.2 of Chapter 5; the amortised bound is Theorem 5.2, and Tracing 12 follows one run.

Idea. `in` receives new items; `out` delivers the oldest. Refill `out` from `in` only when `out` is empty: the transfer reverses the order once, which turns LIFO into FIFO.

Complexity. $\Theta(1)$ amortised per operation, $\Theta(n)$ for a single dequeue that triggers a transfer.

Common mistakes. Transferring whenever `in` is non-empty (older items in `out` get buried under newer ones); transferring back from `out` to `in` after every dequeue ($\Theta(n)$ per operation).

### Solution to Problem 44.

Idea. After enqueueing $x$, rotate the $n-1$ older items behind it (dequeue and re-enqueue each). The front of the queue is then always the most recent item, the top.

**Listing 13.40:** A stack from one queue.

```
1   // A stack built from one queue. After each push the new item is rotated to
2   // the front of the queue, so the front is always the top of the stack.
3   typedef struct {
4       queue q;
5   } queueStack;
6
7   void qsInit(queueStack *s) {
8       qInit(&s->q);
9   }
10
11  void qsPush(queueStack *s, int x) {
12      int y;
```

```
    qEnq(&s->q, x);
    for (int i = 0; i < qSize(&s->q) - 1; i++) {   // older items go behind x
        qDeq(&s->q, &y);
        qEnq(&s->q, y);
    }
}

int qsPop(queueStack *s, int *x) {
    return qDeq(&s->q, x);
}

int qsTop(queueStack *s, int *x) {
    return qFront(&s->q, x);
}

int qsEmpty(queueStack *s) {
    return qEmpty(&s->q);
}
```

Complexity. `qsPush`: $\Theta(n)$; `qsPop`, `qsTop`: $\Theta(1)$.

Common mistakes. Evaluating `qSize` after the enqueue and rotating all $n$ items, which puts $x$ back at the rear; rotating in `qsTop` as well as in `qsPush`.

## Solution to Problem 45.

The solution is Listing 5.3 of Chapter 5.

Idea. A queue is a circle: rotating $k-1$ people from the front to the rear and removing the next one is one round of counting. The bonus uses the recurrence $J(i) = (J(i-1)+k) \bmod i$ proved there.

Complexity. Simulation $\Theta(nk)$; recurrence $\Theta(n)$ time, $O(1)$ space.

Common mistakes. Rotating $k$ people instead of $k-1$; returning the 0-based position of the recurrence without adding 1; numbering from 0 in the simulation.

## Solution to Problem 46.

Idea. A stack reverses: move the first $k$ items from the queue to the stack, then back to the rear of the queue. They are reversed but now stand behind the other $n-k$ items, so rotate those $n-k$ items once around.

**Listing 13.41:** Reversing the first $k$ items of a queue.

```
// Reverses the order of the first k items of the queue; the rest stay.
// [1 2 3 4 5], k = 3 -> [3 2 1 4 5]. Assumes 0 <= k <= size.
void reverseFirstK(queue *q, int k) {
    stack st;
    int x;
    stInit(&st);

    for (int i = 0; i < k; i++) {          // the stack reverses the first k
        qDeq(q, &x);
        stPush(&st, x);
    }
```

```
    while (stPop(&st, &x)) qEnq(q, x);      // put them back at the end

    int rest = qSize(q) - k;                // read the size once
    for (int i = 0; i < rest; i++) {        // rotate the other items
        qDeq(q, &x);                        // behind the reversed ones
        qEnq(q, x);
    }
}
```

Complexity. $\Theta(n)$: $k$ items move twice, $n - k$ once.

Common mistakes. Using `qSize(q)` as the loop bound while the loop changes the queue (the solution reads it into `rest` first); rotating $n$ items instead of $n - k$.

## Solution to Problem 47.

Idea. Pings expire in the order they arrived, so the recent ones form a queue: enqueue `t`, then dequeue from the front while the front is older than $t - 3000$. The size of the queue is the answer.

**Listing 13.42:** Number of recent calls.

```
// Counts the requests in the last 3000 ms. ping(t) is called with strictly
// increasing t and returns how many pings happened in [t - 3000, t].
typedef struct {
    queue times;                    // the pings that are still recent
} recentCounter;

void rcInit(recentCounter *r) {
    qInit(&r->times);
}

int rcPing(recentCounter *r, int t) {
    int old;
    qEnq(&r->times, t);
    // the oldest pings leave first: drop them while they are too old
    while (qFront(&r->times, &old) && old < t - 3000) {
        qDeq(&r->times, &old);
    }
    return qSize(&r->times);
}
```

Complexity. $\Theta(1)$ amortised: each ping is enqueued once and dequeued at most once.

Common mistakes. Using `<=` (the window is inclusive: a ping at exactly $t - 3000$ counts); scanning every stored ping on each call.

Juevesano D. 2026 All rights reserved. 

## Solution to Problem 48.

Idea. With one stack and the queue itself, five passes. (1) Push the first half; (2) pour the stack back to the rear (the first half is now reversed at the rear); (3) rotate the second half behind it; (4) push the first half again — the double reversal restores its order on the stack, with $x_1$ on top; (5) alternately pop one item from the stack and rotate one item of the queue to the rear. The comments in the listing show the queue for [1..8] after each pass.

**Listing 13.43:** Interleaving the halves of a queue.

```
// Interleaves the first half of a queue of even length with the second:
// [1 2 3 4 5 6 7 8] -> [1 5 2 6 3 7 4 8]. Uses one stack.
void interleaveHalves(queue *q) {
    stack st;
    int x, half = qSize(q) / 2;
    stInit(&st);

    for (int i = 0; i < half; i++) {     // 1. first half onto the stack
        qDeq(q, &x);                     //    q: 5 6 7 8   st: 1 2 3 4 (top 4)
        stPush(&st, x);
    }
    while (stPop(&st, &x)) qEnq(q, x);   // 2. q: 5 6 7 8 4 3 2 1
    for (int i = 0; i < half; i++) {     // 3. q: 4 3 2 1 5 6 7 8
        qDeq(q, &x);
        qEnq(q, x);
    }
    for (int i = 0; i < half; i++) {     // 4. first half onto the stack again
        qDeq(q, &x);                     //    q: 5 6 7 8   st: 4 3 2 1 (top 1)
        stPush(&st, x);
    }
    for (int i = 0; i < half; i++) {     // 5. alternate: one from the stack,
        stPop(&st, &x);                  //    one from the front of q
        qEnq(q, x);
        qDeq(q, &x);
        qEnq(q, x);
    }
}
```

Complexity. $\Theta(n)$: a constant number of passes over $n/2$ items.

Common mistakes. Pushing the first half once only, which interleaves it in reverse (`4 5 3 6 2 7 1 8`); using a second queue, which is not permitted.

## Solution to Problem 49.

Idea. Keep a queue of the *indices* of the negative numbers in the current window, in order. When the window `a[i-k+1..i]` is complete, drop from the front the indices that have slid out; the front, if any, is the first negative of the window.

**Listing 13.44:** First negative number in every window.

```
// out[i] = the first negative number of the window a[i..i+k-1], or 0 if the
// window has none, for every window of size k. Returns n - k + 1.
// [12 -1 -7 8 -15 30 16 28], k = 3 -> [-1 -1 -7 -15 -15 0]
```

```c
int firstNegatives(int a[], int n, int k, int out[]) {
    queue neg;                      // indices of negatives, oldest first
    int front, m = 0;
    qInit(&neg);

    for (int i = 0; i < n; i++) {
        if (a[i] < 0) qEnq(&neg, i);
        if (i >= k - 1) {                       // window a[i-k+1..i] is full
            while (qFront(&neg, &front) && front < i - k + 1) {
                qDeq(&neg, &front);             // fell out of the window
            }
            if (qFront(&neg, &front)) out[m] = a[front];
            else out[m] = 0;
            m++;
        }
    }
    return m;
}
```

Complexity. $\Theta(n)$: every index is enqueued and dequeued at most once. The window scan is $\Theta(nk)$.

Common mistakes. Storing values instead of indices (then it cannot be decided when an item leaves the window); an off-by-one in the window start (`front < i - k + 1`).

## 13.5 Deques

### Solution to Problem 50.

Idea. The circular deque of Listing 6.1 with a run-time capacity. Front insertion steps `front` back with `(front - 1 + cap) % cap`; rear deletion only decreases `size`; the rear item is at `(front + size - 1) % cap`.

**Listing 13.45:** A circular deque of capacity $k$.

```c
// A deque of capacity k (1 <= k <= 1000) in a circular array.
// Item i of the deque lives at items[(front + i) % cap].
typedef struct {
    int items[1000];
    int cap;
    int front;                      // index of the first item
    int size;
} circularDeque;

void cdInit(circularDeque *d, int k) {
    d->cap = k;
    d->front = 0;
    d->size = 0;
}

int cdIsEmpty(circularDeque *d) { return d->size == 0; }
int cdIsFull(circularDeque *d) { return d->size == d->cap; }

```

Juevesano D. 2026 All rights reserved. 

```c
int cdInsertFront(circularDeque *d, int x) {
    if (cdIsFull(d)) return 0;
    d->front = (d->front - 1 + d->cap) % d->cap;   // NOT (front - 1) % cap
    d->items[d->front] = x;
    d->size++;
    return 1;
}

int cdInsertLast(circularDeque *d, int x) {
    if (cdIsFull(d)) return 0;
    d->items[(d->front + d->size) % d->cap] = x;
    d->size++;
    return 1;
}

int cdDeleteFront(circularDeque *d) {
    if (cdIsEmpty(d)) return 0;
    d->front = (d->front + 1) % d->cap;
    d->size--;
    return 1;
}

int cdDeleteLast(circularDeque *d) {
    if (cdIsEmpty(d)) return 0;
    d->size--;                       // the last cell simply stops counting
    return 1;
}

// The first / last item, or -1 if the deque is empty.
int cdGetFront(circularDeque *d) {
    if (cdIsEmpty(d)) return -1;
    return d->items[d->front];
}

int cdGetRear(circularDeque *d) {
    if (cdIsEmpty(d)) return -1;
    return d->items[(d->front + d->size - 1) % d->cap];
}
```

Complexity. Every operation $\Theta(1)$.

Common mistakes. `(front - 1) % cap`, which is $-1$ when `front` is 0 (Tracing 21); using the constant 1000 instead of `cap` in the modulo (the deque then wraps at the wrong place); testing fullness with `size == 1000`.

Juevesano D. 2026 All rights reserved. 

### Solution to Problem 51.

The solution is Listing 6.3 of Chapter 6; Tracing 25 runs it.

Idea. With the sentinel closed into a circle, every insertion is `ldLinkBetween(a, b)` for two neighbours and every removal is `ldUnlink(n)`; the four end operations differ only in the neighbours they pass.

Complexity. Every operation $\Theta(1)$; `ldFree` $\Theta(n)$.

Common mistakes. Initialising the sentinel's pointers to `NULL` instead of to itself; copying the struct (bonus: the copy's sentinel has a different address, while the nodes still point to the original sentinel, so walks from the copy never return to it); freeing a node before reading its neighbours.

### Solution to Problem 52.

Idea. Filter and lower-case the characters into a deque; then repeatedly pop one from each end and compare, until at most one is left.

**Listing 13.46:** Valid palindrome with a deque.

```
// 1 if s reads the same forwards and backwards when only its letters and
// digits are kept and upper case is treated as lower case.
// "A man, a plan, a canal: Panama" -> 1, "race a car" -> 0.
int isPalindromeText(char s[]) {
    deque d;
    int a, b;
    dqInit(&d);

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c >= 'A' && c <= 'Z') c = c - 'A' + 'a';
        if ((c >= 'a' && c <= 'z') || (c >= '0' && c <= '9')) dqPushBack(&d, c);
    }
    while (dqSize(&d) > 1) {            // compare the two ends, then drop them
        dqPopFront(&d, &a);
        dqPopBack(&d, &b);
        if (a != b) return 0;
    }
    return 1;                           // 0 or 1 character left
}
```

Complexity. $\Theta(n)$ time and $O(n)$ space. Bonus: two indices from both ends that skip non-alphanumeric characters give $O(1)$ space.

Common mistakes. Comparing upper and lower case directly; keeping spaces and punctuation; forgetting the digits (`"0P"` is not a palindrome).

Juevesano D. 2026 All rights reserved. 

### Solution to Problem 53.

The solution is Listing 6.4 of Chapter 6, with the correctness proof there; Tracing 23 follows the deque.

Idea. A deque of indices with decreasing values: expired indices leave from the front, dominated ones from the back, and the front is the maximum.

Complexity. $\Theta(n)$ time, $O(k)$ space.

Common mistakes. Storing values (expiry cannot be detected); expiring with `j < i - k` (one window too late); reading the maximum before the first window is complete.

### Solution to Problem 54.

Idea. Two pointers `left` and `right` delimit the window. For each new `right`, update a decreasing deque (front: index of the maximum) and an increasing deque (front: index of the minimum). While $\max - \min > \mathit{limit}$, advance `left` and drop the front of a deque when its index has fallen out. The window is then the longest valid one ending at `right`.

**Listing 13.47:** Longest subarray with $\max - \min \leq \mathit{limit}$.

```
// Length of the longest contiguous subarray in which max - min <= limit.
// [8 2 4 7], limit 4 -> 2 ([2 4] or [4 7]).
// Two monotonic deques of indices of the window a[left..right]:
//   maxd: values decreasing (front = index of the maximum)
//   mind: values increasing (front = index of the minimum)
int longestSubarray(int a[], int n, int limit) {
    deque maxd, mind;
    int left = 0, best = 0, j, f, g;
    dqInit(&maxd);
    dqInit(&mind);

    for (int right = 0; right < n; right++) {
        while (dqBack(&maxd, &j) && a[j] <= a[right]) dqPopBack(&maxd, &j);
        dqPushBack(&maxd, right);
        while (dqBack(&mind, &j) && a[j] >= a[right]) dqPopBack(&mind, &j);
        dqPushBack(&mind, right);

        // shrink from the left until the window is valid again
        while (dqFront(&maxd, &f) && dqFront(&mind, &g) && a[f] - a[g] > limit) {
            left++;
            if (f < left) dqPopFront(&maxd, &f);   // the front fell out
            if (g < left) dqPopFront(&mind, &g);
        }
        if (right - left + 1 > best) best = right - left + 1;
    }
    return best;
}
```

Complexity. $\Theta(n)$: each index enters and leaves each deque at most once, and `left` only increases.

Common mistakes. Recomputing the maximum and minimum of the window by a scan ($\Theta(n^2)$); moving `left` by more than one per iteration of the inner loop without checking the deques.

## Solution to Problem 55.

Idea. Split the queue into two deques, `left` (the first half) and `right` (the second half), with the invariant $|right| - |left| \in \{0, 1\}$. Then the middle is next to the boundary: `pushMiddle` goes to the back of `left` or the front of `right`, `popMiddle` takes the back of `left` (equal sizes) or the front of `right`. Front and back operations touch one deque and then `rebalance` moves at most one item across the boundary.

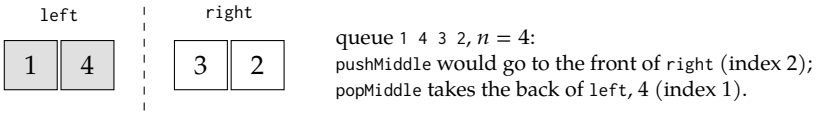



**Figure 13.2:** The front-middle-back queue after the first four test operations.

**Listing 13.48:** Front-middle-back queue.

```
// A queue that can push and pop at the front, the middle and the back.
// left holds the first half, right the second half, and always
//     size(right) == size(left)  or  size(right) == size(left) + 1.
typedef struct {
    deque left;
    deque right;
} fmbQueue;

void fmbInit(fmbQueue *q) {
    dqInit(&q->left);
    dqInit(&q->right);
}

// Restores the size invariant by moving one item across the middle.
void rebalance(fmbQueue *q) {
    int x;
    if (dqSize(&q->left) > dqSize(&q->right)) {
        dqPopBack(&q->left, &x);
        dqPushFront(&q->right, x);
    } else if (dqSize(&q->right) > dqSize(&q->left) + 1) {
        dqPopFront(&q->right, &x);
        dqPushBack(&q->left, x);
    }
}

void fmbPushFront(fmbQueue *q, int x) {
    dqPushFront(&q->left, x);
    rebalance(q);
}

void fmbPushMiddle(fmbQueue *q, int x) {
    if (dqSize(&q->left) == dqSize(&q->right)) dqPushFront(&q->right, x);
    else dqPushBack(&q->left, x);
}

void fmbPushBack(fmbQueue *q, int x) {
    dqPushBack(&q->right, x);
    rebalance(q);
```

Juevesano D. 2026 All rights reserved. 

```
}

// The pops return the removed item, or -1 if the queue is empty.
int fmbPopFront(fmbQueue *q) {
    int x;
    if (dqEmpty(&q->right)) return -1;              // then left is empty too
    if (dqEmpty(&q->left)) dqPopFront(&q->right, &x);
    else dqPopFront(&q->left, &x);
    rebalance(q);
    return x;
}

int fmbPopMiddle(fmbQueue *q) {
    int x;
    if (dqEmpty(&q->right)) return -1;
    // frontmost middle: the back of left if the halves are equal
    if (dqSize(&q->left) == dqSize(&q->right)) dqPopBack(&q->left, &x);
    else dqPopFront(&q->right, &x);
    return x;
}

int fmbPopBack(fmbQueue *q) {
    int x;
    if (dqEmpty(&q->right)) return -1;
    dqPopBack(&q->right, &x);
    rebalance(q);
    return x;
}
```

Complexity. Every operation $\Theta(1)$.

Common mistakes. Using the *back* middle when the size is even (the statement asks for the front-most); rebalancing after `pushMiddle/popMiddle` with the wrong rule — here the choice of side already preserves the invariant; forgetting the empty case (`right` is empty only if both are).

## Solution to Problem 56.

The solution is Listing 6.2 of Chapter 6; Tracing 22 counts the moves.

Idea. If $i < n/2$, move the $i$ items before position $i$ one cell to the left (after `j` steps back), otherwise move the $n - i$ items from position $i$ on one cell to the right. Removal mirrors this.

Complexity. $O(1 + \min(i, n - i))$ per operation.

Common mistakes. Moving the left block in the wrong order (when moving left, copy from the lowest index up; when moving right, from the highest index down); `j-1` without `+100`; forgetting that `adAdd` accepts $i = n$ but `adRemove` does not.

Juevesano D. 2026 All rights reserved. 

### Solution to Problem 57.

Idea. Dynamic programming: $best[i] = a[i] + \max(best[i-k..i-1])$, with $best[0] = a[0]$. The maximum over the last $k$ values is a sliding window maximum, kept by a deque of indices whose *best* values decrease from front to back.

**Listing 13.49:** Jump game VI.

```
// Jump Game VI. Start on index 0, finish on index n-1; from index i you may
// jump to any of i+1..i+k. The score is the sum of the values you land on
// (both ends included). Returns the largest possible score.
// [1 -1 -2 4 -7 3], k = 2 -> 7 (path 1 -1 4 3).
// best[i] = a[i] + max(best[i-k..i-1]); a monotonic deque of indices
// (best values decreasing) gives that window maximum in O(1).
int maxResult(int a[], int n, int k) {
    int best[10000];
    deque d;
    int j;
    dqInit(&d);

    best[0] = a[0];
    dqPushBack(&d, 0);
    for (int i = 1; i < n; i++) {
        if (dqFront(&d, &j) && j < i - k) dqPopFront(&d, &j);   // too far back
        dqFront(&d, &j);
        best[i] = a[i] + best[j];
        while (dqBack(&d, &j) && best[j] <= best[i]) dqPopBack(&d, &j);
        dqPushBack(&d, i);
    }
    return best[n - 1];
}
```

Complexity. $\Theta(n)$ time (each index enters and leaves the deque once) and $\Theta(n)$ space for `best`; the direct DP is $\Theta(nk)$.

Common mistakes. A greedy jump to the largest reachable value (it fails on the third sample test); expiring the front with `j <= i - k` (index $i-k$ is still reachable); storing $a$ instead of *best* in the comparisons.

Juevesano D. 2026 All rights reserved. 

Appendix A

# Complexity Summary

| Structure / operation | access $i$ | insert front | insert back | erase front | erase back | insert at $i$ |
|---|---|---|---|---|---|---|
| Array list | 1 | $n$ | 1 | $n$ | 1 | $n-i$ |
| Dynamic array (vector) | 1 | $n$ | $1^*$ | $n$ | $1^*$ | $n-i$ |
| SLL, head only | $i$ | 1 | $n$ | 1 | $n$ | $i$ |
| SLL, head and tail | $i$ | 1 | 1 | 1 | $n$ | $i$ |
| DLL, head and tail | $\min(i, n-i)$ | 1 | 1 | 1 | 1 | $\min(i, n-i)$ |
| Circular array (queue/deque) | 1 | 1 | 1 | 1 | 1 | — |
| ArrayDeque | 1 | 1 | 1 | 1 | 1 | $1 + \min(i, n-i)$ |
| Linked deque with sentinel | $\min(i, n-i)$ | 1 | 1 | 1 | 1 | $\min(i, n-i)$ |

**Table A.1:** Costs of the list operations ($\Theta$, worst case; $^*$ amortised, if the array grows). Insertion or removal *after a known node* is $\Theta(1)$ in linked lists (also *before* it in a DLL).

| ADT | implementation | push / pop / top (or front) |
|---|---|---|
| Stack | array (top at `items[size-1]`) | $\Theta(1)$ each |
| Stack | SLL (top = head) | $\Theta(1)$ each |
| Queue | circular array (`front`, `size`) | $\Theta(1)$ each |
| Queue | SLL (front = head, rear = tail) | $\Theta(1)$ each |
| Queue | array with front at index 0 | enqueue $\Theta(1)$, dequeue $\Theta(n)$ |
| Queue | two stacks | $\Theta(1)$ amortised, $\Theta(n)$ worst |
| Stack | one queue | push $\Theta(n)$, pop/top $\Theta(1)$ |
| Deque | circular array, DLL, ArrayDeque | $\Theta(1)$ at both ends |

**Table A.2:** Stacks, queues and deques.

***Frequently needed sums.***

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \qquad \sum_{i=0}^{k} 2^i = 2^{k+1} - 1,$$

$$\sum_{i=0}^{\lfloor \lg n \rfloor} 2^i < 2n, \qquad C_n = \frac{1}{n+1}\binom{2n}{n} : \ 1, 1, 2, 5, 14, 42.$$

Juevesano D. 2026 All rights reserved. 

| Task | Time | Extra space |
|---|---|---|
| linear search in a list | $\Theta(n)$ | 1 |
| binary search in a sorted array | $\Theta(\log n)$ | 1 |
| insert into a sorted array list | $\Theta(\log n)$ search + $\Theta(n)$ shift | 1 |
| reverse a list, in place | $\Theta(n)$ | 1 |
| middle / $k$-th from end of an SLL | $\Theta(n)$, one pass | 1 |
| merge two sorted lists (relinking) | $\Theta(n+m)$ | 1 |
| splice after a known node (with tail of the donor) | $\Theta(1)$ | 1 |
| cycle detection (Floyd) | $\Theta(n)$ | 1 |
| balanced brackets, postfix evaluation, infix→postfix | $\Theta(n)$ | $\Theta(n)$ |
| next greater / daily temperatures / stock span | $\Theta(n)$ total | $\Theta(n)$ |
| is a pop order possible? (stack permutation) | $\Theta(n)$ | $\Theta(n)$ |
| Josephus: simulation / recurrence | $\Theta(nk)$ / $\Theta(n)$ | $\Theta(n)$ / 1 |
| sliding window maximum (monotonic deque) | $\Theta(n)$ | $O(k)$ |
| longest window with $\max - \min \leq L$ | $\Theta(n)$ | $O(n)$ |
| DP over the last $k$ values (Jump Game VI) | $\Theta(n)$ | $\Theta(n)$ |

**Table A.3:** Summary of the algorithmic tasks of this volume.

Juevesano D. 2026 All rights reserved. 

Appendix B

# Pitfall Checklist

(i) **Empty, one, two.** Run every function mentally on 0, 1 and 2 items, and on a full fixed-capacity structure.

(ii) **Save before you free or overwrite.** Read `cur->next` into a variable before `free(cur)` or before reassigning `cur->next`.

(iii) **Every pointer that names an end must follow the end.** `head`, `tail`, `size`, `mid`, `front`: update all of them, in every case.

(iv) **A function that may change the head** needs the address of the head (`node **`) or must return the new head.

(v) **Shallow versus deep.** Copying a struct copies pointers, not nodes. A struct that *contains* its own sentinel must never be copied at all.

(vi) **Do not walk until `NULL` when the structure changes under you**; walk a counted number of steps. Read `qSize`/`stSize` once before a loop that changes the structure.

(vii) **Condition order.** Test the pointer before dereferencing it: `p != NULL && p->item == x`, `fast && fast->next`.

(viii) **Bounds.** Array list positions: legal insert positions are 0..`size`, legal erase/access positions 0..`size` $-$ 1. State whether positions are 0- or 1-based.

(ix) **Modulo in C.** `-1 % n` is $-1$. Step back with `(i - 1 + n) % n`. Integer division truncates toward zero: `-7/2` is $-3$.

(x) **Stack discipline.** Pop the right operand first. A transfer between two stacks reverses the order; two transfers restore it.

(xi) **Failed pops.** The course library leaves the output variable unchanged when a pop fails; the STL's `pop()` returns nothing and must not be called on an empty container.

(xii) **Monotonic structures store indices**, not values, whenever an item can expire or an answer must be written to its position. Decide < versus <= from the word "strictly".

(xiii) **Overflow.** Sums such as $n(n+1)/2$ and products overflow `int`; midpoints use `lo + (hi - lo) / 2`.

(xiv) **Half-open ranges** $[lo, hi)$ make binary search and window operations uniform.

(xv) **Recursion depth** equals the length of the list; a recursive walk over $10^5$ nodes can overflow the call stack.

(xvi) **State the complexity** of your solution in a comment, and the extra space.

Juevesano D. 2026 All rights reserved. 

Appendix C

# STL Quick Reference

Table 6.3 in Section 6.7 lists the containers. The snippets below show the idioms that replace the course library; they are not used anywhere else in this volume (the headers `iostream` and `using namespace std;` are omitted).

```
#include <stack>
#include <queue>
#include <deque>

int main() {
    stack<int> s;
    s.push(1); s.push(2);
    int t = s.top();            // read first ...
    s.pop();                    // ... then pop: pop() returns void
    queue<int> q;
    q.push(1); q.push(2);
    int f = q.front();
    q.pop();
    deque<int> d;
    d.push_front(1); d.push_back(2); d.push_front(0);
    int a = d.front(), b = d.back(), c = d[1];   // random access
    d.pop_front(); d.pop_back();
    if (!s.empty() && s.top() == 1) s.pop();     // test empty() first
    cout << t << " " << f << " " << a << " " << b << " " << c << " "
         << s.size() << " " << q.size() << " " << d.size() << endl;
    return 0;
}
```

It prints:

```
2 1 0 2 1 0 1 1
```

| Course library | STL |
|---|---|
| `stPush(&s,x)` / `stPop(&s,&x)` / `stTop(&s,&x)` | `s.push(x)` / `x=s.top(); s.pop();` / `x=s.top()` |
| `qEnq(&q,x)` / `qDeq(&q,&x)` / `qFront(&q,&x)` | `q.push(x)` / `x=q.front(); q.pop();` / `x=q.front()` |
| `dqPushFront`, `dqPushBack` | `d.push_front(x)`, `d.push_back(x)` |
| `dqPopFront`, `dqPopBack` | `d.pop_front()`, `d.pop_back()` |
| `dqFront`, `dqBack`, `dqSize`, `dqEmpty` | `d.front()`, `d.back()`, `d.size()`, `d.empty()` |
| return value 0 on an empty structure | undefined behaviour: test `empty()` first |

**Table C.1:** Translating the course library to the STL.

Juevesano D. 2026 All rights reserved. 

# Bibliography

[1] P. Morin, *Open Data Structures*, open content text (array-based lists, ArrayDeque, linked lists).

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[3] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., Pearson, 2013.

[4] R. Sedgewick, K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011 (§1.3: bags, queues and stacks).

[5] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997 (§2.2: linear lists, stacks, queues and deques; stack permutations).

[6] R. W. Floyd, Nondeterministic algorithms, *J. ACM* 14 (1967), 636–644.

[7] LeetCode problems 20, 61, 83, 141, 150, 155, 206, 232, 225, 239, 394, 641, 735, 739, 901, 933, 946, 1047, 1438, 1670, 1696 (the sources of the adapted problems).

Juevesano D. 2026 All rights reserved. 